

hw4_part1

November 20, 2025

```
[50]: ## IMPORTANT: On Colab, we expect your homework to be in the cs189 folder  
## Please contact staff if you encounter any problems with installing  
↳dependencies  
  
import sys  
IS_COLAB = 'google.colab' in sys.modules  
if IS_COLAB:  
    from google.colab import drive  
    drive.mount('/content/drive')  
    %cd /content/drive/MyDrive/cs189/hw/hw4  
    %pip install -r ./requirements.txt  
    !pip install -U kaleido plotly  
    import kaleido  
    kaleido.get_chrome_sync()  
  
import plotly.io as pio  
pio.renderers.default = pio.renderers.default + "+png"  
  
# Last updated: 2025-11-09 13:21:26
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

/content/drive/MyDrive/cs189/hw/hw4

Collecting otter_grader (from -r ./requirements.txt (line 1))

Cloning https://github.com/BerkeleyML/otter-grader-189.git (to revision master) to /tmp/pip-install-fxid33si/otter-grader_93c54e45e3ad40d7b9a068559fb1a93d

Running command git clone --filter=blob:none --quiet https://github.com/BerkeleyML/otter-grader-189.git /tmp/pip-install-fxid33si/otter-grader_93c54e45e3ad40d7b9a068559fb1a93d

Resolved https://github.com/BerkeleyML/otter-grader-189.git to commit 35e43fdf64a55f4b9222d7b531890fb7d5cb498c

Installing build dependencies ... done

Getting requirements to build wheel ... done

Preparing metadata (pyproject.toml) ... done

Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (from -r ./requirements.txt (line 2)) (4.0.0)

Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from -r ./requirements.txt (line 3)) (3.10.0)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from -r ./requirements.txt (line 4)) (2.0.2)

Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from -r ./requirements.txt (line 5)) (2.2.2)

Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from -r ./requirements.txt (line 6)) (11.3.0)

Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (from -r ./requirements.txt (line 7)) (6.5.0)

Requirement already satisfied: pygame in /usr/local/lib/python3.12/dist-packages (from -r ./requirements.txt (line 8)) (2.6.1)

Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from -r ./requirements.txt (line 9)) (1.6.1)

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from -r ./requirements.txt (line 10)) (4.67.1)

Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (from -r ./requirements.txt (line 11)) (2.8.0+cu126)

Requirement already satisfied: torchaudio in /usr/local/lib/python3.12/dist-packages (from -r ./requirements.txt (line 12)) (2.8.0+cu126)

Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (from -r ./requirements.txt (line 13)) (0.23.0+cu126)

Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (from -r ./requirements.txt (line 14)) (4.57.1)

Requirement already satisfied: click<9.0.0,>=8.1.7 in /usr/local/lib/python3.12/dist-packages (from otter_grader->-r ./requirements.txt (line 1)) (8.3.0)

Requirement already satisfied: dill>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from otter_grader->-r ./requirements.txt (line 1)) (0.3.8)

Requirement already satisfied: fica>=0.4.1 in /usr/local/lib/python3.12/dist-packages (from otter_grader->-r ./requirements.txt (line 1)) (0.4.1)

Requirement already satisfied: ipylab<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from otter_grader->-r ./requirements.txt (line 1)) (1.1.0)

Requirement already satisfied: ipython in /usr/local/lib/python3.12/dist-packages (from otter_grader->-r ./requirements.txt (line 1)) (7.34.0)

Requirement already satisfied: ipywidgets<9.0.0,>=8.1.5 in /usr/local/lib/python3.12/dist-packages (from otter_grader->-r ./requirements.txt (line 1)) (8.1.8)

Requirement already satisfied: jinja2<4.0,>=3.1 in /usr/local/lib/python3.12/dist-packages (from otter_grader->-r ./requirements.txt (line 1)) (3.1.6)

Requirement already satisfied: jupyter<2.0.0,>=1.16.4 in /usr/local/lib/python3.12/dist-packages (from otter_grader->-r ./requirements.txt (line 1)) (1.17.3)

Requirement already satisfied: nbconvert>=6.0.0 in /usr/local/lib/python3.12/dist-packages (from nbconvert[webpdf]>=6.0.0; sys_platform != "emscripten" and sys_platform != "wasi"->otter_grader->-r ./requirements.txt (line 1)) (7.16.6)

Requirement already satisfied: nbformat>=5.0.0 in

```

/usr/local/lib/python3.12/dist-packages (from otter_grader->-r
./requirements.txt (line 1)) (5.10.4)
Requirement already satisfied: python-on-whales<1.0.0,>=0.72.0 in
/usr/local/lib/python3.12/dist-packages (from otter_grader->-r
./requirements.txt (line 1)) (0.79.0)
Requirement already satisfied: pyyaml<7,>=6 in /usr/local/lib/python3.12/dist-
packages (from otter_grader->-r ./requirements.txt (line 1)) (6.0.3)
Requirement already satisfied: requests<3.0,>=2.31 in
/usr/local/lib/python3.12/dist-packages (from otter_grader->-r
./requirements.txt (line 1)) (2.32.4)
Requirement already satisfied: wrapt<2.0.0,>=1.16.0 in
/usr/local/lib/python3.12/dist-packages (from otter_grader->-r
./requirements.txt (line 1)) (1.17.3)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
packages (from datasets->-r ./requirements.txt (line 2)) (3.20.0)
Requirement already satisfied: pyarrow>=15.0.0 in
/usr/local/lib/python3.12/dist-packages (from datasets->-r ./requirements.txt
(line 2)) (18.1.0)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages
(from datasets->-r ./requirements.txt (line 2)) (3.6.0)
Requirement already satisfied: multiprocessing<0.70.17 in
/usr/local/lib/python3.12/dist-packages (from datasets->-r ./requirements.txt
(line 2)) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in
/usr/local/lib/python3.12/dist-packages (from
fsspec[http]<=2025.3.0,>=2023.1.0->datasets->-r ./requirements.txt (line 2))
(2025.3.0)
Requirement already satisfied: huggingface-hub>=0.24.0 in
/usr/local/lib/python3.12/dist-packages (from datasets->-r ./requirements.txt
(line 2)) (0.36.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-
packages (from datasets->-r ./requirements.txt (line 2)) (25.0)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->-r ./requirements.txt
(line 3)) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-
packages (from matplotlib->-r ./requirements.txt (line 3)) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->-r ./requirements.txt
(line 3)) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->-r ./requirements.txt
(line 3)) (1.4.9)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->-r ./requirements.txt
(line 3)) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->-r ./requirements.txt

```

(line 3)) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->-r ./requirements.txt (line 5)) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->-r ./requirements.txt (line 5)) (2025.2)

Requirement already satisfied: narwhals>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from plotly->-r ./requirements.txt (line 7)) (2.11.0)

Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->-r ./requirements.txt (line 9)) (1.16.3)

Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->-r ./requirements.txt (line 9)) (1.5.2)

Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->-r ./requirements.txt (line 9)) (3.6.0)

Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line 11)) (4.15.0)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line 11)) (75.2.0)

Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line 11)) (1.13.3)

Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line 11)) (3.5)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line 11)) (12.6.77)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line 11)) (12.6.77)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line 11)) (12.6.80)

Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line 11)) (9.10.2.21)

Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line 11)) (12.6.4.1)

Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line 11)) (11.3.0.4)

Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line 11)) (10.3.7.77)

Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line 11)) (11.7.1.2)

Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in
 /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line
 11)) (12.5.4.2)

Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in
 /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line
 11)) (0.7.1)

Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in
 /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line
 11)) (2.27.3)

Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in
 /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line
 11)) (12.6.77)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in
 /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line
 11)) (12.6.85)

Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in
 /usr/local/lib/python3.12/dist-packages (from torch->-r ./requirements.txt (line
 11)) (1.11.1.6)

Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-
 packages (from torch->-r ./requirements.txt (line 11)) (3.4.0)

Requirement already satisfied: regex!=2019.12.17 in
 /usr/local/lib/python3.12/dist-packages (from transformers->-r
 ./requirements.txt (line 14)) (2024.11.6)

Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in
 /usr/local/lib/python3.12/dist-packages (from transformers->-r
 ./requirements.txt (line 14)) (0.22.1)

Requirement already satisfied: safetensors>=0.4.3 in
 /usr/local/lib/python3.12/dist-packages (from transformers->-r
 ./requirements.txt (line 14)) (0.6.2)

Requirement already satisfied: docutils in /usr/local/lib/python3.12/dist-
 packages (from fica>=0.4.1->otter_grader->-r ./requirements.txt (line 1))
 (0.21.2)

Requirement already satisfied: sphinx in /usr/local/lib/python3.12/dist-packages
 (from fica>=0.4.1->otter_grader->-r ./requirements.txt (line 1)) (8.2.3)

Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in
 /usr/local/lib/python3.12/dist-packages (from
 fsspec[http]<=2025.3.0,>=2023.1.0->datasets->-r ./requirements.txt (line 2))
 (3.13.2)

Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in
 /usr/local/lib/python3.12/dist-packages (from huggingface-
 hub>=0.24.0->datasets->-r ./requirements.txt (line 2)) (1.2.0)

Requirement already satisfied: comm>=0.1.3 in /usr/local/lib/python3.12/dist-
 packages (from ipywidgets<9.0.0,>=8.1.5->otter_grader->-r ./requirements.txt
 (line 1)) (0.2.3)

Requirement already satisfied: traitlets>=4.3.1 in
 /usr/local/lib/python3.12/dist-packages (from
 ipywidgets<9.0.0,>=8.1.5->otter_grader->-r ./requirements.txt (line 1)) (5.7.1)

Requirement already satisfied: widgetsnbextension~=4.0.14 in

```

/usr/local/lib/python3.12/dist-packages (from
ipywidgets<9.0.0,>=8.1.5->otter_grader->-r ./requirements.txt (line 1)) (4.0.15)
Requirement already satisfied: jupyterlab_widgets~=3.0.15 in
/usr/local/lib/python3.12/dist-packages (from
ipywidgets<9.0.0,>=8.1.5->otter_grader->-r ./requirements.txt (line 1)) (3.0.16)
Requirement already satisfied: jedi>=0.16 in /usr/local/lib/python3.12/dist-
packages (from ipython->otter_grader->-r ./requirements.txt (line 1)) (0.19.2)
Requirement already satisfied: decorator in /usr/local/lib/python3.12/dist-
packages (from ipython->otter_grader->-r ./requirements.txt (line 1)) (4.4.2)
Requirement already satisfied: pickleshare in /usr/local/lib/python3.12/dist-
packages (from ipython->otter_grader->-r ./requirements.txt (line 1)) (0.7.5)
Requirement already satisfied: prompt-toolkit!=3.0.0,!<3.0.1,<3.1.0,>=2.0.0 in
/usr/local/lib/python3.12/dist-packages (from ipython->otter_grader->-r
./requirements.txt (line 1)) (3.0.52)
Requirement already satisfied: pygments in /usr/local/lib/python3.12/dist-
packages (from ipython->otter_grader->-r ./requirements.txt (line 1)) (2.19.2)
Requirement already satisfied: backcall in /usr/local/lib/python3.12/dist-
packages (from ipython->otter_grader->-r ./requirements.txt (line 1)) (0.2.0)
Requirement already satisfied: matplotlib-inline in
/usr/local/lib/python3.12/dist-packages (from ipython->otter_grader->-r
./requirements.txt (line 1)) (0.2.1)
Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.12/dist-
packages (from ipython->otter_grader->-r ./requirements.txt (line 1)) (4.9.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2<4.0,>=3.1->otter_grader->-r
./requirements.txt (line 1)) (3.0.3)
Requirement already satisfied: markdown-it-py>=1.0 in
/usr/local/lib/python3.12/dist-packages (from
jupyter-text<2.0.0,>=1.16.4->otter_grader->-r ./requirements.txt (line 1)) (4.0.0)
Requirement already satisfied: mdit-py-plugins in
/usr/local/lib/python3.12/dist-packages (from
jupyter-text<2.0.0,>=1.16.4->otter_grader->-r ./requirements.txt (line 1)) (0.5.0)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-
packages (from nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0; sys_platform !=
"emscripten" and sys_platform != "wasi"->otter_grader->-r ./requirements.txt
(line 1)) (4.13.5)
Requirement already satisfied: bleach!=5.0.0 in /usr/local/lib/python3.12/dist-
packages (from bleach[css]!=5.0.0->nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0;
sys_platform != "emscripten" and sys_platform != "wasi"->otter_grader->-r
./requirements.txt (line 1)) (6.3.0)
Requirement already satisfied: defusedxml in /usr/local/lib/python3.12/dist-
packages (from nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0; sys_platform !=
"emscripten" and sys_platform != "wasi"->otter_grader->-r ./requirements.txt
(line 1)) (0.7.1)
Requirement already satisfied: jupyter-core>=4.7 in
/usr/local/lib/python3.12/dist-packages (from
nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0; sys_platform != "emscripten" and
sys_platform != "wasi"->otter_grader->-r ./requirements.txt (line 1)) (5.9.1)

```

Requirement already satisfied: jupyterlab-pygments in
 /usr/local/lib/python3.12/dist-packages (from
 nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0; sys_platform != "emscripten" and
 sys_platform != "wasi"->otter_grader->-r ./requirements.txt (line 1)) (0.3.0)

Requirement already satisfied: mistune<4,>=2.0.3 in
 /usr/local/lib/python3.12/dist-packages (from
 nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0; sys_platform != "emscripten" and
 sys_platform != "wasi"->otter_grader->-r ./requirements.txt (line 1)) (3.1.4)

Requirement already satisfied: nbclient>=0.5.0 in
 /usr/local/lib/python3.12/dist-packages (from
 nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0; sys_platform != "emscripten" and
 sys_platform != "wasi"->otter_grader->-r ./requirements.txt (line 1)) (0.10.2)

Requirement already satisfied: pandocfilters>=1.4.1 in
 /usr/local/lib/python3.12/dist-packages (from
 nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0; sys_platform != "emscripten" and
 sys_platform != "wasi"->otter_grader->-r ./requirements.txt (line 1)) (1.5.1)

Requirement already satisfied: playwright in /usr/local/lib/python3.12/dist-
 packages (from nbconvert[webpdf]>=6.0.0; sys_platform != "emscripten" and
 sys_platform != "wasi"->otter_grader->-r ./requirements.txt (line 1)) (1.56.0)

Requirement already satisfied: fastjsonschema>=2.15 in
 /usr/local/lib/python3.12/dist-packages (from nbformat>=5.0.0->otter_grader->-r
 ./requirements.txt (line 1)) (2.21.2)

Requirement already satisfied: jsonschema>=2.6 in
 /usr/local/lib/python3.12/dist-packages (from nbformat>=5.0.0->otter_grader->-r
 ./requirements.txt (line 1)) (4.25.1)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
 packages (from python-dateutil>=2.7->matplotlib->-r ./requirements.txt (line 3))
 (1.17.0)

Requirement already satisfied: pydantic!=2.0.*,<3,>=2 in
 /usr/local/lib/python3.12/dist-packages (from python-on-
 whales<1.0.0,>=0.72.0->otter_grader->-r ./requirements.txt (line 1)) (2.11.10)

Requirement already satisfied: charset_normalizer<4,>=2 in
 /usr/local/lib/python3.12/dist-packages (from
 requests<3.0,>=2.31->otter_grader->-r ./requirements.txt (line 1)) (3.4.4)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-
 packages (from requests<3.0,>=2.31->otter_grader->-r ./requirements.txt (line
 1)) (3.11)

Requirement already satisfied: urllib3<3,>=1.21.1 in
 /usr/local/lib/python3.12/dist-packages (from
 requests<3.0,>=2.31->otter_grader->-r ./requirements.txt (line 1)) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in
 /usr/local/lib/python3.12/dist-packages (from
 requests<3.0,>=2.31->otter_grader->-r ./requirements.txt (line 1)) (2025.10.5)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in
 /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch->-r
 ./requirements.txt (line 11)) (1.3.0)

Requirement already satisfied: aiohappyeyeballs>=2.5.0 in
 /usr/local/lib/python3.12/dist-packages (from

```

aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets->-r
./requirements.txt (line 2)) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in
/usr/local/lib/python3.12/dist-packages (from
aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets->-r
./requirements.txt (line 2)) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-
packages (from
aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets->-r
./requirements.txt (line 2)) (25.4.0)
Requirement already satisfied: frozenlist>=1.1.1 in
/usr/local/lib/python3.12/dist-packages (from
aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets->-r
./requirements.txt (line 2)) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
/usr/local/lib/python3.12/dist-packages (from
aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets->-r
./requirements.txt (line 2)) (6.7.0)
Requirement already satisfied: propcache>=0.2.0 in
/usr/local/lib/python3.12/dist-packages (from
aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets->-r
./requirements.txt (line 2)) (0.4.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in
/usr/local/lib/python3.12/dist-packages (from
aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2025.3.0,>=2023.1.0->datasets->-r
./requirements.txt (line 2)) (1.22.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-
packages (from
bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0;
sys_platform != "emscripten" and sys_platform != "wasi"->otter_grader->-r
./requirements.txt (line 1)) (0.5.1)
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from
bleach[css]!=5.0.0->nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0; sys_platform !=
"emscripten" and sys_platform != "wasi"->otter_grader->-r ./requirements.txt
(line 1)) (1.4.0)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in
/usr/local/lib/python3.12/dist-packages (from
jedi>=0.16->ipython->otter_grader->-r ./requirements.txt (line 1)) (0.8.5)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
/usr/local/lib/python3.12/dist-packages (from
jsonschema>=2.6->nbformat>=5.0.0->otter_grader->-r ./requirements.txt (line 1))
(2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in
/usr/local/lib/python3.12/dist-packages (from
jsonschema>=2.6->nbformat>=5.0.0->otter_grader->-r ./requirements.txt (line 1))
(0.37.0)
Requirement already satisfied: rpds-py>=0.7.1 in /usr/local/lib/python3.12/dist-

```


packages (from jsonschema>=2.6->nbformat>=5.0.0->otter_grader->-r
 ./requirements.txt (line 1)) (0.28.0)
 Requirement already satisfied: platformdirs>=2.5 in
 /usr/local/lib/python3.12/dist-packages (from jupyter-
 core>=4.7->nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0; sys_platform !=
 "emscripten" and sys_platform != "wasi"->otter_grader->-r ./requirements.txt
 (line 1)) (4.5.0)
 Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-
 packages (from markdown-it-py>=1.0->jupyter-text<2.0.0,>=1.16.4->otter_grader->-r
 ./requirements.txt (line 1)) (0.1.2)
 Requirement already satisfied: jupyter-client>=6.1.12 in
 /usr/local/lib/python3.12/dist-packages (from
 nbclient>=0.5.0->nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0; sys_platform !=
 "emscripten" and sys_platform != "wasi"->otter_grader->-r ./requirements.txt
 (line 1)) (7.4.9)
 Requirement already satisfied: ptyprocess>=0.5 in
 /usr/local/lib/python3.12/dist-packages (from
 pexpect>4.3->ipython->otter_grader->-r ./requirements.txt (line 1)) (0.7.0)
 Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-
 packages (from prompt-
 toolkit!=3.0.0,!<3.0.1,<3.1.0,>=2.0.0->ipython->otter_grader->-r
 ./requirements.txt (line 1)) (0.2.14)
 Requirement already satisfied: annotated-types>=0.6.0 in
 /usr/local/lib/python3.12/dist-packages (from pydantic!=2.0.*,<3,>=2->python-on-
 whales<1.0.0,>=0.72.0->otter_grader->-r ./requirements.txt (line 1)) (0.7.0)
 Requirement already satisfied: pydantic-core==2.33.2 in
 /usr/local/lib/python3.12/dist-packages (from pydantic!=2.0.*,<3,>=2->python-on-
 whales<1.0.0,>=0.72.0->otter_grader->-r ./requirements.txt (line 1)) (2.33.2)
 Requirement already satisfied: typing-inspection>=0.4.0 in
 /usr/local/lib/python3.12/dist-packages (from pydantic!=2.0.*,<3,>=2->python-on-
 whales<1.0.0,>=0.72.0->otter_grader->-r ./requirements.txt (line 1)) (0.4.2)
 Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-
 packages (from beautifulsoup4->nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0;
 sys_platform != "emscripten" and sys_platform != "wasi"->otter_grader->-r
 ./requirements.txt (line 1)) (2.8)
 Requirement already satisfied: pyee<14,>=13 in /usr/local/lib/python3.12/dist-
 packages (from playwright->nbconvert[webpdf]>=6.0.0; sys_platform !=
 "emscripten" and sys_platform != "wasi"->otter_grader->-r ./requirements.txt
 (line 1)) (13.0.0)
 Requirement already satisfied: greenlet<4.0.0,>=3.1.1 in
 /usr/local/lib/python3.12/dist-packages (from
 playwright->nbconvert[webpdf]>=6.0.0; sys_platform != "emscripten" and
 sys_platform != "wasi"->otter_grader->-r ./requirements.txt (line 1)) (3.2.4)
 Requirement already satisfied: sphinxcontrib-applehelp>=1.0.7 in
 /usr/local/lib/python3.12/dist-packages (from
 sphinx->fika>=0.4.1->otter_grader->-r ./requirements.txt (line 1)) (2.0.0)
 Requirement already satisfied: sphinxcontrib-devhelp>=1.0.6 in
 /usr/local/lib/python3.12/dist-packages (from

```

sphinx->fica>=0.4.1->otter_grader->-r ./requirements.txt (line 1)) (2.0.0)
Requirement already satisfied: sphinxcontrib-htmlhelp>=2.0.6 in
/usr/local/lib/python3.12/dist-packages (from
sphinx->fica>=0.4.1->otter_grader->-r ./requirements.txt (line 1)) (2.1.0)
Requirement already satisfied: sphinxcontrib-jsmath>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from
sphinx->fica>=0.4.1->otter_grader->-r ./requirements.txt (line 1)) (1.0.1)
Requirement already satisfied: sphinxcontrib-qthelp>=1.0.6 in
/usr/local/lib/python3.12/dist-packages (from
sphinx->fica>=0.4.1->otter_grader->-r ./requirements.txt (line 1)) (2.0.0)
Requirement already satisfied: sphinxcontrib-serializinghtml>=1.1.9 in
/usr/local/lib/python3.12/dist-packages (from
sphinx->fica>=0.4.1->otter_grader->-r ./requirements.txt (line 1)) (2.0.0)
Requirement already satisfied: snowballstemmer>=2.2 in
/usr/local/lib/python3.12/dist-packages (from
sphinx->fica>=0.4.1->otter_grader->-r ./requirements.txt (line 1)) (3.0.1)
Requirement already satisfied: babel>=2.13 in /usr/local/lib/python3.12/dist-
packages (from sphinx->fica>=0.4.1->otter_grader->-r ./requirements.txt (line
1)) (2.17.0)
Requirement already satisfied: alabaster>=0.7.14 in
/usr/local/lib/python3.12/dist-packages (from
sphinx->fica>=0.4.1->otter_grader->-r ./requirements.txt (line 1)) (1.0.0)
Requirement already satisfied: imagesize>=1.3 in /usr/local/lib/python3.12/dist-
packages (from sphinx->fica>=0.4.1->otter_grader->-r ./requirements.txt (line
1)) (1.4.1)
Requirement already satisfied: roman-numerals-py>=1.0.0 in
/usr/local/lib/python3.12/dist-packages (from
sphinx->fica>=0.4.1->otter_grader->-r ./requirements.txt (line 1)) (3.1.0)
Requirement already satisfied: entrypoints in /usr/local/lib/python3.12/dist-
packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0;
sys_platform != "emscripten" and sys_platform != "wasi"->otter_grader->-r
./requirements.txt (line 1)) (0.4)
Requirement already satisfied: nest-asyncio>=1.5.4 in
/usr/local/lib/python3.12/dist-packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0;
sys_platform != "emscripten" and sys_platform != "wasi"->otter_grader->-r
./requirements.txt (line 1)) (1.6.0)
Requirement already satisfied: pyzmq>=23.0 in /usr/local/lib/python3.12/dist-
packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0;
sys_platform != "emscripten" and sys_platform != "wasi"->otter_grader->-r
./requirements.txt (line 1)) (26.2.1)
Requirement already satisfied: tornado>=6.2 in /usr/local/lib/python3.12/dist-
packages (from jupyter-
client>=6.1.12->nbclient>=0.5.0->nbconvert>=6.0.0->nbconvert[webpdf]>=6.0.0;
sys_platform != "emscripten" and sys_platform != "wasi"->otter_grader->-r
./requirements.txt (line 1)) (6.5.1)

```

Requirement already satisfied: kaleido in /usr/local/lib/python3.12/dist-packages (1.2.0)
 Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (6.5.0)
 Requirement already satisfied: choreographer>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from kaleido) (1.2.1)
 Requirement already satisfied: logistro>=1.0.8 in /usr/local/lib/python3.12/dist-packages (from kaleido) (2.0.1)
 Requirement already satisfied: orjson>=3.10.15 in /usr/local/lib/python3.12/dist-packages (from kaleido) (3.11.4)
 Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kaleido) (25.0)
 Requirement already satisfied: pytest-timeout>=2.4.0 in /usr/local/lib/python3.12/dist-packages (from kaleido) (2.4.0)
 Requirement already satisfied: narwhals>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from plotly) (2.11.0)
 Requirement already satisfied: simplejson>=3.19.3 in /usr/local/lib/python3.12/dist-packages (from choreographer>=1.1.1->kaleido) (3.20.2)
 Requirement already satisfied: pytest>=7.0.0 in /usr/local/lib/python3.12/dist-packages (from pytest-timeout>=2.4.0->kaleido) (8.4.2)
 Requirement already satisfied: iniconfig>=1 in /usr/local/lib/python3.12/dist-packages (from pytest>=7.0.0->pytest-timeout>=2.4.0->kaleido) (2.3.0)
 Requirement already satisfied: pluggy<2,>=1.5 in /usr/local/lib/python3.12/dist-packages (from pytest>=7.0.0->pytest-timeout>=2.4.0->kaleido) (1.6.0)
 Requirement already satisfied: pygments>=2.7.2 in /usr/local/lib/python3.12/dist-packages (from pytest>=7.0.0->pytest-timeout>=2.4.0->kaleido) (2.19.2)

```
[51]: # Initialize Otter
import otter
grader = otter.Notebook("hw4_part1.ipynb")
```

1 Homework 4.1

CNNs and Transformers have *transformed* machine learning, opening up whole new domains of innovation: [object detection for autonomous vehicles](#), image generation ([Dall-E](#)), [LLMs](#), [protein-sequence modeling](#), and more!

In this homework, we're going to explore CNNs and Transformers by implementing some foundational CNN architectures using [PyTorch](#) and coding some key transformer building blocks from scratch.

1.1 Due Date: Friday, November 21, 11:59 PM

This assignment is due on **Friday, November 21, at 11:59 PM**. You must submit your work to Gradescope by this deadline. Please refer to the syllabus for the [Slip Day policy](#). No late submissions will be accepted beyond the details outlined in the Slip Day policy.

1.2 Deliverables:

- Homework 4.1 consists of paper questions and coding questions.
- Paper questions:
 - Please write or type your answers to the paper questions and submit your answers as a **PDF** to the HW 4 Written Questions assignment on Gradescope.
- Code [PDF]:
 - Please submit a **PDF** of your coding notebook to the HW 4.1 Coding [PDF] assignment on Gradescope.
 - Please tag the pages in your PDF to the corresponding questions on Gradescope.
- Code [Notebook]:
 - Please submit the zip file generated by the final export cell in this coding notebook to the HW 4.1 Coding [Code] assignment on Gradescope.
 - Code submitted in your notebook must match the code in your PDF submission, and produce the *exact* same outputs if re-run by course staff. Inconsistent or incomplete code might not receive marks.
- Note: Make sure you submit *both* a PDF version of your notebook (to the Coding [PDF] assignment) and a zip file of the notebook (to the Coding [Code] assignment) on Gradescope.

1.2.1 Submission Tips:

- **Plan ahead:** We strongly encourage you to submit your work several hours before the deadline. This will give you ample time to address any submission issues.
 - **Reach out for help early:** If you encounter difficulties, contact course staff well before the deadline. While we are happy to assist with submission issues, we cannot guarantee responses to last-minute requests.
-

1.3 Assignment Overview

This notebook contains a series of tasks designed to help you practice and apply key concepts in data manipulation and visualization. You will complete all the TODOs in the notebook, which include both coding and written response questions. Some tasks are open-ended, which allows you to explore and experiment with different approaches.

1.3.1 Key Learning Objectives:

1. Implement your own neural networks in PyTorch
2. Create your own custom Datasets, DataLoaders, and training loops
3. Gain hands-on experience implementing an architecture from a paper
4. Learn about the ResNet architecture

5. Implement a transformer block in PyTorch
- 6.

1.4 Understand how the key components in a transformer fit together

1.5 Grading Breakdown

Question	Points
1a	1
1b	1
1c	1
1d	1
1e	1
2a	1
2b	1
2c	1
2d	1
3a	1
3b	1
3c	1
3d	1
3e	1
3f	1
3g	1
3h	1
3i	1
3j	1
3k	1
3l	1
3m	1
3n	1
3o	1
Total	25

1.6 Instructions:

1. Carefully read each question and its requirements.
2. Complete all TODOs in the notebook. You may add extra lines of code if needed to implement your solution.
3. For manual questions, provide clear and concise written responses.
4. Test your code thoroughly to ensure it meets the requirements.

Good luck! Autobots, roll out!

```
[52]: import numpy as np
import torch
```

```

import torch.nn as nn
import torch.nn.functional as F
from torch.optim import AdamW
import random
from datasets import load_dataset
import matplotlib.pyplot as plt
import plotly
import plotly.subplots
import plotly.graph_objects
import os
import json
import re
from tqdm import tqdm
from torch.utils.data import DataLoader, Dataset, TensorDataset
import torchvision.transforms as transforms

```

1.6.1 IMPORTANT:

- Do not change the random seed values!!!
- Make sure to turn on the GPU accelerator in Google Colab's runtime settings before starting the notebook.

```

[53]: def set_seed(seed):
    """
    Set the seed for all random number generators
    """
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

    # Set random seeds for reproducible results
    SEED = 42
    set_seed(SEED)

    IS_GRADING_ENV = os.getenv("IS_GRADING_ENV") == "true"

    if torch.accelerator.is_available():
        device = torch.accelerator.current_accelerator()
    else:
        device = "cpu"

    print(f"Using device: {device}") # Check that the current device is cuda or
    ↪ another GPU accelerator!

```

Using device: cuda

2 Subpart 1: Convolutional Neural Networks

First, let's warm up by building a CNN (Convolutional Neural Network) in PyTorch to get familiar with the PyTorch library.

2.0.1 So what is PyTorch?

PyTorch is a powerful Python library for building and training neural networks. It is built around data structures called tensors (multi-dimensional arrays!). PyTorch is awesome because it handles creating computational graphs and calculating gradients for us.

PyTorch Modules PyTorch's main building block are *modules*, which you can think of as different layers in your neural net. PyTorch has a couple of pre-defined classes which inherit from the base `nn.Module` class.

Modules can be as simple as individual layers: * [Linear fully-connected layer](#)

... activation functions: * [ReLU](#)

... entire blocks of a model, which often have *other* modules inside of them! * [LSTM/long short-term memory blocks](#)

... or complete models! * [Transformer - we'll be implementing our own version later in this homework](#)

In short, you can think of modules as LEGO bricks that you can piece together in different ways.

Implementing custom `nn.Module` subclasses! We can also define our own classes that inherit from the `nn.Module` base class! To implement your own subclass of `nn.Module`, you need to define 2 methods: * `__init__(self, ...)`: The constructor method where you initialize all the layers and parameters your module will use. Call `super().__init__()` first to properly initialize the parent `nn.Module` class. Define any layers (e.g., `Conv2d`, `Linear`, etc.) as instance attributes here. PyTorch will automatically track these as submodules and register their parameters.

- `forward(self, x)`: Defines the forward pass computation. Takes input tensor(s) and returns output tensor(s). This is where you specify how data flows through your layers—apply operations, activations, skip connections, etc. You call your model with `output = model(input)`, which internally invokes the forward method. PyTorch's autograd automatically builds the backward pass from your forward operations.

PyTorch Functionals Another important part of PyTorch is `torch.nn.functional`. This is a module (note: “module” in this case refers to a sublibrary of a Python library. It's not the same thing as the `Module` classes we were just talking about! I know, it's confusing!) of stateless functions, which means that these functions perform operations on tensors without maintaining any state or tracking parameters. So these don't contain learnable parameters we can train and update. `torch.nn.functional` is often imported as `F`; in practice you'll often see it being called like this: `F.relu(x)`.

Often times, certain kinds of layers have both `nn.Module` and `nn.functional` implementations in PyTorch. How do you know when to use which kind? As a rule of thumb, whenever you need to the layer's parameters to be learnable (i.e. changeable over the course of training), use the `nn.Module` version. On the other hand, for operations like activation functions (e.g. `ReLU` or `TanH`) we don't

have any parameters that we can learn! So their `nn.Module` and `nn.functional` version can be used interchangeably. In your code below, you can choose to use either `nn.ReLU` or `F.relu`.

Aspect	<code>nn.Module</code>	<code>nn.Functional</code>
State	Stateful - maintains parameters	Stateless - just functions
Parameters	Automatically registers learnable parameters and tracks them for optimization	No parameter tracking
Implementation details	Need to initialize the module first (e.g. using <code>self.relu = F.relu()</code>) before using it (e.g. <code>a = self.relu(x)</code>)	Can call it directly on an input (e.g. <code>a = F.relu(x)</code>)
Usage examples	<code>nn.Linear(10, 5)</code> , <code>nn.ReLU()</code>	<code>F.linear(x)</code> , <code>F.relu(x)</code>

2.1 Question 1a

Using some of PyTorch's existing `nn.Module` classes, we can build our own neural networks. Let's try building a CNN with the following architecture:

- **Convolution Layer 1:** 16 output channels, kernel size 3 x 3, stride 2, padding 1
- **ReLU Activation:** Element-wise ReLU
- **Convolution Layer 2:** 16 output channels, kernel size 7 x 7, stride 2, padding 1
- **ReLU Activation:** Element-wise ReLU
- **Linear Classifier:** Linear layer for classification

Task: 1. Implement the `__init__` method. 1. Don't forget to call `super().__init__()`! 2. Instantiate all the layers we'll need for our CNN. 2. Implement the `forward` method. 1. Pass the input `x` through each of the layers. 2. Return the final output.

Hints: * The documentation for the `Conv2d` class [here](#) will contain helpful information about what parameters you need to specify to instantiate a `Conv2d` layer * Since our images have 3 channels for Red, Green, and Blue, how many input channels should our first `Conv2d` layer have? * How many channels does the output of the 1st ReLU layer have? What should be the number of input channels for the 2nd `Conv2d` layer? * The final linear layer expects a flattened tensor for each item in the batch. What shape is the output after the 2nd ReLU? (Hint: you can either calculate the shape explicitly using formulas for what the dimensions of a conv layer's feature map will be, or add a print statement!) How can you flatten it into a 1D tensor with shape `(Batch_size, 1)`? * Helpful website to calculate the output shapes after a convolution: <https://madebyollin.github.io/convnet-calculator/> * The linear classifier will make predictions about which class the image belongs to by outputting a vector of dimension `(num_classes)` containing the probabilities that the image belongs to a certain class for each image in the batch. What should be the output dimension for the linear classifier? * You can choose to either instantiate a `nn.ReLU` layer in `__init__`, or directly call the functional `F.ReLU` in your forward pass!

Helpful Documentation * `nn.Conv2d` * `nn.Linear` * `nn.ReLU` * `F.ReLU`

```
[54]: class CNN(nn.Module):
        def __init__(self, num_classes):
            super().__init__()
```



```

# TODO: Instantiate the layers your CNN will have!
self.conv1 = nn.Conv2d(3,16,3,stride=2,padding=1) # 112 x 112 x 16
self.conv2 = nn.Conv2d(16,16,7,stride=2,padding=1) # 54 x 54 x 16
self.linear = nn.Linear(46656,num_classes)

def forward(self, x):
    # TODO: 1. Pass the input through the first conv layer
    conv1_out = self.conv1(x)

    # TODO: 2. Apply ReLU
    relu1_out = F.relu(conv1_out)

    # TODO: 3. Pass to the 2nd conv layer
    conv2_out = self.conv2(relu1_out)

    # TODO: 4. Apply ReLU
    relu2_out = F.relu(conv2_out)

    # TODO: 5. Flatten before linear layer
    flat = torch.flatten(relu2_out,start_dim=1)

    # TODO: 6. Pass to linear layer
    out = self.linear(flat)

    return out

```

2.1.1 Check: CNN output shape

A helpful debugging tip while building your own deep-learning models or custom modules is to check the shapes of your outputs!

Below, we create a tensor of shape (1, 224, 224, 3) filled with random numbers between [0, 1] and pass it through our CNN to check if the output has the shape we expect.

How do we determine what the shape of our output should be? We can trace through what the shapes should be after each of our layers.

Assume we instantiate a CNN with `num_classes=10` and we pass an input of shape (1, 3, 224, 224) through the CNN.

These would be the shapes after each layer: 1. Input: 1 x 3 x 224 x 224 -> dimensions: Batch size x 3 RGB channels x height of 224 pixels x width of 224 pixels 2. After conv1: 1 x 16 * 112 * 112 -> The first convolution produces feature maps with shape 112 x 112 and increases the number of channels to 16 4. After the first ReLU: 1 x 16 * 112 * 112 -> ReLU doesn't change the output dimensions 5. After conv2: 1 x 16 x 54 x 54 -> The second convolution produces feature maps with shape 54 x 54. This convolution doesn't change the number of channels 6. After the 2nd ReLU: 1 x 16 x 54 x 54 -> ReLU doesn't change the output dimensions 7. After the linear layer: 1 x 10 -> A tensor containing the logits for each class

You should verify that the shape of your actual output is also 1 x 10!

```
[55]: x = torch.rand((1, 3, 224, 224)) # 1 image, 3 RGB channels, 224 x 224 pixels
      print(f"Shape of input: {x.shape}")

      res = CNN(num_classes=10)
      out = res(x)
      print(f"Shape of output: {out.shape}")
      assert out.shape == torch.Size([1, 10]), f"Expected output shape (1, 10), but_
      ↪got {out.shape}"
```

```
Shape of input: torch.Size([1, 3, 224, 224])
```

```
Shape of output: torch.Size([1, 10])
```

2.2 Load the Data

Before we jump into training our models, let's discuss how to work with data in our training loop. First, we'll load in a image classification dataset from HuggingFace.

For this assignment, we'll use a "mini" version of ImageNet-1k from HuggingFace, which contains 100 of the original 1000 ImageNet-1k classes. The Mini Imagenet dataset comes from timm (PyTorch Image Models), which is a popular Python library that provides a comprehensive collection of pre-trained computer vision models, particularly focused on image classification.

The mini-imagenet dataset on HuggingFace has: * 50,000 training samples * 10,000 validation images * 5,000 test samples (50 samples per class)

The `split` parameter in HuggingFace's `load_dataset` function specifies which portion of the dataset to load. You can load a complete split like `split='train'`, `split='validation'`, or `split='test'`. Additionally, you can load just a subset of a split using slice notation - for example, `split='train[:10%]'` will load only the first **10% of the classes in the training data** (i.e. 10 of the 100 classes), which is useful for faster experimentation and development.

We want to work with just 10 classes for learning efficiency. To simplify our experiments and ensure balanced class representation, we'll use a helper function `balanced_split` to sample exactly 1000 training images, 200 validation images, and 200 test images, with images distributed evenly between the selected classes (as much as possible). This means each class will contribute an equal number of samples, unless a class has too few images, in which case we'll include all available samples from that class.

Notes: * We're only using 1000 training images, 200 validation images, and 200 test images for learning purposes. In a real-world scenario, you'd likely use a far larger dataset for training and evaluation :) * Loading the dataset and creating a balanced split may take a couple minutes to complete.

2.2.1 Understanding HuggingFace Dataset Objects

HuggingFace datasets are structured like lists of dictionaries, where each element represents one data sample. For image datasets like mini-imagenet, each sample has two key fields:

- **image:** A PIL Image object. This is the actual image data that can be displayed, transformed, or converted to a numpy array using `np.array(image)`. The images are typically stored in

RGB format with pixel values ranging from 0-255.

- **label:** An integer representing the class ID for that image. For example, `label=0` might correspond to “house_finch”, `label=1` to “robin”, etc. The mapping between label integers and class names is typically stored in the dataset’s features or metadata.

Example usage:

```
# Access a single sample
sample = mini_imagenet_train[0]
print(sample.keys()) # Output: dict_keys(['image', 'label'])

# Get the image and label
image = sample['image'] # PIL Image object
label = sample['label'] # Integer class ID

# Convert image to numpy array (height, width, channels)
image_array = np.array(image)
print(image_array.shape) # e.g., (84, 84, 3) for RGB
```

```
[56]: # Helper function to sample a balanced subset of the dataset
def balanced_split(dataset, n_total: int, classes_to_keep: set[int] = None):
    """
    Returns a balanced subset of the dataset with `n_total` samples, equally
    ↪divided among the specified classes.

    If `classes_to_keep` is provided, only samples from these classes are
    ↪included, each contributing exactly (n_total // num_classes) samples.
    If not provided, all unique classes in the dataset are used.
    Assumes the dataset is a Hugging Face Dataset object with a 'label' field.

    Args:
        dataset (datasets.Dataset):
            The dataset to subsample. Must have a 'label' column.
        n_total (int):
            Total number of samples to return. Must be divisible by the number of
            ↪selected classes.
        classes_to_keep (set[int], optional):
            Class labels to include in the balanced subset. Each class will have
            ↪n_total // num_classes samples.
            If None, all classes in the dataset will be used.

    Returns:
        datasets.Dataset:
            A new dataset with `n_total` samples, equally distributed across the
            ↪specified classes.

    Raises:
        ValueError:
```

```

        If the number of samples per class is greater than the available
        samples for any class,
        or if n_total is not divisible by the number of classes.
    """
    if not classes_to_keep:
        classes_to_keep = set(dataset['label'])
    n_per_class = n_total // len(classes_to_keep)

    # Dictionary of class to list of indices that correspond to that class
    cls_to_indices = {cls: [] for cls in classes_to_keep }

    # Iterate through all indices and labels in the dataset's label list
    for i, cls in enumerate(dataset['label']):
        # If this label corresponds to a class we want to keep, append it to the
        appropriate class's list
        if cls in cls_to_indices:
            cls_to_indices[cls].append(i)

    sampled_idx = []
    for cls, idx_list in cls_to_indices.items():
        if len(idx_list) < n_per_class:
            # If not enough samples, include them all
            sampled_idx.extend(idx_list)
        else:
            sampled_idx.extend(random.sample(idx_list, k=n_per_class))

    return dataset.select(sampled_idx)

# Load a dictionary that maps numeric class IDs to string class names
with open('imagenet_class_id_to_name.json', 'r') as file:
    class_id_to_name = json.load(file)
    class_id_to_name = {int(key): value for key, value in class_id_to_name.
        items()}
    print(f"All classes in the ImageNet dataset: {class_id_to_name}")

# Load the training, validation, and test sets
mini_imagenet_train = load_dataset("timm/mini-imagenet", split="train[:10%]")
mini_imagenet_train = balanced_split(mini_imagenet_train, n_total=1000,
    classes_to_keep={0,1,2,3,4,5,6,7,8,9})
print(mini_imagenet_train)

mini_imagenet_val = load_dataset("timm/mini-imagenet", split="validation[:10%]")
mini_imagenet_val = balanced_split(mini_imagenet_val, n_total=200,
    classes_to_keep={0,1,2,3,4,5,6,7,8,9})
print(mini_imagenet_val)

```

All classes in the ImageNet dataset: {0: 'house_finch', 1: 'robin', 2:

```

'triceratops', 3: 'green_mamba', 4: 'harvestman', 5: 'toucan', 6: 'goose', 7:
'jellyfish', 8: 'nematode', 9: 'king_crab', 10: 'dugong', 11: 'Walker_hound',
12: 'Ibizan_hound', 13: 'Saluki', 14: 'golden_retriever', 15: 'Gordon_setter',
16: 'komondor', 17: 'boxer', 18: 'Tibetan_mastiff', 19: 'French_bulldog', 20:
'malamute', 21: 'dalmatian', 22: 'Newfoundland', 23: 'miniature_poodle', 24:
'white_wolf', 25: 'African_hunting_dog', 26: 'Arctic_fox', 27: 'lion', 28:
'meerkat', 29: 'ladybug', 30: 'rhinoceros_beetle', 31: 'ant', 32: 'black-
footed_ferret', 33: 'three-toed_sloth', 34: 'rock_beauty', 35:
'aircraft_carrier', 36: 'ashcan', 37: 'barrel', 38: 'beer_bottle', 39:
'bookshop', 40: 'cannon', 41: 'carousel', 42: 'carton', 43: 'catamaran', 44:
'chime', 45: 'clog', 46: 'cocktail_shaker', 47: 'combination_lock', 48: 'crate',
49: 'cuirass', 50: 'dishrag', 51: 'dome', 52: 'electric_guitar', 53: 'file', 54:
'fire_screen', 55: 'frying_pan', 56: 'garbage_truck', 57: 'hair_slide', 58:
'holster', 59: 'horizontal_bar', 60: 'hourglass', 61: 'iPod', 62: 'lipstick',
63: 'miniskirt', 64: 'missile', 65: 'mixing_bowl', 66: 'oboe', 67: 'organ', 68:
'parallel_bars', 69: 'pencil_box', 70: 'photocopier', 71: 'poncho', 72:
'prayer_rug', 73: 'reel', 74: 'school_bus', 75: 'scoreboard', 76: 'slot', 77:
'snorkel', 78: 'solar_dish', 79: 'spider_web', 80: 'stage', 81: 'tank', 82:
'theater_curtain', 83: 'tile_roof', 84: 'tobacco_shop', 85: 'unicycle', 86:
'upright', 87: 'vase', 88: 'wok', 89: 'worm_fence', 90: 'yawl', 91:
'street_sign', 92: 'consomme', 93: 'trifle', 94: 'hotdog', 95: 'orange', 96:
'cliff', 97: 'coral_reef', 98: 'bolete', 99: 'ear'}
Dataset({
    features: ['image', 'label'],
    num_rows: 1000
})
Dataset({
    features: ['image', 'label'],
    num_rows: 200
})

```

2.3 Question 1b: Prepare the Dataset

2.3.1 PyTorch Dataset and DataLoaders

Now that we have our data, we need to prepare it for model training. When we train machine learning models, we need an efficient way to access our data (often in batches, which are subsets of our data that we pass into our model all at once to take advantage of our computer/server's full computation power). PyTorch's `Dataset` and `DataLoader` classes allow us to neatly encapsulate code for processing and accessing our data.

- PyTorch's `Dataset` class is an abstract class that represents a dataset—it stores the samples and their corresponding labels.
- Once we have a `Dataset`, we can pass it into a `DataLoader`, which wraps an iterable around the `Dataset` so we can easily access samples individually or in batches in our training loop.
- PyTorch has a great tutorial about `Datasets` and `DataLoaders` you can check out [here](#)

Why is PyTorch's `Dataset` class useful? 1. `Datasets` can integrate with PyTorch's `DataLoader` objects for batching, shuffling, and parallel loading. 2. You have more flexible control over how

data is loaded and preprocessed. 3. You can encapsulate all data logic in one place, making your training code cleaner. 4. The `Dataset` and `Dataloader`'s standard interface means your dataset works with any PyTorch training loop.

PyTorch provides many built-in datasets (MNIST, CIFAR-10, ImageNet, etc.) through `torchvision.datasets`, but you'll need a custom Dataset when:

- * Your data has a unique structure, such as being stored in a local folder on your computer.
- * The data has non-standard formats (e.g. medical imaging, satellite data, audio spectrograms).
- * You have special loading requirements, like combining data from multiple sources.
- * You need custom preprocessing logic.

In our case, since we're loading Mini ImageNet from HuggingFace and subsampling only 10 classes out of it, let's get some hands-on practice making our own custom dataset!

To create your own custom dataset, you need to inherit from `torch.utils.data.Dataset` and implement three key methods.

1. `__init__(self, ...)` 1. This is your custom dataset's constructor! Here, you should save your data, initialize any transforms/preprocessing, and set up any configurations you may need.
2. `__len__(self)` 1. Returns the total number of samples in your dataset. This is used by `DataLoader` to know when to stop iterating.
3. `__getitem__(self, idx)` 1. The most important method! Returns a single sample from your dataset at the given index `idx`. This is where you: 1. Load the actual data (e.g., index from an array, read image from disk). 2. Apply transformations. 3. Return the sample and its label.

Example:

```
import torch
from torch.utils.data import Dataset

class CustomDataset(Dataset):
    def __init__(self, custom_data, labels, preprocess=None):
        """
        Args:
            custom_data (str): list of data samples
            labels (list): List of labels corresponding to images
            preprocess (callable, optional): Optional preprocessing function to apply
        """
        self.data = custom_data
        self.labels = labels
        self.preprocess = preprocess

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        # Get the data at this index
        item = self.data[idx]
        # Get the label for this data point
        label = self.labels[idx]

        # Apply preprocessing
        if self.preprocess:
```

```

        item = self.preprocess(item)

    return item, label

```

Task: Implement the `__init__`, `__len__`, `__getitem__`, and `show_images` methods of the `MiniImageNetDataset`

- * `__init__`: * Save the input arguments (`dataset`, `class_id_to_name`, `split`, and `transform`) as instance attributes (`self.dataset`, etc.)
- * `__len__`: * Return the length of the dataset
- * `__getitem__`: * Get the item (image) and label in the HuggingFace dataset at the given index. * Make sure all the images have 3 channels—convert any grayscale images to RGB by calling `convert("RGB")`. * Apply the transform if it's not `None`. * Note that the standard series of transformations used on ImageNet models includes functions that expect a PIL image and return a tensor in the shape (C, H, W) with dtype `torch.float`. Otherwise, we will need to explicitly make sure that our image is in the correct format (a Tensor with dtype=`torch.float` and shape (C, H, W)) for training our model. * For this function, you can assume that if we apply the transform, we won't need to apply the other operations to cast the item into a tensor, permute it, and type-cast it because we will provide the standard ImageNet transforms for you later on!
- * Otherwise, if the transform is `None`, you should:
 - * Convert the item's image into a 3D numeric array.
 - * Cast the image into a `torch.Tensor` with dtype=`torch.float`.
 - * Permute the image into the shape (C, H, W).
 - * Cast the label to a `torch.Tensor` with dtype=`torch.long`.
- * Return the transformed image and label.
- * `show_images`: * Sample `num_images` random images from the dataset. * Display the images in a grid layout. * For each image, use `class_id_to_name` to get a readable class name and set the readable class name as the title of the image. * You can use `matplotlib` or `plotly` to display the images.

Hints:

- * It might be helpful to refer to the section on “Understanding HuggingFace Dataset Objects” to understand how to extract an image and its label from the dataset.
- * In the `__getitem__` method, we should make sure the item and label we return are both of type `torch.Tensor` and have the correct dtype. If the item and label aren't tensors with the correct dtype, you should:
 - * Cast the item to `torch.Tensor` with dtype=`torch.float` after the transform is applied.
 - * Cast the label to `torch.Tensor` with dtype=`torch.long`.
- * Note that PyTorch expects the inputs to the models to have dtype=`torch.float` to match the dtype of the model weights, and the labels to have dtype=`torch.long` when we pass them into loss functions.
- * Make sure that the final tensor your `__getitem__` method returns has the shape (C, H, W) where C = number of channels (usually 3 for R, G, and B), H = image height, and W = image width. * Images are usually represented as arrays in the shape (H, W, C) but libraries like PyTorch usually expect each image to have shape (C, H, W) to optimize the matrix math that goes on behind the scenes! For instance, when we visualize nd.arrays using Plotly or Matplotlib, your image arrays should have the shape (H, W, C). But when we want to use arrays/tensors to train machine learning models, your image arrays should have the shape (C, H, W). * `torch.permute` might be helpful.

```

[57]: class MiniImageNetDataset(Dataset):
    def __init__(
        self,
        dataset,
        class_id_to_name: dict[int, str],
        split: str = "train",
        transform: transforms.Compose = None,
    ):

```

```

"""
Args:
    dataset (datasets.Dataset): HuggingFace dataset object
    Each sample in the dataset has:
        - An "image" field that contains the image data
        - A "label" field that contains the numeric class ID
    class_id_to_name (dict): dictionary mapping numeric class IDs to
↳ class names
    split (str): "train" or "val" or "test"
"""
# TODO: Implement the __init__ method of the MiniImageNetDataset
self.split = split
self.dataset = dataset
self.transform = transform
self.class_id_to_name = class_id_to_name

def __len__(self):
    # TODO: Implement the __len__ method of the MiniImageNetDataset
    return len(self.dataset)

def __getitem__(self, index):
    """
    Returns a single sample from the dataset at the given index.
    Args:
        index (int): index of the item to get
    Returns:
        tuple: (image, label) where image is a tensor of shape C x H x W
↳ and label is a tensor of shape 1
    """
    item = self.dataset[index]['image'].convert("RGB")
    label = torch.tensor(self.dataset[index]['label'], dtype=torch.long)
    if self.transform is not None:
        item = self.transform(item)
    else:
        item = np.array(item)
        item = torch.tensor(item, dtype=torch.float)
        item = torch.permute(item, (2, 0, 1)) # permute to C X H X W instead
↳ of H x W x C

    # TODO: Implement the __getitem__ method of the MiniImageNetDataset
    return item, label

def show_images(self, num_images: int = 10):
    """
    Visualize images in a grid layout.

    Args:

```



```

        num_images (int): Number of images to display per class

    Returns:
        tuple: (fig, ax) - Matplotlib figure and axes objects
    """

    cols = 5
    rows = num_images / cols

    fig, axs = plt.subplots(rows, cols, figsize=(cols * 3, rows * 3))

    axs = axs.flatten()
    # TODO: Implement the show_images method
    # Make sure to call fig.show() or plt.show() to display your grid of
    ↪ images at the end!
    for i in range(num_images):
        im, label = self.getitem()
        name = self.class_id_to_name[label]

        axs[i].imshow(im)
        axs[i].set_title(name)
        axs[i].axis("off")

    # hide unused subplots
    for j in range(i + 1, len(axs)):
        axs[j].axis("off")

    plt.tight_layout()
    plt.show()

    return fig, axs

```

2.4 Question 1c: Create Datasets and DataLoaders

Using the custom Dataset class we implemented, let's create MiniImageNetDatasets that contain our training and validation data.

In our Dataset implementation, we specified a **transform** parameter that we can use to apply transformations to our images. For most vision-tasks, we often apply transformations to our images to help the model generalize better.

PyTorch has a **transforms** module that contains many built-in transformations you can use. Additionally, we can compose multiple transformations into a single **transforms.Compose** object.

For this assignment, we'll use the following transformations: * **transforms.Resize**: Resize the image to a smaller size * **transforms.RandomCrop**: Randomly crop the image to a smaller size * **transforms.RandomHorizontalFlip**: Randomly flip the image horizontally * **transforms.ToTensor**: Convert the image to a tensor

In the following block, we've defined the standard preprocessing pipeline for ImageNet models:

- * Resize images to 256 pixels on the shorter side.
- * Crop to 224×224 (the standard input size for most ImageNet-trained architectures like ResNet).
- * For training, we use data augmentation with random crops and horizontal flips to help the model generalize better.
- * For validation we use deterministic center crops to ensure consistent evaluation.
- * Lastly, we normalize the images using means and standard deviations computed from the entire ImageNet training set. The mean [0.485, 0.456, 0.406] and standard deviations [0.229, 0.224, 0.225] have 3 values, one for each channel (R, G, B). By normalizing our Mini-ImageNet data using these same statistics, we ensure that the pixel value distributions match what models expect if we were to use pretrained ImageNet weights for transfer learning. This normalization also helps with training stability by centering the data around zero.

Task: Create training and validation instances of our custom dataset class using the appropriate transformations.

- * Define a `train_dataset` that uses `mini_imagenet_train` as the dataset, `class_id_to_name` as the `class_id_to_name`, `split="train"`, and `transform=train_transform`.
- * Define a `val_dataset` that uses `mini_imagenet_val` as the dataset, `class_id_to_name` as the `class_id_to_name`, `split="val"`, and `transform=val_transform`.

```
[58]: train_transform = transforms.Compose([
    transforms.Resize(256),          # Resize the shorter side to 256
    transforms.RandomCrop(224),      # Random crop into shape 224×224
    transforms.RandomHorizontalFlip(), # Random horizontal flip
    transforms.ToTensor(),           # Convert to tensor, scale to [
    ↪ [0,1], and permute into shape (C, H, W)
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]) # Normalize
])

val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

train_dataset = ↪
    ↪ MiniImageNetDataset(mini_imagenet_train, class_id_to_name, split="train", transform=train_tran
val_dataset = ↪
    ↪ MiniImageNetDataset(mini_imagenet_val, class_id_to_name, split="val", transform=val_transform)
```

Then, we will create `DataLoaders` for our Datasets.

Task: Create a `dataloader` for our training, validation, and test datasets. Use a `batch_size` of 32, and set `shuffle=True`.

```
[59]: train_dataloader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_dataloader = DataLoader(val_dataset, batch_size=32, shuffle=True)
```

```

# Print the first batch of data and labels
for batch in train_dataloader: # each batch is a tuple of (data, labels)
    data = batch[0] # data is a tensor of shape (B, 3, 224, 224)
    labels = batch[1] # labels is a tensor of shape (B,)
    print(f"Shape of data: {data.shape}") # batch_size * channels * height *
    ↪width
    print(f"Shape of labels: {labels.shape}")
    break

```

Shape of data: torch.Size([32, 3, 224, 224])

Shape of labels: torch.Size([32])

2.5 Question 1d: Train your CNN

We are ready to train our CNN model! To train our CNN model, we need to write a training loop that passes inputs into our model, updates the weights of our model, and repeats!

Important terminology: * *epoch*: one complete pass through the entire training dataset * *batch*: a subset of samples we pass into the model at the same time * *optimizer*: what kind of optimization algorithm we want to use (e.g. Adam, Momentum) * *criterion*: the loss function we want to use to evaluate our model on

2.5.1 Training Loops

A training loop is where your model actually learns! Each iteration through the loop, the model:
 * Makes predictions * Calculates how wrong it was (loss) * Updates its weights to do better next time

2.5.2 PyTorch Training Loops: Step-by-Step Guide and Best Practices

1. Setup - Move model to device
 1. It's a good practice to begin by moving your model to the device (either `cuda` aka GPU or `cpu`). Your model and data must be on the same device. Moving the model to the GPU can speed up training.

```
model.to(device) # device = 'cuda' or 'cpu'
```

2. Outer Loop - Iterate for `num_epochs` epochs
 1. Models rarely learn perfectly in one pass. Multiple epochs let the model see the data repeatedly and gradually improve.

```

for epoch in range(num_epochs):
    # An epoch = one complete pass through the entire training dataset

```

3. Set the model to training mode before passing in training data
 1. Some layers (Dropout, BatchNorm) behave differently during training vs testing.

```
model.train()
```

4. Inner loop - Iterate over training batches
 1. Process data in batches to make the most of your GPU's computational power without overflowing memory.

2. Move the data to the same device as the model, and if needed, cast them to the appropriate dtypes.
 1. Data should have the same dtype as your model's parameters.
 2. Usually, loss functions expect the target labels to have `dtype=torch.long`.

```
for x, y in train_dataloader:
    # Process one batch at a time
    x = x.to(device, dtype=torch.float)
    y = y.to(device, dtype=torch.long)
```

5. Zero out the gradients - PyTorch accumulates gradients by default. If you don't reset them, old gradients will add to new ones, causing wrong updates! **Always** zero out the gradients BEFORE the forward pass of each batch.

```
optimizer.zero_grad()
```

6. Make a forward pass to get your model's predictions on the input data.

```
y_hat = model(x)
```

7. Compute the loss between your predictions and the target labels
 1. Loss measures how wrong your predictions are. Lower loss = better predictions.
 2. The criterion will return a single number representing the loss.

```
loss = criterion(y_hat, y)
```

8. Backward pass to compute gradients
 1. Call `loss.backward()` to compute gradients for each learnable weight in your model.

9. Update the weights

1. Call `optimizer.step()` to update the weights in your model based on the gradients computed during backpropagation!

10. Track training metrics (optional but a good practice!)

1. Track the training loss and training accuracy
 2. Since loss is a `torch.Tensor`, we can get its scalar value by calling `loss.item()`. Otherwise, we can't directly add a `torch.Tensor` (even if it's just a one-dimensional scalar!) to another Python primitive like `train_loss`, which might be a Python float.
 3. In multi-class classification, the model's output `y_hat` will be an array of length `num_classes` where each element is the probability that the given data point belongs to that class. To get the class with the highest probability, we can call `argmax` (returns the index corresponding to the maximum value). Remember that we are using batches in our training loop! So `y_hat` will be a tensor of shape `(batch_size, num_classes)`. By specifying `dim=1`, we can get the predicted class for each item in the batch. So `preds` will have shape `(batch_size, 1)` → one predicted class for each item in the batch.
 4. To compute the training accuracy, we can check when the predicted class equals the actual true targets `y` and sum up all the correct predictions. Since `preds` and `y` are both tensors, the expression `(preds == y)` and the sum is also a `torch.Tensor`. So similarly to the loss, we can get the scalar value of this tensor by calling `.item()`.

```
train_loss += loss.item() # Accumulate loss
preds = torch.argmax(y_hat, dim=1) # Get predicted class for each item in the batch
train_correct += (preds == y).sum().item() # Count correct predictions
```

11. Measure validation metrics after each epoch

1. Set the model to `eval` mode - Disables dropout, uses running stats for BatchNorm, etc.
2. Make predictions on the validation data, and measure validation loss and/or accuracy.
3. **Important note:** Make sure to do all your validation under a `with torch.no_grad()` header! This tells PyTorch not to keep track of forward pass operations or compute gradients during this code block. This saves memory and speeds up validation because we don't want to train the model during validation—we want to measure how it's performing.

Complete training loop structure (simplified)

```
# === SETUP ===
model.to(device)

# === EPOCH LOOP ===
for epoch in range(num_epochs):
    train_loss = 0.0
    train_correct = 0.0

    # === TRAINING PHASE ===
    model.train() # Enable training mode

    for x, y in train_dataloader:
        # 1. Move data to device
        x, y = x.to(device), y.to(device)

        # 2. Reset gradients
        optimizer.zero_grad()

        # 3. Forward pass
        y_hat = model(x)

        # 4. Compute loss
        loss = criterion(y_hat, y)

        # 5. Backward pass (compute gradients)
        loss.backward()

        # 6. Update weights
        optimizer.step()

        # 7. Track metrics (optional)
        # ...

    # === VALIDATION PHASE ===
    model.eval() # Disable training-specific layers
    with torch.no_grad(): # Don't compute gradients
        for x, y in val_dataloader:
            # Forward pass only, no backward pass!
            y_hat = model(x)
            loss = criterion(y_hat, y)
```

```
# Track validation metrics
```

2.5.3 How we calculate average loss and accuracy

- When we train a model, we want to report the average loss and average accuracy for each epoch, not just the total. This helps us understand how well the model is performing per sample, and lets us compare results across different epochs and experiments.

Loss Calculation * During each epoch, for each batch we encounter, we add the loss computed over all items in the batch to the total loss: `train_loss += loss.item()`. * At the end of the epoch, we divide by the number of batches: `train_loss /= len(train_dataloader)`. * Each batch's loss is an average over the samples in that batch, so dividing by the number of batches gives us the average loss per batch for the whole epoch.

Accuracy Calculation * We count the total number of correct predictions for all samples in the epoch: `train_correct += (preds == targets).sum().item()`. * At the end of the epoch, we divide by the total number of samples in the dataset: `train_acc = train_correct / len(train_dataloader.dataset)`. * This gives us the fraction of samples the model predicted correctly (i.e., the overall accuracy for the epoch).

In the following block, we've implemented a training loop for you!

Note: `tqdm` is a Python library used to display progress bars. In the `train` function we've implemented, we use it as a wrapper around iterators such as the dataloaders so that we can track the progress of how many epochs we've trained for. It's a nice-to-have, but when you implement your own training loops in the future, you can also simply iterate through the dataloaders like in the simplified example above too.

```
[60]: # Common practice to check if GPU is available
if torch.accelerator.is_available():
    device = torch.accelerator.current_accelerator()
else:
    device = "cpu"
print(f"Using device: {device}")
assert str(device) in ["cuda", "mps"], "Please make sure you are using an
↪accelerator (e.g. GPU or MPS)"

def train(
    model, optimizer, criterion, num_epochs, train_dataloader,
    ↪val_dataloader=None
):
    """
    Args:
        model: the model to train
        optimizer: the optimizer to use
        criterion: the loss function to use
        num_epochs: the number of epochs to train for
        train_dataloader: the dataloader for the training set
        val_dataloader: the dataloader for the validation set
```

```

"""
# === SETUP ===
model.to(device)

# == (Optional) Perform a validation loop before training as a baseline ==
if val_dataloader:
    model.eval() # Set model to evaluation mode
    num_correct = 0.0
    val_loss = 0.0
    with torch.no_grad(): # Don't compute gradients
        for x, y in val_dataloader:
            x = x.to(device, dtype=torch.float)
            y = y.to(device, dtype=torch.long)
            y_hat = model(x)

            loss = criterion(y_hat, y) # pred, target
            val_loss += loss.item()

            preds = torch.argmax(y_hat, dim=1)
            num_correct += (preds == y).sum().item()

    val_acc = num_correct / len(val_dataloader.dataset)
    tqdm.write(f"Initial validation loss: {val_loss:.4f}")
    tqdm.write(f"Initial validation accuracy: {val_acc:.4f}")

# Lists to store metrics across epochs
train_accuracies = []
train_losses = []
val_accuracies = []
val_losses = []

epoch_pbar = tqdm(range(num_epochs), desc="Training Progress", position=0)

# === EPOCH LOOP ===
for epoch in epoch_pbar:
    # === TRAINING PHASE ===
    model.train() # Set model to training mode

    # Initialize metrics for this epoch
    train_loss = 0.0
    train_correct = 0.0

    # === INNER LOOP (iterate over training batches) ===
    for x, y in train_dataloader:
        # 1. Move data to device
        x = x.to(device, dtype=torch.float)
        y = y.to(device, dtype=torch.long)

```

```

    # 2. Reset gradients
    optimizer.zero_grad()

    # 3. Forward pass
    y_hat = model(x)

    # 4. Compute loss
    loss = criterion(y_hat, y)

    # 5. Backward pass (compute gradients)
    loss.backward()

    # 6. Update weights
    optimizer.step()

    # 7. Track metrics
    train_loss += loss.item() # Accumulate loss (convert to Python
    ↪float)

    preds = torch.argmax(
        y_hat, dim=1
    ) # Get predicted class for each item in the batch (highest prob =
    ↪highest confidence)

    train_correct += (preds == y).sum().item() # Count correct
    ↪predictions

# === END OF INNER LOOP ===

# Compute average metrics for the epoch
avg_train_loss = train_loss / len(
    train_dataloader
) # Average loss = total loss / total number of batches
train_acc = (
    train_correct / len(train_dataloader.dataset)
) # Average accuracy = accuracy / total number of data points seen
    ↪across the whole epoch (i.e. in the entire dataset)
train_losses.append(avg_train_loss)
train_accuracies.append(train_acc)

print(f"\nEpoch {epoch + 1} - Training accuracy: {train_acc:.3f}")
print(f"Epoch {epoch + 1} - Training loss: {train_loss:.3f}")

# === END OF TRAINING PHASE ===

# === VALIDATION PHASE ===
if val_dataloader:
    model.eval() # Set model to evaluation mode

```



```

val_loss = 0.0
val_correct = 0.0

with torch.no_grad(): # Don't compute gradients
    # === INNER LOOP (iterate over validation batches) ===
    for x, y in val_dataloader:
        # 1. Move data to device
        x = x.to(device, dtype=torch.float)
        y = y.to(device, dtype=torch.long)

        # 2. Forward pass only, no backward pass!
        y_hat = model(x)

        # 3. Compute loss
        loss = criterion(y_hat, y)

        # 4. Track validation metrics
        val_loss += loss.item()
        preds = torch.argmax(y_hat, dim=1)
        val_correct += (preds == y).sum().item()

    # === END OF INNER LOOP ===

    # Compute average metrics for the epoch
    avg_val_loss = val_loss / len(val_dataloader)
    val_acc = val_correct / len(val_dataloader.dataset)

    val_losses.append(avg_val_loss)
    val_accuracies.append(val_acc)

# === END OF VALIDATION PHASE ===

# === END OF EPOCH LOOP ===

# Print metrics for the epoch
print(f"Epoch {epoch + 1} - Validation accuracy: {val_acc:.3f}")
print(f"Epoch {epoch + 1} - Validation loss: {val_loss:.3f}")

epoch_pbar.set_postfix(
    {
        "train_loss": f"{train_loss:.3f}",
        "train_acc": f"{train_acc:.3f}",
        "val_loss": f"{val_loss:.3f}",
        "val_acc": f"{val_acc:.3f}",
    }
)

```

```
return train_accuracies, train_losses, val_accuracies, val_losses
```

Using device: cuda

Task: 1. Instantiate a CNN model which will classify which of the 10 classes from our Mini Imagenet dataset an image belongs to. 2. Use AdamW as the optimizer. 1. [PyTorch documentation for AdamW](#) 3. Use CrossEntropyLoss as our criterion. 1. [PyTorch documentation for CrossEntropyLoss](#) 4. Train your CNN model for **20** epochs! You should be able to reach **~50% training accuracy** after 20 epochs.

Hints: * When you instantiate an optimizer, you also need to pass in what parameters you want this optimizer to update. You can access the parameters of any class that inherits from `nn.Module` by calling its `.parameters()` function! * Make sure that you turn on a GPU runtime before training. It may take a few minutes to complete all 20 epochs of training. For reference, it took me (Deena) about 15 seconds to train my model for 20 epochs on Google Colab's T4 GPU :)

Notes: * You may notice that the validation set's training accuracy lags below the training accuracy or jumps around a lot. Since we're using a toy set with only 1,000 training images and 200 validation images, that is to be expected! These metrics are an indicator that our model might be overfitting to our training set.

```
[61]: # TODO: Instantiate a CNN model
      cnn = CNN(num_classes=10)

      # TODO: Set the number of epochs
      num_epochs = 20

      # TODO: Set the optimizer
      optimizer = AdamW(cnn.parameters(),lr=0.0003, betas=(0.9, 0.999))

      # TODO: Set the loss criterion
      criterion = torch.nn.CrossEntropyLoss()

      # TODO: Train the model
      cnn_train_accuracies, cnn_train_losses, cnn_val_accuracies, cnn_val_losses = ␣
      ↪train(cnn,optimizer,criterion,num_epochs,train_dataloader,val_dataloader)
```

Initial validation loss: 16.1283

Initial validation accuracy: 0.1000

Training Progress: 0%| | 0/20 [00:00<?, ?it/s]

Epoch 1 - Training accuracy: 0.245

Epoch 1 - Training loss: 67.811

Training Progress: 5%| | 1/20 [00:10<03:18, 10.45s/it,
train_loss=67.811, train_acc=0.245, val_loss=13.910, val_acc=0.265]

Epoch 1 - Validation accuracy: 0.265

Epoch 1 - Validation loss: 13.910

Epoch 2 - Training accuracy: 0.340
Epoch 2 - Training loss: 59.979
Training Progress: 10%| | 2/20 [00:20<03:07, 10.41s/it,
train_loss=59.979, train_acc=0.340, val_loss=12.961, val_acc=0.370]
Epoch 2 - Validation accuracy: 0.370
Epoch 2 - Validation loss: 12.961
Epoch 3 - Training accuracy: 0.373
Epoch 3 - Training loss: 58.239
Training Progress: 15%| | 3/20 [00:31<02:59, 10.55s/it,
train_loss=58.239, train_acc=0.373, val_loss=12.795, val_acc=0.345]
Epoch 3 - Validation accuracy: 0.345
Epoch 3 - Validation loss: 12.795
Epoch 4 - Training accuracy: 0.405
Epoch 4 - Training loss: 54.402
Training Progress: 20%| | 4/20 [00:41<02:47, 10.48s/it,
train_loss=54.402, train_acc=0.405, val_loss=11.856, val_acc=0.385]
Epoch 4 - Validation accuracy: 0.385
Epoch 4 - Validation loss: 11.856
Epoch 5 - Training accuracy: 0.458
Epoch 5 - Training loss: 50.395
Training Progress: 25%| | 5/20 [00:52<02:35, 10.39s/it,
train_loss=50.395, train_acc=0.458, val_loss=11.658, val_acc=0.400]
Epoch 5 - Validation accuracy: 0.400
Epoch 5 - Validation loss: 11.658
Epoch 6 - Training accuracy: 0.462
Epoch 6 - Training loss: 50.435
Training Progress: 30%| | 6/20 [01:02<02:25, 10.38s/it,
train_loss=50.435, train_acc=0.462, val_loss=11.565, val_acc=0.355]
Epoch 6 - Validation accuracy: 0.355
Epoch 6 - Validation loss: 11.565
Epoch 7 - Training accuracy: 0.479
Epoch 7 - Training loss: 48.748
Training Progress: 35%| | 7/20 [01:12<02:14, 10.37s/it,
train_loss=48.748, train_acc=0.479, val_loss=12.270, val_acc=0.430]
Epoch 7 - Validation accuracy: 0.430
Epoch 7 - Validation loss: 12.270

Epoch 8 - Training accuracy: 0.500
Epoch 8 - Training loss: 47.414
Training Progress: 40%| | 8/20 [01:23<02:04, 10.36s/it,
train_loss=47.414, train_acc=0.500, val_loss=12.364, val_acc=0.425]
Epoch 8 - Validation accuracy: 0.425
Epoch 8 - Validation loss: 12.364
Epoch 9 - Training accuracy: 0.499
Epoch 9 - Training loss: 46.772
Training Progress: 45%| | 9/20 [01:33<01:53, 10.35s/it,
train_loss=46.772, train_acc=0.499, val_loss=11.932, val_acc=0.460]
Epoch 9 - Validation accuracy: 0.460
Epoch 9 - Validation loss: 11.932
Epoch 10 - Training accuracy: 0.519
Epoch 10 - Training loss: 44.773
Training Progress: 50%| | 10/20 [01:43<01:43, 10.30s/it,
train_loss=44.773, train_acc=0.519, val_loss=12.093, val_acc=0.415]
Epoch 10 - Validation accuracy: 0.415
Epoch 10 - Validation loss: 12.093
Epoch 11 - Training accuracy: 0.554
Epoch 11 - Training loss: 42.083
Training Progress: 55%| | 11/20 [01:54<01:32, 10.31s/it,
train_loss=42.083, train_acc=0.554, val_loss=11.909, val_acc=0.445]
Epoch 11 - Validation accuracy: 0.445
Epoch 11 - Validation loss: 11.909
Epoch 12 - Training accuracy: 0.538
Epoch 12 - Training loss: 42.798
Training Progress: 60%| | 12/20 [02:04<01:22, 10.31s/it,
train_loss=42.798, train_acc=0.538, val_loss=11.106, val_acc=0.450]
Epoch 12 - Validation accuracy: 0.450
Epoch 12 - Validation loss: 11.106
Epoch 13 - Training accuracy: 0.555
Epoch 13 - Training loss: 41.644
Training Progress: 65%| | 13/20 [02:14<01:12, 10.30s/it,
train_loss=41.644, train_acc=0.555, val_loss=13.118, val_acc=0.425]
Epoch 13 - Validation accuracy: 0.425
Epoch 13 - Validation loss: 13.118

Epoch 14 - Training accuracy: 0.566
Epoch 14 - Training loss: 41.756
Training Progress: 70%| | 14/20 [02:24<01:01, 10.28s/it,
train_loss=41.756, train_acc=0.566, val_loss=12.233, val_acc=0.480]
Epoch 14 - Validation accuracy: 0.480
Epoch 14 - Validation loss: 12.233
Epoch 15 - Training accuracy: 0.569
Epoch 15 - Training loss: 41.225
Training Progress: 75%| | 15/20 [02:35<00:51, 10.33s/it,
train_loss=41.225, train_acc=0.569, val_loss=10.741, val_acc=0.515]
Epoch 15 - Validation accuracy: 0.515
Epoch 15 - Validation loss: 10.741
Epoch 16 - Training accuracy: 0.573
Epoch 16 - Training loss: 39.788
Training Progress: 80%| | 16/20 [02:45<00:41, 10.32s/it,
train_loss=39.788, train_acc=0.573, val_loss=12.381, val_acc=0.480]
Epoch 16 - Validation accuracy: 0.480
Epoch 16 - Validation loss: 12.381
Epoch 17 - Training accuracy: 0.576
Epoch 17 - Training loss: 40.266
Training Progress: 85%| | 17/20 [02:56<00:31, 10.35s/it,
train_loss=40.266, train_acc=0.576, val_loss=11.187, val_acc=0.480]
Epoch 17 - Validation accuracy: 0.480
Epoch 17 - Validation loss: 11.187
Epoch 18 - Training accuracy: 0.596
Epoch 18 - Training loss: 39.038
Training Progress: 90%| | 18/20 [03:06<00:20, 10.39s/it,
train_loss=39.038, train_acc=0.596, val_loss=11.797, val_acc=0.520]
Epoch 18 - Validation accuracy: 0.520
Epoch 18 - Validation loss: 11.797
Epoch 19 - Training accuracy: 0.586
Epoch 19 - Training loss: 39.095
Training Progress: 95%| | 19/20 [03:16<00:10, 10.37s/it,
train_loss=39.095, train_acc=0.586, val_loss=11.572, val_acc=0.470]
Epoch 19 - Validation accuracy: 0.470
Epoch 19 - Validation loss: 11.572

Epoch 20 - Training accuracy: 0.579

Epoch 20 - Training loss: 37.341

Training Progress: 100%| | 20/20 [03:27<00:00, 10.36s/it,
train_loss=37.341, train_acc=0.579, val_loss=12.042, val_acc=0.520]

Epoch 20 - Validation accuracy: 0.520

Epoch 20 - Validation loss: 12.042

2.6 Question 1e: Visualize the CNN's Training Performance

Let's make some plots to see how our model's loss and accuracy (hopefully) improved over the course of its training!

Task: Implement the `plot_loss_and_accuracy` function below to plot the CNN's training loss, training accuracy (if provided), validation loss, and validation accuracy (if provided). * The loss/accuracy should be on the y-axis, and the number of epochs should be on the x-axis. * You can use Matplotlib or Plotly. * When only the training loss and validation loss are provided, your function should produce 2 total plots in total: (1) training loss, and (2) validation loss. * When all 4 metrics are provided, your function should produce 4 total plots in total: (1) training loss, (2) training accuracy, (3) validation loss, and (4) validation accuracy.

```
[100]: def plot_metrics(train_losses, val_losses, train_accuracies=None,
    ↪ val_accuracies=None, num_epochs=None, title=""):
    """
    Plots the training loss, training accuracy, validation loss, and validation
    ↪ accuracy.
    Args:
        train_losses: list of training losses
        val_losses: list of validation losses
        train_accuracies: list of training accuracies
        val_accuracies: list of validation accuracies
        num_epochs: number of epochs
        title: title of the plot
    """
    num_epochs = range(1, num_epochs + 1)
    if train_accuracies and val_accuracies:
        fig, axs = plt.subplots(2, 2, figsize=(12, 10))
        fig.suptitle(title, fontsize=16)
        axs = axs.flatten()

        # Training Loss
        axs[0].plot(num_epochs, train_losses, color = 'blue')
        axs[0].set_title("Training Loss")
        axs[0].set_xlabel("Epoch")
        axs[0].set_ylabel("Loss")
```

```

    axs[0].legend()
    axs[0].grid(True)

    # Validation Loss
    axs[1].plot(num_epochs, val_losses, color='orange')
    axs[1].set_title("Validation Loss")
    axs[1].set_xlabel("Epoch")
    axs[1].set_ylabel("Loss")
    axs[1].legend()
    axs[1].grid(True)

    # Training Accuracy
    axs[2].plot(num_epochs, train_accuracies, color='green')
    axs[2].set_title("Training Accuracy")
    axs[2].set_xlabel("Epoch")
    axs[2].set_ylabel("Accuracy")
    axs[2].legend()
    axs[2].grid(True)

    # Validation Accuracy
    axs[3].plot(num_epochs, val_accuracies, color='red')
    axs[3].set_title("Validation Accuracy")
    axs[3].set_xlabel("Epoch")
    axs[3].set_ylabel("Accuracy")
    axs[3].legend()
    axs[3].grid(True)

else:
    fig, axs = plt.subplots(1, 2, figsize=(12, 5))
    fig.suptitle(title, fontsize=16)

    axs = axs.flatten()

    # Training Loss
    axs[0].plot(num_epochs, train_losses, color = 'blue')
    axs[0].set_title("Training Loss")
    axs[0].set_xlabel("Epoch")
    axs[0].set_ylabel("Loss")
    axs[0].legend()
    axs[0].grid(True)

    # Validation Loss
    axs[1].plot(num_epochs, val_losses, color='orange')
    axs[1].set_title("Validation Loss")
    axs[1].set_xlabel("Epoch")
    axs[1].set_ylabel("Loss")
    axs[1].legend()

```

```

    axs[1].grid(True)

    plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Adjust layout to prevent title
    ↪overlap
    plt.show()

```

3 Subpart 2: The ResNet Architecture

Deep neural networks can be incredibly powerful and expressive, but they can also be difficult to train! There are a few problems that deep neural networks encounter: * Vanishing gradients: Earlier layers receive very small gradients because the upstream gradients get multiplied by many local gradients. Thus, during gradient descent, their weights move by very small amounts and they don't "learn" as effectively. * Accuracy degradation: Adding more layers to a deep model causes accuracy to saturate, contributing to higher *training error* and *test error* compared to a shallower network with less layers.

Normalization can help address the vanishing gradient problem. But accuracy degradation can still persist in deep networks.

3.1 Residual Learning

In 2015, He et al. proposed a solution to the degradation problem by introducing the concept of **residual learning**.

[Deep Residual Learning for Image Recognition](#) Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun

Neural networks can be thought of as being composed of many different "blocks", which are just 1 or more layers (e.g. linear layers, activation functions, convolution layers, etc.) we think of as a single group.

Recall that we can view the blocks in our neural networks as functions mapping some input x to some output z (note that z might not be the final output of our neural network like a class prediction, but could also be the input into another hidden layer).

Let's denote the ground truth function we want this block in our neural network to learn as f , where $f(x) = z$.

Instead of forcing the layer(s) in our block to learn $f(x)$, we let them learn another mapping $g(x)$ where $g(x) = f(x) - x$. Then, we can "add in" our original input x to the output of $g(x)$ to achieve our desired mapping $f(x)$!

The authors showed that with **residual learning**: 1. Deep residual networks were easier to optimize than deep networks without residual connections 2. Residual networks demonstrate significant accuracy gains even as they get deeper, avoiding the degradation problem 3. They were able to win 1st place awards in the ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation in ILSVRC & COCO 2015 competitions.

3.2 Benefits of Residual Learning

You might ask why we want to do this! Aren't we just trying to learn some other function $g(x)$ now? Why are we complicating things by subtracting x from $f(x)$ if we have to add it back in anyways?

There are a few intuitions behind why the ResNet architecture is very powerful!

Reason 1: Adding a residual connection lets more of the upstream gradient flow backwards

Imagine layer N , whose input is x and whose output is z . Denote the upstream gradient that backpropagates from the loss L to the current layer N as $\frac{\partial L}{\partial z}$. Without a residual connection, the downstream gradient layer N propagates down to layer $N - 1$ is: $\frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial x}$.

With a residual connection, the downstream gradient that layer N propagates down to layer $N - 1$ is now $\frac{\partial L}{\partial z} \cdot (\frac{\partial z}{\partial x} + 1)$. Thus, even if the local gradient term $\frac{\partial z}{\partial x}$ is very small, the $+ 1$ ensures that gradients don't vanish completely and allow information to flow backwards more effectively!

Reason 2: Blocks can learn residual mappings!

Source: Dive Into Deep Learning

Let's zoom in on a block of a neural network (outlined by a dotted box), with input x . We assume that $f(x)$, the desired ground truth mapping we want the block to learn, will be used as the input to the activation function at the top. The "weight layers" represented by the white rectangles can be fully connected linear layers or convolutional layers.

On the left, we have a block without a residual connection. The dotted-line box must directly learn $f(x)$. On the right, the layers in the dotted-line box have to learn the **residual mapping** $g(x) = f(x) - x$.

If the actual desired function we want to learn is the identity function ($f(x) = x$), then the residual mapping is just $g(x) = 0$ very easy for the layers on the right to learn—we just need to push the weights and biases of the "weight layer" to 0.

In conclusion, sometimes, a neural network has already learned everything it needs from its data to perform its task! Adding additional layers is like forcing the network to keep "squeezing" more information out from the data. Using a residual connection lets a network block decide whether or not it might be more effective to simply pass only x forward, without applying additional non-linearities onto it.

3.3 Question 2a: Implement a Residual Block in PyTorch

Task: Use PyTorch to instantiate a Residual Block with the following architecture: `> * Convolution layer 1 with 64 channels, 3x3 kernel, padding = 1, stride = 1 (set stride = 2 if downsampling)`
`> * Batchnorm` `> * ReLU activation` `> * Convolution layer 2 with 64 channels, 3x3 kernel, padding = 1, stride = 1` `> * Batchnorm` `> * Residual AKA skip connection` `> * ReLU activation`

Notes about downsampling: * **Downsampling** refers to reducing the size of the feature map. For instance, let's say our original input to this layer was a 28 x 28 image. We can downsample it to a 14 x 14 image, essentially halving the H and W. Downsampling is often combined with an increase in the number of output channels. The main benefits of downsampling are 1) reducing computation, 2) increasing the receptive field of the output units (i.e. how many input pixels each

output unit “looks” at), and 3) enabling hierarchical feature learning (i.e. letting the model learn fine details at high resolutions, then reducing the resolution by making the input that gets passed down to later layers of a smaller size so that the model can learn bigger-picture details at a lower resolution). Many ResNet implementations downsample the input in the first convolution layer. Downsampling can be implemented by using a stride of 2 instead of a stride of 1 in a Conv2d layer. This means the kernel will shift over by 2 pixels each time instead of 1. * When downsampling, we can’t directly add the input to the output of the convolution layers anymore because the dimensions won’t match! Instead, we can also perform downsampling during the skip connection by applying a 1x1 convolution with stride=2 on the skip connection to match the shape of the convolution output before adding them together.

Hints: 1. Use separate batchnorm layers after each convolution layer! 2. `BatchNorm2d` normalizes each channel independently over the batch and spatial dimensions, so it needs one γ and β per channel. When initializing a `BatchNorm2d` layer, you should pass in the number of input channels as the `num_features` argument. 3. The residual connection and the output of the 2 convolutions should have the same shape (i.e. the same C, H, and W dimensions). What is the shape of the output after the 2 convolutions? 1. To make sure the dimensions match, you can apply a 1x1 convolution to the input x . Think about how a 1x1 convolution affects the H, W of the input, and how you can modify it to match the C dimension of the convolution output. 4. If we want to downsample an image, we typically apply a stride of 2 in the first conv layer. If `initial_downsample = True`, make sure to adjust the stride in your conv layers in the `__init__` method appropriately! Also make sure to use a stride of 2 in the residual connection if we downsample.

Helpful website to calculate the output shapes after a convolution:
<https://madebyollin.github.io/convnet-calculator/>

```
[63]: class ResidualBlock(nn.Module):
    def __init__(
        self,
        in_channels=3,
        out_channels=64,
        kernel_size=3,
        initial_downsample=False, # whether to downsample the input
        verbose=False, # whether to print debug statements
    ):
        super().__init__()
        self.verbose = verbose

        # TODO: Define the first conv layer
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride=1 if not
↪ initial_downsample else 2, padding=1)
        # 64 x 224 x 224 if no downsample
        # 64 x 112 x 112 if downsample

        # TODO: Define the first batchnorm layer
        self.batchnorm1 = nn.BatchNorm2d(num_features=out_channels)
```

```

# TODO: Define the second conv layer
self.conv2 = nn.Conv2d(out_channels,out_channels,3,stride=1,padding=1)
# 64 x 224 x 224 if no downsample
# 64 x 112 x 112 if downsample

# TODO: Define the second batchnorm layer
self.batchnorm2 = nn.BatchNorm2d(num_features=out_channels)
# TODO: Define the residual connection
if initial_downsample:
    self.residual_connection = nn.
↳Conv2d(in_channels,out_channels,1,stride=2,padding=0)
else:
    self.residual_connection = nn.
↳Conv2d(in_channels,out_channels,1,stride=1,padding=0)

def forward(self, x):
    # TODO: Implement the forward pass
    conv1 = self.conv1(x)
    if self.verbose:
        print(f"conv1 shape: {conv1.shape}")
    bn1 = self.batchnorm1(conv1)
    if self.verbose:
        print(f"bn1 shape: {bn1.shape}")
    z1 = F.relu(bn1)
    if self.verbose:
        print(f"z1 shape: {z1.shape}")
    conv2 = self.conv2(z1)
    if self.verbose:
        print(f"conv2 shape: {conv2.shape}")
    bn2 = self.batchnorm2(conv2)
    if self.verbose:
        print(f"bn2 shape: {bn2.shape}")
    residual = self.residual_connection(x)
    if self.verbose:
        print(f"residual shape: {residual.shape}")

    out = bn2 + residual
    # Don't forget to apply ReLU activation after adding the residual
    out = F.relu(out)
    return out

```

3.3.1 Check: Residual Block output shape

Below, we create a tensor of shape (1, 224, 224, 3) filled with random numbers between [0, 1] and pass it through our ResidualBlock. Let's do some math to determine what the expected dimensions of our ResidualBlock output should be!

Let's say we instantiate a ResidualBlock with 3 input channels, 64 output channels per hidden

convolution, a kernel size of 3, and initial downsampling during the first convolution. These would be the shapes after each layer: 1. Input: 1 x 3 x 224 x 224 -> dimensions: Batch size x 3 RGB channels x height of 224 pixels x width of 224 pixels 2. After conv1: 1 x 64 x 112 x 112 -> the first convolution downsamples the image to half its original height and width, and also increases the number of channels to 64 3. After bn1: 1 x 64 x 112 x 112 -> batchnorm doesn't change the output dimensions 4. After the first ReLU: 1 x 64 x 112 x 112 -> ReLU doesn't change the output dimensions 5. After conv2: 1 x 64 x 112 x 112 -> the second conv has padding = 1. This is called *same padding* because we pad the input so that the output of this layer has the same dimensions. 6. After batch norm 2: 1 x 64 x 112 x 112 -> batchnorm doesn't change the output dimensions 7. After adding the residual: 1 x 64 x 112 x 112 8. After the final ReLU: 1 x 64 x 112 x 112 -> ReLU doesn't change the output dimensions

You should verify that the shape of your actual output is also 1 x 64 x 112 x 112!

```
[64]: x = torch.rand((1, 3, 224, 224)) # 1 image, 3 RGB channels, 224 x 224 pixels
print(f"Shape of input: {x.shape}")

res = ResidualBlock(in_channels=3, out_channels=64, kernel_size=3,
    ↪initial_downsample=True, verbose=False)
resblock_out = res(x)
print(f"Shape of output: {resblock_out.shape}")
assert resblock_out.shape == torch.Size([1, 64, 112, 112]), f"Expected output_
    ↪shape (1, 64, 112, 112), but got {resblock_out.shape}"
```

```
Shape of input: torch.Size([1, 3, 224, 224])
Shape of output: torch.Size([1, 64, 112, 112])
```

```
[65]: # Verify the shapes of the weights
conv1_shape = res.conv1.weight.shape
print(f"Shape of conv1 weight: {conv1_shape}")
assert conv1_shape == torch.Size([64, 3, 3, 3]), f"Expected Conv1 weights to_
    ↪have shape (64, 3, 3, 3), but got {conv1_shape}"

conv2_shape = res.conv2.weight.shape
print(f"Shape of conv2 weight: {conv2_shape}")
assert conv2_shape == torch.Size([64, 64, 3, 3]), f"Expected Conv2 weights to_
    ↪have shape (64, 64, 3, 3), but got {conv2_shape}"

residual_shape = res.residual_connection.weight.shape
print(f"Shape of residual weight: {residual_shape}")
assert residual_shape == torch.Size([64, 3, 1, 1]), f"Expected residual_
    ↪connection to have shape (64, 3, 1, 1), but got {residual_shape}"
```

```
Shape of conv1 weight: torch.Size([64, 3, 3, 3])
Shape of conv2 weight: torch.Size([64, 64, 3, 3])
Shape of residual weight: torch.Size([64, 3, 1, 1])
```

3.4 Question 2b: ResNet-18(9)

Awesome! We've put together one building block of a ResNet. Now, let's implement a full ResNet-18 architecture using PyTorch.

ResNet-18 is an implementation of ResNet that has 18 total learnable layers: an initial 7x7 convolution, a series of Residual Blocks with 2 convolutions each, and a final fully connected layer for classification. There also exist other models in the ResNet family such as ResNet34, ResNet 50, and even ResNet 152!

3.4.1 ResNet 18 Architecture

ResNet-18 has 4 main stages, each with a different number of residual blocks. You should use the `ResidualBlock` class you implemented above to build the ResNet-18 architecture.

1. Initial Convolution Layer

- **Convolution Layer:** 3 input channels, 64 output channels, kernel size 7×7 , stride 2, padding 3
- **BatchNorm:** Normalize 64 channels
- **ReLU Activation:** Apply ReLU after BatchNorm
- **MaxPooling:** 3×3 kernel, stride 2, padding 1

2. Stage 1: 2 Residual Blocks (64 channels)

- **Residual Block 1:**
 - Input: 64 channels $\rightarrow$ Output: 64 channels
 - Kernel size: 3×3 , stride 1
 - Include skip connection with 1×1 conv for dimension matching
- **Residual Block 2:**
 - Input: 64 channels $\rightarrow$ Output: 64 channels
 - Kernel size: 3×3 , stride 1
 - Include skip connection

3. Stage 2: 2 Residual Blocks (128 channels)

- **Residual Block 3:**
 - Input: 64 channels $\rightarrow$ Output: 128 channels
 - Kernel size: 3×3 , stride 2 (downsampling)
 - Include skip connection with 1×1 conv for dimension matching
- **Residual Block 4:**
 - Input: 128 channels $\rightarrow$ Output: 128 channels
 - Kernel size: 3×3 , stride 1
 - Include skip connection

4. Stage 3: 2 Residual Blocks (256 channels)

- **Residual Block 5:**

- Input: 128 channels $\rightarrow$ Output: 256 channels
- Kernel size: 3×3 , stride 2 (downsampling)
- Include skip connection with 1×1 conv for dimension matching
- **Residual Block 6:**
 - Input: 256 channels $\rightarrow$ Output: 256 channels
 - Kernel size: 3×3 , stride 1
 - Include skip connection

5. Stage 4: 2 Residual Blocks (512 channels)

- **Residual Block 7:**
 - Input: 256 channels $\rightarrow$ Output: 512 channels
 - Kernel size: 3×3 , stride 2 (downsampling)
 - Include skip connection with 1×1 conv for dimension matching
- **Residual Block 8:**
 - Input: 512 channels $\rightarrow$ Output: 512 channels
 - Kernel size: 3×3 , stride 1
 - Include skip connection

6. Classification Head

- **Global Average Pooling:** 4×4 kernel, stride 1 (or `AdaptiveAvgPool2d`)
- **Flatten Layer:** Convert the the output of the global average pooling to a 1D value for each channel
- **Fully Connected Layer:** 512 input features $\rightarrow$ `num_classes` output

Key Implementation Notes

- **Downsampling:** The first residual block in stages 2, 3, and 4 use `stride=2` to downsample the input while simultaneously increasing the number of channels. Combining downsampling with an increase in the channel count allows the network to extract more high-level features and reduces computational costs because smaller feature maps are faster to process.
 - **Channel Changes:** The first residual block in stages 2-4 double the channel count of the input
 - **Skip Connections:** Use 1×1 convolutions when the number of input/output channels differ
-

Task: Fill in the `__init__` and `forward` method of the `ResNet18` class.

Hints: 1. `nn.Sequential` is a very helpful PyTorch class when we need to group together many layers that we want to apply one after another. Try using `nn.Sequential` to group together different blocks in each stage! You can find the docs and examples for `nn.Sequential` [here](#).

2. Note that the stages are composed of multiple residual blocks in sequence. In the ResNet-18 implementation, whenever we want to downsample or change the number of channels, we do that in the first residual block of each stage.
3. You can use `AdaptiveAvgPool2d` to get a 1×1 feature map for the final fully-connected layer without having to specify the `input_size`, `kernel_size`, `stride`, `padding`, etc. explicitly.

4. You can use `nn.Flatten()` to flatten the output of the global average pooling layer to a 1D value for each channel.

```
[66]: class ResNet18(nn.Module):
    def __init__(self, num_classes, verbose=False):

        # TODO: Instantiate all the layers/stages of ResNet-18
        # Hint: don't forget to call super().__init__() first!
        super().__init__()
        self.verbose = verbose

        # INITIAL CONVOLUTIONAL LAYER
        self.convLayer = nn.Sequential(
            nn.Conv2d(3,64,7,stride=2,padding=3),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(3,stride=2,padding=1)
        )

        # STAGE 1
        self.res1 = nn.Sequential(
            ResidualBlock(64,64,3),
            ResidualBlock(64,64,3)
        )
        # SKIP CONNECTION 1
        self.skip1 = nn.Conv2d(64,64,1,stride=1,padding=0)

        # STAGE 2
        self.res2 = nn.Sequential(
            ResidualBlock(64,128,3, initial_downsample=True),
            ResidualBlock(128,128,3)
        )
        # SKIP CONNECTION 2
        self.skip2 = nn.Conv2d(64,128,1,stride=2,padding=0)

        # STAGE 3
        self.res3 = nn.Sequential(
            ResidualBlock(128,256,3,initial_downsample=True),
            ResidualBlock(256,256,3)
        )
        # SKIP CONNECTION 3
        self.skip3 = nn.Conv2d(128,256,1,stride=2,padding=0)

        # STAGE 4
        self.res4 = nn.Sequential(
            ResidualBlock(256,512,3,initial_downsample=True),
            ResidualBlock(512,512,3)
```

```

    )
    # SKIP CONNECTION 4
    self.skip4 = nn.Conv2d(256,512,1,stride=2,padding=0)

    # Classification Head
    self.classHead = nn.Sequential(
        nn.AdaptiveAvgPool2d((1,1)),
        nn.Flatten(),
        nn.Linear(512,num_classes)
    )
    def forward(self, x):

        # TODO: Implement the forward pass of ResNet-18
        conv1 = self.convLayer(x)
        if self.verbose:
            print(f"conv1 shape: {conv1.shape}")
        res1 = self.res1(conv1) + self.skip1(conv1)
        if self.verbose:
            print(f"res1 shape: {res1.shape}")
        res2 = self.res2(res1) + self.skip2(res1)
        if self.verbose:
            print(f"res2 shape: {res2.shape}")
        res3 = self.res3(res2) + self.skip3(res2)
        if self.verbose:
            print(f"res3 shape: {res3.shape}")
        res4 = self.res4(res3) + self.skip4(res3)
        if self.verbose:
            print(f"res4 shape: {res4.shape}")
        out = self.classHead(res4)

        return out

```

3.4.2 Check: ResNet output shape

Once again, let's check the output shape of our ResNet.

What should be the output's expected shape?

If the dimensions and channels of all our intermediate convolution layers match up, we should get an output with shape (Batch_size, 10) since we specified 10 as the number of classes!

```

[67]: x = torch.rand((1, 3, 224, 224)) # 1 image, 3 RGB channels, 224 x 224 pixels
      print(f"Shape of input: {x.shape}")

      resnet18 = ResNet18(num_classes=10, verbose=True)
      resnet18_out = resnet18(x)
      print(f"Shape of output: {resnet18_out.shape}")

```



```
assert resnet18_out.shape == torch.Size([1, 10]), f"Expected output shape (1, 10), but got {resnet18_out.shape}"
```

```
Shape of input: torch.Size([1, 3, 224, 224])
conv1 shape: torch.Size([1, 64, 56, 56])
res1 shape: torch.Size([1, 64, 56, 56])
res2 shape: torch.Size([1, 128, 28, 28])
res3 shape: torch.Size([1, 256, 14, 14])
res4 shape: torch.Size([1, 512, 7, 7])
Shape of output: torch.Size([1, 10])
```

3.5 Question 2c: Let the training begin!

Task: 1. Instantiate a ResNet18 model which will classify which of the 10 classes from our Mini Imagenet dataset an image belongs to. 2. Use AdamW as the optimizer. 1. [PyTorch documentation for AdamW](#) 3. Use CrossEntropyLoss as our criterion. 1. [PyTorch documentation for CrossEntropyLoss](#) 4. Train your ResNet18 model for **50** epochs! You should be able to reach **~70% training accuracy** and **~50% validation accuracy** after 50 epochs.

Hints: * When you instantiate an optimizer, you also need to pass in what parameters you want this optimizer to update. You can access the parameters of any class that inherits from `nn.Module` by calling its `.parameters()` function! * Make sure that you turn on a GPU runtime before training. It may take a few minutes to complete all 50 epochs of training. For reference, it took me (Deena) about 5-10 minutes to train my model for 50 epochs on Google Colab's T4 GPU :)

Notes: * You may notice that the validation set's training accuracy lags below the training accuracy or jumps around a lot. Since we're using a toy set with only 1,000 training images and 200 validation images, that is to be expected! These metrics are an indicator that our model might be overfitting to our training set.

```
[68]: # TODO: Instantiate a ResNet18 model
resnet = ResNet18(num_classes=10)

# TODO: Set the number of epochs
num_epochs = 50

# TODO: Instantiate an optimizer
optimizer = AdamW(resnet.parameters(), lr=0.0005)

# TODO: Instantiate a loss criterion
criterion = torch.nn.CrossEntropyLoss()

# TODO: Train the model

resnet_train_accuracies, resnet_train_losses, resnet_val_accuracies,
    ↪ resnet_val_losses =
    ↪ train(resnet, optimizer, criterion, num_epochs, train_dataloader, val_dataloader)
```

Initial validation loss: 16.1093

Initial validation accuracy: 0.1200

Training Progress: 0%| | 0/50 [00:00<?, ?it/s]

Epoch 1 - Training accuracy: 0.245

Epoch 1 - Training loss: 70.064

Training Progress: 2%| | 1/50 [00:13<10:51, 13.29s/it,
train_loss=70.064, train_acc=0.245, val_loss=14.492, val_acc=0.300]

Epoch 1 - Validation accuracy: 0.300

Epoch 1 - Validation loss: 14.492

Epoch 2 - Training accuracy: 0.304

Epoch 2 - Training loss: 61.466

Training Progress: 4%| | 2/50 [00:26<10:35, 13.23s/it,
train_loss=61.466, train_acc=0.304, val_loss=15.618, val_acc=0.305]

Epoch 2 - Validation accuracy: 0.305

Epoch 2 - Validation loss: 15.618

Epoch 3 - Training accuracy: 0.332

Epoch 3 - Training loss: 61.165

Training Progress: 6%| | 3/50 [00:39<10:22, 13.23s/it,
train_loss=61.165, train_acc=0.332, val_loss=12.659, val_acc=0.380]

Epoch 3 - Validation accuracy: 0.380

Epoch 3 - Validation loss: 12.659

Epoch 4 - Training accuracy: 0.342

Epoch 4 - Training loss: 59.005

Training Progress: 8%| | 4/50 [00:52<10:08, 13.24s/it,
train_loss=59.005, train_acc=0.342, val_loss=12.121, val_acc=0.345]

Epoch 4 - Validation accuracy: 0.345

Epoch 4 - Validation loss: 12.121

Epoch 5 - Training accuracy: 0.359

Epoch 5 - Training loss: 58.042

Training Progress: 10%| | 5/50 [01:06<09:56, 13.26s/it,
train_loss=58.042, train_acc=0.359, val_loss=12.421, val_acc=0.345]

Epoch 5 - Validation accuracy: 0.345

Epoch 5 - Validation loss: 12.421

Epoch 6 - Training accuracy: 0.387

Epoch 6 - Training loss: 54.936

Training Progress: 12%| | 6/50 [01:19<09:42, 13.24s/it,
train_loss=54.936, train_acc=0.387, val_loss=12.486, val_acc=0.365]

Epoch 6 - Validation accuracy: 0.365

Epoch 6 - Validation loss: 12.486

Epoch 7 - Training accuracy: 0.434

Epoch 7 - Training loss: 52.949

Training Progress: 14%| | 7/50 [01:32<09:28, 13.22s/it,
train_loss=52.949, train_acc=0.434, val_loss=12.505, val_acc=0.390]

Epoch 7 - Validation accuracy: 0.390

Epoch 7 - Validation loss: 12.505

Epoch 8 - Training accuracy: 0.435

Epoch 8 - Training loss: 52.653

Training Progress: 16%| | 8/50 [01:45<09:14, 13.21s/it,
train_loss=52.653, train_acc=0.435, val_loss=12.027, val_acc=0.410]

Epoch 8 - Validation accuracy: 0.410

Epoch 8 - Validation loss: 12.027

Epoch 9 - Training accuracy: 0.423

Epoch 9 - Training loss: 53.332

Training Progress: 18%| | 9/50 [01:59<09:02, 13.23s/it,
train_loss=53.332, train_acc=0.423, val_loss=12.957, val_acc=0.395]

Epoch 9 - Validation accuracy: 0.395

Epoch 9 - Validation loss: 12.957

Epoch 10 - Training accuracy: 0.445

Epoch 10 - Training loss: 49.893

Training Progress: 20%| | 10/50 [02:12<08:50, 13.25s/it,
train_loss=49.893, train_acc=0.445, val_loss=11.603, val_acc=0.410]

Epoch 10 - Validation accuracy: 0.410

Epoch 10 - Validation loss: 11.603

Epoch 11 - Training accuracy: 0.457

Epoch 11 - Training loss: 49.238

Training Progress: 22%| | 11/50 [02:25<08:37, 13.26s/it,
train_loss=49.238, train_acc=0.457, val_loss=12.243, val_acc=0.400]

Epoch 11 - Validation accuracy: 0.400

Epoch 11 - Validation loss: 12.243

Epoch 12 - Training accuracy: 0.480

Epoch 12 - Training loss: 48.345

Training Progress: 24%| | 12/50 [02:39<08:24, 13.28s/it,
train_loss=48.345, train_acc=0.480, val_loss=11.734, val_acc=0.435]

Epoch 12 - Validation accuracy: 0.435

Epoch 12 - Validation loss: 11.734

Epoch 13 - Training accuracy: 0.516

Epoch 13 - Training loss: 45.043

Training Progress: 26%| | 13/50 [02:52<08:09, 13.23s/it,
train_loss=45.043, train_acc=0.516, val_loss=10.982, val_acc=0.490]

Epoch 13 - Validation accuracy: 0.490

Epoch 13 - Validation loss: 10.982

Epoch 14 - Training accuracy: 0.516

Epoch 14 - Training loss: 44.117

Training Progress: 28%| | 14/50 [03:05<07:56, 13.25s/it,
train_loss=44.117, train_acc=0.516, val_loss=12.002, val_acc=0.465]

Epoch 14 - Validation accuracy: 0.465

Epoch 14 - Validation loss: 12.002

Epoch 15 - Training accuracy: 0.508

Epoch 15 - Training loss: 45.483

Training Progress: 30%| | 15/50 [03:18<07:43, 13.25s/it,
train_loss=45.483, train_acc=0.508, val_loss=11.747, val_acc=0.460]

Epoch 15 - Validation accuracy: 0.460

Epoch 15 - Validation loss: 11.747

Epoch 16 - Training accuracy: 0.551

Epoch 16 - Training loss: 41.742

Training Progress: 32%| | 16/50 [03:31<07:30, 13.24s/it,
train_loss=41.742, train_acc=0.551, val_loss=9.531, val_acc=0.510]

Epoch 16 - Validation accuracy: 0.510

Epoch 16 - Validation loss: 9.531

Epoch 17 - Training accuracy: 0.559

Epoch 17 - Training loss: 41.221

Training Progress: 34%| | 17/50 [03:45<07:18, 13.28s/it,
train_loss=41.221, train_acc=0.559, val_loss=10.340, val_acc=0.510]

Epoch 17 - Validation accuracy: 0.510

Epoch 17 - Validation loss: 10.340

Epoch 18 - Training accuracy: 0.567

Epoch 18 - Training loss: 40.208

Training Progress: 36%| | 18/50 [03:58<07:03, 13.23s/it,
train_loss=40.208, train_acc=0.567, val_loss=9.794, val_acc=0.515]

Epoch 18 - Validation accuracy: 0.515
Epoch 18 - Validation loss: 9.794

Epoch 19 - Training accuracy: 0.592
Epoch 19 - Training loss: 37.978

Training Progress: 38%| | 19/50 [04:11<06:50, 13.24s/it,
train_loss=37.978, train_acc=0.592, val_loss=11.892, val_acc=0.450]

Epoch 19 - Validation accuracy: 0.450
Epoch 19 - Validation loss: 11.892

Epoch 20 - Training accuracy: 0.620
Epoch 20 - Training loss: 36.849

Training Progress: 40%| | 20/50 [04:24<06:37, 13.26s/it,
train_loss=36.849, train_acc=0.620, val_loss=12.333, val_acc=0.455]

Epoch 20 - Validation accuracy: 0.455
Epoch 20 - Validation loss: 12.333

Epoch 21 - Training accuracy: 0.599
Epoch 21 - Training loss: 37.045

Training Progress: 42%| | 21/50 [04:38<06:26, 13.33s/it,
train_loss=37.045, train_acc=0.599, val_loss=12.869, val_acc=0.460]

Epoch 21 - Validation accuracy: 0.460
Epoch 21 - Validation loss: 12.869

Epoch 22 - Training accuracy: 0.629
Epoch 22 - Training loss: 35.253

Training Progress: 44%| | 22/50 [04:51<06:12, 13.29s/it,
train_loss=35.253, train_acc=0.629, val_loss=9.895, val_acc=0.490]

Epoch 22 - Validation accuracy: 0.490
Epoch 22 - Validation loss: 9.895

Epoch 23 - Training accuracy: 0.637
Epoch 23 - Training loss: 34.402

Training Progress: 46%| | 23/50 [05:04<05:57, 13.26s/it,
train_loss=34.402, train_acc=0.637, val_loss=9.343, val_acc=0.535]

Epoch 23 - Validation accuracy: 0.535
Epoch 23 - Validation loss: 9.343

Epoch 24 - Training accuracy: 0.632
Epoch 24 - Training loss: 32.652

Training Progress: 48%| | 24/50 [05:18<05:44, 13.25s/it,
train_loss=32.652, train_acc=0.632, val_loss=13.129, val_acc=0.520]

Epoch 24 - Validation accuracy: 0.520

Epoch 24 - Validation loss: 13.129

Epoch 25 - Training accuracy: 0.675

Epoch 25 - Training loss: 30.246

Training Progress: 50%| | 25/50 [05:31<05:31, 13.25s/it,
train_loss=30.246, train_acc=0.675, val_loss=12.897, val_acc=0.490]

Epoch 25 - Validation accuracy: 0.490

Epoch 25 - Validation loss: 12.897

Epoch 26 - Training accuracy: 0.648

Epoch 26 - Training loss: 32.346

Training Progress: 52%| | 26/50 [05:44<05:17, 13.23s/it,
train_loss=32.346, train_acc=0.648, val_loss=10.664, val_acc=0.540]

Epoch 26 - Validation accuracy: 0.540

Epoch 26 - Validation loss: 10.664

Epoch 27 - Training accuracy: 0.678

Epoch 27 - Training loss: 30.908

Training Progress: 54%| | 27/50 [05:57<05:03, 13.19s/it,
train_loss=30.908, train_acc=0.678, val_loss=10.310, val_acc=0.545]

Epoch 27 - Validation accuracy: 0.545

Epoch 27 - Validation loss: 10.310

Epoch 28 - Training accuracy: 0.678

Epoch 28 - Training loss: 30.626

Training Progress: 56%| | 28/50 [06:10<04:50, 13.21s/it,
train_loss=30.626, train_acc=0.678, val_loss=9.383, val_acc=0.565]

Epoch 28 - Validation accuracy: 0.565

Epoch 28 - Validation loss: 9.383

Epoch 29 - Training accuracy: 0.689

Epoch 29 - Training loss: 28.552

Training Progress: 58%| | 29/50 [06:24<04:37, 13.23s/it,
train_loss=28.552, train_acc=0.689, val_loss=8.974, val_acc=0.590]

Epoch 29 - Validation accuracy: 0.590

Epoch 29 - Validation loss: 8.974

Epoch 30 - Training accuracy: 0.718

Epoch 30 - Training loss: 26.792

Training Progress: 60%| | 30/50 [06:37<04:24, 13.21s/it,
train_loss=26.792, train_acc=0.718, val_loss=8.636, val_acc=0.605]
Epoch 30 - Validation accuracy: 0.605
Epoch 30 - Validation loss: 8.636

Epoch 31 - Training accuracy: 0.699
Epoch 31 - Training loss: 27.751

Training Progress: 62%| | 31/50 [06:50<04:10, 13.20s/it,
train_loss=27.751, train_acc=0.699, val_loss=11.307, val_acc=0.555]
Epoch 31 - Validation accuracy: 0.555
Epoch 31 - Validation loss: 11.307

Epoch 32 - Training accuracy: 0.744
Epoch 32 - Training loss: 26.411

Training Progress: 64%| | 32/50 [07:03<03:57, 13.21s/it,
train_loss=26.411, train_acc=0.744, val_loss=9.044, val_acc=0.605]
Epoch 32 - Validation accuracy: 0.605
Epoch 32 - Validation loss: 9.044

Epoch 33 - Training accuracy: 0.762
Epoch 33 - Training loss: 23.278

Training Progress: 66%| | 33/50 [07:17<03:45, 13.24s/it,
train_loss=23.278, train_acc=0.762, val_loss=10.110, val_acc=0.600]
Epoch 33 - Validation accuracy: 0.600
Epoch 33 - Validation loss: 10.110

Epoch 34 - Training accuracy: 0.753
Epoch 34 - Training loss: 24.566

Training Progress: 68%| | 34/50 [07:30<03:33, 13.32s/it,
train_loss=24.566, train_acc=0.753, val_loss=9.415, val_acc=0.600]
Epoch 34 - Validation accuracy: 0.600
Epoch 34 - Validation loss: 9.415

Epoch 35 - Training accuracy: 0.745
Epoch 35 - Training loss: 23.603

Training Progress: 70%| | 35/50 [07:44<03:20, 13.37s/it,
train_loss=23.603, train_acc=0.745, val_loss=8.036, val_acc=0.640]
Epoch 35 - Validation accuracy: 0.640
Epoch 35 - Validation loss: 8.036

Epoch 36 - Training accuracy: 0.762
Epoch 36 - Training loss: 21.986

Training Progress: 72%| | 36/50 [07:57<03:06, 13.31s/it,
train_loss=21.986, train_acc=0.762, val_loss=9.143, val_acc=0.590]

Epoch 36 - Validation accuracy: 0.590

Epoch 36 - Validation loss: 9.143

Epoch 37 - Training accuracy: 0.780

Epoch 37 - Training loss: 21.738

Training Progress: 74%| | 37/50 [08:10<02:52, 13.25s/it,
train_loss=21.738, train_acc=0.780, val_loss=8.316, val_acc=0.645]

Epoch 37 - Validation accuracy: 0.645

Epoch 37 - Validation loss: 8.316

Epoch 38 - Training accuracy: 0.780

Epoch 38 - Training loss: 21.675

Training Progress: 76%| | 38/50 [08:23<02:38, 13.23s/it,
train_loss=21.675, train_acc=0.780, val_loss=9.923, val_acc=0.615]

Epoch 38 - Validation accuracy: 0.615

Epoch 38 - Validation loss: 9.923

Epoch 39 - Training accuracy: 0.789

Epoch 39 - Training loss: 21.230

Training Progress: 78%| | 39/50 [08:36<02:25, 13.20s/it,
train_loss=21.230, train_acc=0.789, val_loss=7.383, val_acc=0.675]

Epoch 39 - Validation accuracy: 0.675

Epoch 39 - Validation loss: 7.383

Epoch 40 - Training accuracy: 0.786

Epoch 40 - Training loss: 21.756

Training Progress: 80%| | 40/50 [08:49<02:11, 13.18s/it,
train_loss=21.756, train_acc=0.786, val_loss=8.976, val_acc=0.630]

Epoch 40 - Validation accuracy: 0.630

Epoch 40 - Validation loss: 8.976

Epoch 41 - Training accuracy: 0.794

Epoch 41 - Training loss: 20.426

Training Progress: 82%| | 41/50 [09:02<01:58, 13.20s/it,
train_loss=20.426, train_acc=0.794, val_loss=10.842, val_acc=0.590]

Epoch 41 - Validation accuracy: 0.590

Epoch 41 - Validation loss: 10.842

Epoch 42 - Training accuracy: 0.766

Epoch 42 - Training loss: 20.315

Training Progress: 84%| | 42/50 [09:16<01:45, 13.22s/it,
train_loss=20.315, train_acc=0.766, val_loss=9.073, val_acc=0.635]

Epoch 42 - Validation accuracy: 0.635

Epoch 42 - Validation loss: 9.073

Epoch 43 - Training accuracy: 0.817

Epoch 43 - Training loss: 17.025

Training Progress: 86%| | 43/50 [09:29<01:32, 13.24s/it,
train_loss=17.025, train_acc=0.817, val_loss=7.930, val_acc=0.610]

Epoch 43 - Validation accuracy: 0.610

Epoch 43 - Validation loss: 7.930

Epoch 44 - Training accuracy: 0.836

Epoch 44 - Training loss: 16.046

Training Progress: 88%| | 44/50 [09:42<01:19, 13.27s/it,
train_loss=16.046, train_acc=0.836, val_loss=8.277, val_acc=0.650]

Epoch 44 - Validation accuracy: 0.650

Epoch 44 - Validation loss: 8.277

Epoch 45 - Training accuracy: 0.816

Epoch 45 - Training loss: 17.142

Training Progress: 90%| | 45/50 [09:56<01:06, 13.28s/it,
train_loss=17.142, train_acc=0.816, val_loss=9.210, val_acc=0.650]

Epoch 45 - Validation accuracy: 0.650

Epoch 45 - Validation loss: 9.210

Epoch 46 - Training accuracy: 0.814

Epoch 46 - Training loss: 18.575

Training Progress: 92%| | 46/50 [10:09<00:53, 13.30s/it,
train_loss=18.575, train_acc=0.814, val_loss=8.284, val_acc=0.655]

Epoch 46 - Validation accuracy: 0.655

Epoch 46 - Validation loss: 8.284

Epoch 47 - Training accuracy: 0.853

Epoch 47 - Training loss: 15.368

Training Progress: 94%| | 47/50 [10:22<00:39, 13.28s/it,
train_loss=15.368, train_acc=0.853, val_loss=7.455, val_acc=0.660]

Epoch 47 - Validation accuracy: 0.660

Epoch 47 - Validation loss: 7.455

Epoch 48 - Training accuracy: 0.839

Epoch 48 - Training loss: 14.682

```
Training Progress: 96%|      | 48/50 [10:35<00:26, 13.25s/it,  
train_loss=14.682, train_acc=0.839, val_loss=8.877, val_acc=0.685]
```

```
Epoch 48 - Validation accuracy: 0.685
```

```
Epoch 48 - Validation loss: 8.877
```

```
Epoch 49 - Training accuracy: 0.845
```

```
Epoch 49 - Training loss: 14.399
```

```
Training Progress: 98%|      | 49/50 [10:49<00:13, 13.26s/it,  
train_loss=14.399, train_acc=0.845, val_loss=8.669, val_acc=0.680]
```

```
Epoch 49 - Validation accuracy: 0.680
```

```
Epoch 49 - Validation loss: 8.669
```

```
Epoch 50 - Training accuracy: 0.876
```

```
Epoch 50 - Training loss: 13.415
```

```
Training Progress: 100%|     | 50/50 [11:02<00:00, 13.25s/it,  
train_loss=13.415, train_acc=0.876, val_loss=8.064, val_acc=0.685]
```

```
Epoch 50 - Validation accuracy: 0.685
```

```
Epoch 50 - Validation loss: 8.064
```

3.6 Question 2d: Visualize the ResNet's Training Performance

Task: Plot the ResNet's training loss, training accuracy, validation loss, and validation accuracy! You can use the `plot_metrics` function we defined earlier :)

```
[69]: plot_metrics(resnet_train_losses, resnet_val_losses, resnet_train_accuracies,   
↳ resnet_val_accuracies, num_epochs=50, title="ResNet Training Metrics")
```

```
/tmp/ipython-input-906427852.py:24: UserWarning:
```

```
No artists with labels found to put in legend. Note that artists whose label  
start with an underscore are ignored when legend() is called with no argument.
```

```
/tmp/ipython-input-906427852.py:32: UserWarning:
```

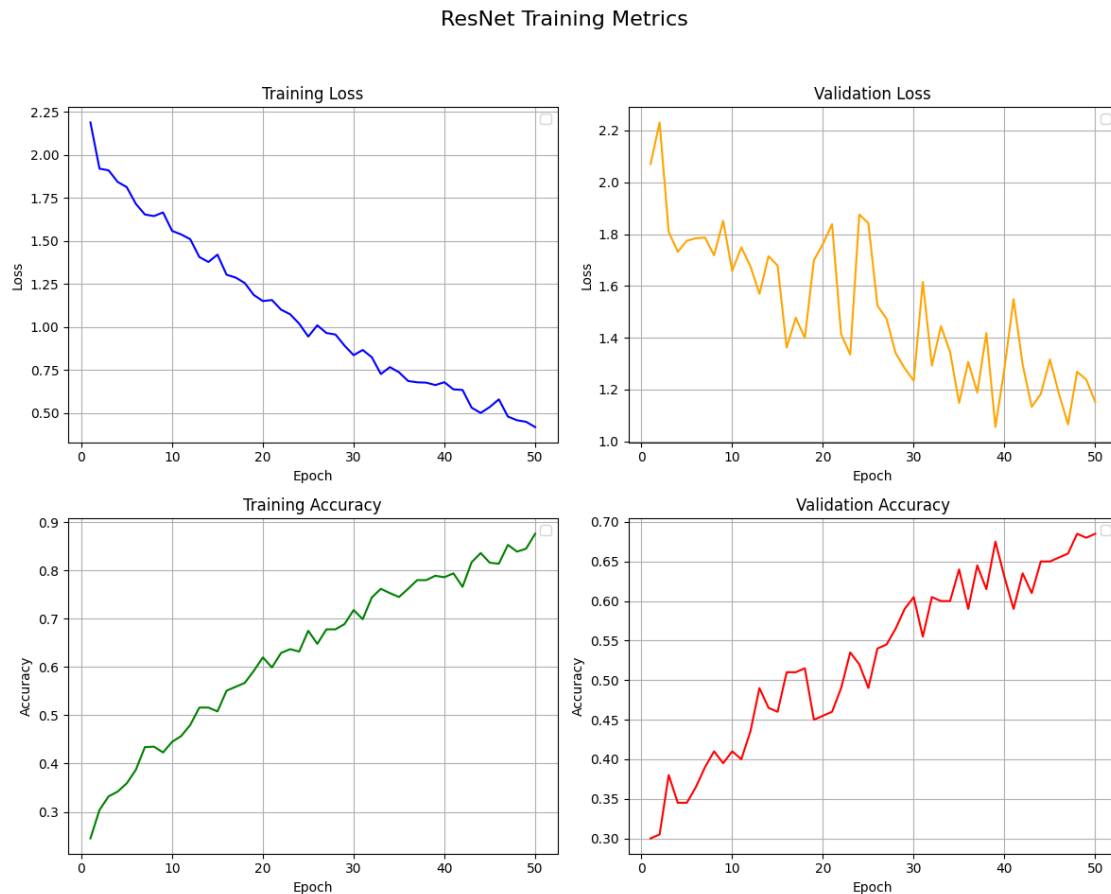
```
No artists with labels found to put in legend. Note that artists whose label  
start with an underscore are ignored when legend() is called with no argument.
```

```
/tmp/ipython-input-906427852.py:40: UserWarning:
```

```
No artists with labels found to put in legend. Note that artists whose label  
start with an underscore are ignored when legend() is called with no argument.
```

```
/tmp/ipython-input-906427852.py:48: UserWarning:
```

No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.



4 Subpart 3: Transformers

The Transformer Architecture (source: Machine Learning Mastery)

Like ResNets, Transformers are another architecture innovation that *transformed* the world of machine learning.

Transformers were first invented for sequence-to-sequence applications, such as machine translation. In sequence-to-sequence applications, we want to take one sequence (e.g. English words) and output another sequence (e.g. its French translation).

In early seq2seq models, the entire input was compressed by an encoder into a single fixed-length vector (called the context vector). Then, this single context vector was fed into a decoder which would produce an output. [RNNs \(recurrent neural networks\)](#) improved this idea by processing data step by step, and passing information along through hidden vectors. But they struggled to capture long-range contexts and took a long time to train because they had to process each input token

sequentially.

Transformers solved this problem by using *attention*. The intuition behind attention is we sometimes want to dynamically *attend* to only the most important parts of the input—or of the output we have generated so far—when we want to predict what should come next in our output sequence.

4.1 Attention

So how exactly do you determine which parts of your sequence are most relevant to each other?

Recall that we can use dot products to compute how similar 2 vectors are to each other! The higher the value of two vectors' dot product, the more 'similar' they are to each other/the closer they are to each other in vector space.

So Transformers use a mechanism called *scaled dot-product attention* to implement attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- For each position in the input, the model computes three vectors: a query vector q , a key vector k , and a value vector v . We can stack the query, key, and value vectors into matrices Q, K, V , where the i -th row of Q corresponds to the query vector q_i of the i -th token in the sequence, etc.
- These vectors are learned linear projections of the input embeddings.
 - A helpful analogy for understanding the role that the query, key, and value vectors play is thinking about going to the library! The *query* represents what you are searching for (for instance, you might be looking for a book about matrix calculus). The *keys* for each book in the library are their titles—you use this to judge how relevant the content in the book might be to your search. And the *value* of each library book is the actual content inside.
- To determine how much attention to pay to each part of the input, the model takes the dot product between the query vector of the current position and the key vectors of all positions in the sequence. This produces a set of scores that measure the similarity or relevance between the current position and every other position. We can represent taking dot products between all the query vectors and key vectors all at once as the matrix multiplication of QK^T . The i -th row of the matrix product QK^T represents how similar the i -th query vector was to all the key vectors. Specifically, the i, j -th entry represents how 'similar' the i -th token was to the j -th token.
- Then, these scores are divided by the square root of the key dimension d_k (to prevent their variance from growing too large), and passed through a softmax function.
- The softmax turns the scores into a probability distribution, so that all attention weights for a given token in the sequence sum to one. In other words, if you were to sum the attention weights across a single row, they would all sum up to 1. Remember that the i -th row corresponds to the attention weights for the i -th input token! So we are simply re-scaling the 'similarity scores' in each row/for each input token.
- Finally, these weights are used to compute a weighted sum of the value vectors for each position. In matrix notation, this is represented by multiplying the *softmax* matrix by V .

The result is another matrix which represents the attention output for all positions in the sequence—you can think of this matrix as telling us how much each input token should “attend” to every other

token!

This process allows the model to dynamically focus on the most relevant parts of the input when making predictions, and is the core building block of the Transformer architecture.

4.2 Encoders and Decoders

Transformers consist of 2 main parts, which both use attention but in slightly different ways!

4.2.1 Encoder

The encoder takes in the original input sequence and produces a vector for each input position that encodes information about that token based on the context of the entire input.

The encoder is a stack of identical layers. Each layer has: 1. **Self-Attention:** Lets every token attend to all other tokens in the input to gather global context. 2. Position-wise Feed-Forward Network (two linear layers with a nonlinearity). 3. Residual connections and LayerNorm around both sublayers to stabilize training and ease gradient flow.

4.2.2 Decoder

The decoder generates outputs autoregressively (which means it generates the next token based on previous tokens it has already outputted)

The decoder is also a stack of identical layers, but each layer has three sublayers: 1. **Masked Self-Attention:** tokens from the output sequence attend to one another. The query, key, and value vectors all come from the decoder. We also apply a causal (look-ahead) mask in the decoder self-attention, which prevents a position from attending to future target tokens. During training, we pass in the entire target sequence to the decoder so we want to make sure the decoder can't "cheat" by looking ahead at future tokens when trying to predict them. 2. **Cross-Attention:** tokens from the output sequence also attend to the encoder's output, "enriching" vectors in the decoder with useful information from the encoder. The query vectors come from the decoder, and the keys/values come from the encoder. 3. Position-wise Feed-Forward Network 4. Each sublayer is wrapped with residual connections and LayerNorm.

Aspect	Encoder	Decoder
Purpose	Processes and encodes input data into rich representations (hidden states).	Generates output sequences by decoding encoded information and attending to previous outputs.
Input/Output	Input: Full input sequence Output: Contextual embeddings	Input: Previously generated tokens + encoder output Output: Next token(s) in output sequence

Aspect	Encoder	Decoder
Attention Mechanism	Self-attention only: Each token attends to all input tokens (no masking).	Masked self-attention: Each token can attend only to previous (and current) tokens (no future). Encoder-decoder attention: Attends to encoder outputs to access input information.
Access to Future Tokens?	Yes - sees the entire input at once; can use information from future tokens in the input.	No (in self-attention) - masking prevents attending to future tokens during generation.
Key Components (per block)	- LayerNorm - Multi-head Self-Attention - Feedforward Network (MLP) - Residual connections	- LayerNorm - Masked Multi-head Self-Attention - Encoder-Decoder Attention - Feedforward Network (MLP) - Residual connections
Typical Uses	- Encoding textual, image, or other input data - Feature extraction - Classifier head preparation	- Sequence generation (translation, summarization) - Language modeling - Autoregressive generation

In practice, many modern machine learning architectures use either only the encoder or only the decoder part of the transformer model, depending on the task. * Decoder-only models, such as GPT (Generative Pretrained Transformer), are very popular for tasks that involve generating text, since they predict the next token one at a time using previously generated words. * For text generation, a decoder-only model is sufficient because the model generates tokens one by one, each time looking at all the tokens it has already produced. * We “initiate” the decoding process by passing in a special start-of-sequence token. * Encoder-only models, like BERT (Bidirectional Encoder Representations from Transformers), are widely used as embedding models to encode and understand text. * Encoders process the entire input sequence at once and create contextual embeddings that represent the relationships between all tokens in the input. * The contextual embeddings can be used for a variety of tasks, such as classification, clustering, or comparison. * This division reflects the strengths of each component: decoders are optimized for generating outputs in sequence, while encoders are designed to understand and embed input data as a whole. * Useful for encoding meaning, finding similarities, and making decisions about the whole input, rather than generating new content one step at a time

4.3 Question 3a: Softmax

The softmax function converts a vector of real numbers between $[-\infty, \infty]$ into probabilities that sum up to one.

For a vector $z = [z_1, z_2, \dots, z_l]$, the softmax output for the i -th element in z is given by:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

Each component $\text{softmax}(z_i) \in (0, 1)$ and all the components sum to 1.

Sometimes, you will also see the softmax output written as

$$\text{softmax}(z_i) = \frac{e^{z_i - \max(z)}}{\sum_{j=1}^k e^{z_j - \max(z)}}$$

where the maximum value of z is subtracted from each element before exponentiation. By subtracting $\max(z)$ from each input, exponentials are all kept between 0 and 1 so large numbers won't overflow into infinity and small numbers won't underflow to 0. This is sometimes used as a more numerically stable version of the softmax function for floating point arithmetic.

Task: Implement the softmax function (either the vanilla version, or stabilized version)

Hints: * The input x might be a 1-D Tensor of shape (d) or a multi-dimensional tensor of shape (B, N, N). When you apply a sum, using parameters such as `axis` to specify which dimension to sum over and `keep_dims` to avoid flattening the Tensor might be helpful! * Some helpful PyTorch functions: * `torch.exp` [documentation](#) * `torch.sum` [documentation](#)

```
[70]: def softmax(x: torch.Tensor):
    """
    Compute the softmax of each element in x.
    """
    dim = x.dim() - 1
    if dim == 1:
        x_max = torch.max(x)
    else:
        x_max = torch.max(x, dim=dim, keepdim=True).values
    numerator = torch.exp(x - x_max) # subtract max
    ↪ for stability
    if dim == 1:
        denominator = torch.sum(numerator)
    else:
        denominator = torch.sum(numerator, dim=dim, keepdim=True)
    return numerator/denominator
```

4.3.1 Check

To check the correctness of our implementation, we can: * Verify that the output of our softmax function has the expected dimensions (the softmax output should have the same shape as the input) * Verify that the elements along the last dimension sum up to 1 (since softmax should transform them into a valid probability distribution) * Compare our implementation with PyTorch's `softmax` function and check that our values are within a reasonable range of PyTorch's

```
[71]: set_seed(SEED)

rand_1d = torch.randn(5)
print(f"Random 1D vector with shape (d): {rand_1d.shape}")
out_1d = softmax(rand_1d)

print(f"Softmax output shape: {out_1d.shape}\nExpected shape: {rand_1d.
    ↪shape}\n")

# Assert that the sum of the softmax output along the last dimension is 1
assert torch.allclose(torch.sum(out_1d, dim=-1), torch.ones_like(out_1d[...],
    ↪0)), rtol=1e-5), "Sum of softmax output along last dimension is not 1"

rand_3d = torch.randn(2, 5, 5)
print(f"Random vector with shape (B, N, N): {rand_3d.shape}")
out_3d = softmax(rand_3d)
print(f"Softmax output shape: {out_3d.shape}\nExpected shape: {rand_3d.
    ↪shape}\n")

# Assert that the sum of the softmax output along the last dimension is 1
assert torch.allclose(torch.sum(out_3d, dim=-1), torch.ones_like(out_3d[...],
    ↪0)), rtol=1e-5), "Sum of softmax output along last dimension is not 1"

# Compare to torch implementation
torch_softmax_1d = torch.nn.functional.softmax(rand_1d, dim=-1)
assert torch.allclose(torch_softmax_1d, out_1d, rtol=1e-5), "Softmax
    ↪implementation for 1D tensor does not match torch implementation"

torch_softmax_3d = torch.nn.functional.softmax(rand_3d, dim=-1)
assert torch.allclose(torch_softmax_3d, out_3d, rtol=1e-5), "Softmax
    ↪implementation for 3D tensor does not match torch implementation"
```

Random 1D vector with shape (d): torch.Size([5])

Softmax output shape: torch.Size([5])

Expected shape: torch.Size([5])

Random vector with shape (B, N, N): torch.Size([2, 5, 5])

Softmax output shape: torch.Size([2, 5, 5])

Expected shape: torch.Size([2, 5, 5])

4.4 Question 3b: Scaled Dot Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Before we dive into implementing the scaled dot product attention mechanism, let's review what **masking** is! Masking in attention serves 2 critical purposes: 1. Causal Masking (Look-Ahead

Mask) prevent future information leakage. In decoder-only models (like GPT) and the decoder portion of encoder-decoder models, we need to ensure that when predicting token at position i , the model can only attend to tokens at positions $\leq i$ and not future positions. 1. Why? - During training, we have access to the entire sequence, but at inference time, we generate tokens one at a time and don't know future tokens yet 2. Without masking, the model would "cheat" by looking at future tokens during training. This creates a mismatch between training and inference behavior

Sentence = "The cat sat on" When predicting "sat", the model should only see: "The" "cat" "sat" "on" (future token - must be masked)

2. Padding Mask - When batching sequences of different lengths, we sometimes pad shorter sequences with special tokens. We need to mask these padding positions so the model doesn't attend to meaningless values.

1. Why? - Padding tokens have no semantic meaning, so they shouldn't influence what the model attends to. We mask [PAD] tokens so they receive zero attention weight.

Batch of 2 sequences: Seq 1: "Hello world" [length 2] Seq 2: "I love dinosaurs" [length 3]

After padding: Seq 1: "Hello world [PAD]" Seq 2: "I love dinosaurs"

We implement masking by setting the attention score for positions we want to mask out to $-\infty$ before we apply softmax. After the softmax operation, these values become 0.

Task: 1. Implement the `scaled_dot_product_attention` function which computes the scaled dot product attention of Q , K , and V . 2. If `mask` is not `None`, use `masked_fill` to fill all values where `mask = True` with `float('-inf')`. 3. Use the `softmax` function you implemented above to compute the softmax.

Hints: * Make sure to transpose K before matrix-multiplying with Q . * Use `masked_fill` to fill masked values with `float('-inf')`. * Don't forget to divide QK^T by the square root of the query/key vectors' dimension (d_k) before applying `softmax`.

Notes: * As an optional debugging tool, you can use the `verbose` parameter as a flag to print out extra debugging statements (e.g. tensor shapes) inside your implementation. It is up to you whether you want to use it or not!

```
[72]: def scaled_dot_product_attention(Q: torch.Tensor, K: torch.Tensor, V: torch.
      ↪Tensor, mask=None, verbose=False):
      """
      Q: matrix of shape (B, N, d_k)
      K: matrix of shape (B, N, d_k)
      V: matrix of shape (B, N, d_v)
      mask: boolean matrix of shape (B, N, N). Values where mask is True will be
      ↪masked out

      B is the batch size
      N is the number of query/key/value vectors
      d_k is the dimensions of the query/key vectors
      d_v is the dimensions of the value vectors
      """
```

```

assert Q.shape[-1] == K.shape[-1], "Q and K must have same d_k dimension"
assert K.shape[-2] == V.shape[-2], "K and V must have same sequence length"

# TODO: Implement the attention function

K_t = torch.transpose(K,-2,-1)
scores = torch.matmul(Q,K_t) / torch.sqrt(torch.tensor(Q.shape[-1]))
if mask is not None:
    scores = scores.masked_fill(mask,float('-inf'))
weights = softmax(scores)
attention = torch.matmul(weights,V)
return attention

```

4.4.1 Check

Once again, let's perform a quick sanity check by checking the shape of the scaled dot product attention's output and comparing it to PyTorch's implementation.

```

[73]: Q_tensor = torch.randn(2, 5, 10)
      K_tensor = torch.randn(2, 5, 10)
      V_tensor = torch.randn(2, 5, 5)

print(f"Shape of Q: {Q_tensor.shape}")
print(f"Shape of K: {K_tensor.shape}")
print(f"Shape of V: {V_tensor.shape}")

sdpa_output = scaled_dot_product_attention(Q_tensor, K_tensor, V_tensor)
print(f"Shape of output: {sdpa_output.shape}")
assert sdpa_output.shape == torch.Size([2, 5, 5]), f"Expected output shape (B, L, d_v) {Q_tensor.shape}, but got {sdpa_output.shape}"

# Test that implementation is equal to torch implementation
torch_sdp = torch.nn.functional.scaled_dot_product_attention(
    Q_tensor, K_tensor, V_tensor, attn_mask=None
)
assert torch.allclose(sdpa_output, torch_sdp, rtol=1e-5), "Scaled dot product_
    ↪attention implementation does not match torch implementation"

```

```

Shape of Q: torch.Size([2, 5, 10])
Shape of K: torch.Size([2, 5, 10])
Shape of V: torch.Size([2, 5, 5])
Shape of output: torch.Size([2, 5, 5])

```

4.5 Question 3c: Attention Head

Similarly to how we implemented our own ResidualBlocks that inherited from the `nn.Module` class, we can do the same for each part of a Transformer!

We'll start by implementing a single attention head.

Task: 1. Implement the `init` method: 1. Initialize the linear transformation matrices for queries, keys, and values. 2. Implement the `forward` method to compute attention using the `scaled_dot_product_attention` function you defined above. 1. The `forward` method should handle both the self-attention (when `encoder_output=None`) and cross-attention (when `encoder_output` is provided) cases. 1. Recall that during self-attention, all three of the Q, K, and V matrices should be generated with the input `x`. 2. During cross-attention, Q should come from the `x`, and K and V should come from the encoder's output. 2. Also, the `forward` method should apply the attention mask if provided.

Hints: * Use `nn.Linear` layers to create the transformation matrices `W_q`, `W_k`, and `W_v` in the `__init__` method. * For self-attention: use the same input `x` for queries, keys, and values. * For cross-attention: use `x` for queries and `encoder_output` for keys and values. * Call your `scaled_dot_product_attention` function with the computed Q, K, V matrices after you apply `W_q`, `W_k`, and `W_v` on the input `x`.

Notes: * Optionally, you can save the `verbose` parameter as a member of your model (`self.verbose = verbose`). Then, you can check if `self.verbose` is `True` in your `forward` function to print extra debugging statements.

Expected behavior: * Input shape: `(batch_size, seq_len, d_model)` * Output shape: `(batch_size, seq_len, d_k)` where `d_k` is the dimension of the key/value vectors

```
[74]: class AttentionHead(nn.Module):
    def __init__(self, d_model=512, d_k=64, verbose=False):

        # TODO: Implement the __init__ method
        # Don't forget to call super().__init__()!
        super().__init__()
        self.W_q = nn.Linear(d_model, d_k)
        self.W_k = nn.Linear(d_model, d_k)
        self.W_v = nn.Linear(d_model, d_k)
        self.verbose = verbose

    def forward(self, x, encoder_output=None, mask=None):
        """
        x is the input to use for the queries, keys, and values
        encoder_output is the output from the encoder (used for cross-attention)
        mask is the mask to use for the attention
        """
        Q = self.W_q(x)
        K = self.W_k(encoder_output if encoder_output is not None else x)
        V = self.W_v(encoder_output if encoder_output is not None else x)

        # TODO: Implement the forward method
        return scaled_dot_product_attention(Q, K, V, mask, self.verbose)
```

```
[75]: # Sanity check the shape of the Attention Head output
set_seed(SEED)
batch_size = 10
seq_len = 20
d_model = 512
d_k = 64

x = torch.rand((batch_size, seq_len, d_model))
print(f"Shape of x: {x.shape}")

ah = AttentionHead(d_model=d_model, d_k=d_k)

ah_out = ah(x)

print(f"Attention head output shape: {ah_out.shape}")

assert ah_out.shape == torch.Size([batch_size, seq_len, d_k]), f"Expected_
↪output shape (B, N, d_k) {batch_size, seq_len, d_k}, but got {ah_out.shape}"
```

Shape of x: torch.Size([10, 20, 512])

Attention head output shape: torch.Size([10, 20, 64])

4.6 Question 3d: Multi-Head Attention

4.6.1 What is Multi-Headed Attention?

Multi-headed attention runs several attention “heads” in parallel. Each head:

- * Projects inputs into its own Q, K, V subspace of size $d_k = d_{\text{model}} / \text{num_heads}$
- * Computes scaled dot-product attention independently
- * Outputs a sequence of shape $(\text{batch}, \text{seq_len}, d_k)$

These head outputs are concatenated back to $(\text{batch}, \text{seq_len}, d_{\text{model}})$ and passed through a final linear layer to mix information across heads.

Expected shapes:

- * Input: $(B, \text{seq_len}, d_{\text{model}})$
- * Per head: $(B, \text{seq_len}, d_k)$
- * Concat: $(B, \text{seq_len}, h \cdot d_k) = (B, \text{seq_len}, d_{\text{model}})$
- * Output projection: $(B, \text{seq_len}, d_{\text{model}})$

Benefits of Multi-Headed Attention

- * Multiple perspectives: Different heads can learn different relationships simultaneously (e.g., syntax vs. semantics, short-range vs. long-range).
- * Subspace specialization: Learned projections let heads focus on complementary features in different representation subspaces.
- * Expressivity without huge cost: Multiple small heads approximate richer interactions than a single large head, with similar compute.
- * More stable learning: Lower-dimensional per-head dot products and diversity across heads can stabilize training and reduce overfitting to one pattern.

In practice, multi-head attention helps models capture richer, multi-scale dependencies across tokens, improving performance on a wide range of sequence tasks.

Task: Implement the `MultiHeadAttention` module

1. Implement the `__init__` method.
2. Compute the dimensions of the query and key vectors in each head (d_k).
3. Instantiate `num_heads AttentionHead` modules with d_k .
4. Instantiate a linear projection that goes from d_{model} to

`d_model`. 2. Implement the `forward` method. 1. Pass the same `x`, `encoder_output` (if provided), and `mask` (if provided) through each head. 2. Concatenate the outputs of all the heads along the last dimension. 3. Pass the concatenated output into `W_out` to compute the final linear projection.

Hints: * `d_model` refers to the the embedding/hidden size of the model's representations. You can calculate the per-head dimension `d_k` for each head's queries/keys by dividing `d_model` by `num_heads`. * If you are storing all your `AttentionHeads` in a plain Python list, PyTorch won't know to register their parameters later on during training! Instead, you should store them in `nn.ModuleList`. * Each head output has shape `(batch_size, seq_len, d_k)`. What dimension should you concatenate them in to get an output with shape `(batch_size, seq_len, d_model)`?

Expected behavior: * Input: `x` shape `(batch_size, seq_len, d_model)` * Output: shape `(batch_size, seq_len, d_model)`

```
[76]: class MultiHeadAttention(nn.Module):
    def __init__(
        self,
        num_heads=8,
        d_model=512,  # dimension of the embeddings
        verbose=False,
    ):
        """
        Args:
            num_heads: number of attention heads
            d_model: dimension of the embeddings
            verbose: whether to print debug information
        """
        # TODO: Implement the __init__ method
        super().__init__()
        d_k = d_model // num_heads
        self.heads = nn.ModuleList([AttentionHead(d_k = d_k) for i in
↪range(num_heads)])
        self.W_out = nn.Linear(d_model, d_model)
        self.verbose = verbose

    def forward(self, x, encoder_output=None, mask=None):
        # TODO: Implement the forward method
        output = torch.cat([head(x, encoder_output, mask) for head in self.
↪heads], dim=-1)
        output = self.W_out(output)
        return output
```

```
[77]: # Sanity check the shape of the Multi-Head Attention output
set_seed(SEED)
batch_size = 10
seq_len = 20
d_model = 512
num_heads = 8
```

```

x = torch.rand((batch_size, seq_len, d_model))
print(f"Shape of x: {x.shape}")

mhsa = MultiHeadAttention(num_heads=num_heads, d_model=d_model, verbose=False)

mhsa_out = mhsa(x)

print(f"Shape of Multi-Head Attention output: {mhsa_out.shape}")

assert mhsa_out.shape == torch.Size([batch_size, seq_len, d_model]), f"Expected_
↳output shape (B, N, d_model) {batch_size, seq_len, d_model}, but got_
↳{mhsa_out.shape}"

```

Shape of x: torch.Size([10, 20, 512])

Shape of Multi-Head Attention output: torch.Size([10, 20, 512])

4.7 Question 3e: Encoder Layer

The encoder stacks together self attention, a feedforward network, residual connections, and layer normalizations to take tokens from an input sequence and produce context-rich vectors.

4.7.1 Feedforward Network (FFN)

The FFN is a small multi-layer perceptron applied independently at each sequence position (same weights at every position). This is just a fancy way of saying that the FFN is a linear layer! It typically follows this structure: $\text{Linear}(d_{\text{model}} \rightarrow d_{\text{ffn}}) \rightarrow \text{nonlinearity (ReLU/GELU)} \rightarrow \text{Linear}(d_{\text{ffn}} \rightarrow d_{\text{model}})$.

d_{ffn} is the hidden/inner width of the FFN (the expansion dimension). Generally, d_{ffn} is chosen to be $4 * d_{\text{model}}$. After the first linear layer “expands” each vector from dimension d_{model} to d_{ffn} , the second linear layer “shrinks” the expanded vector back down to d_{model} so that it can be passed to the next `EncoderLayer` or to the Decoder.

You might ask, why would we expand the dimensions up to d_{ffn} if we’re just going to shrink it back down to d_{model} again? The reason is that generally, vectors in higher-dimensions can encode more diverse types of information. We’re sort of giving the FFN a chance to learn additional information about the vectors in this new, bigger feature space that it can then compress into a more useful vector even back in a feature space with only d_{model} dimensions.

4.7.2 Encoder Layer Architecture

1. MultiHeadAttention with d_{model} model dimensions and `num_heads` heads
2. A residual connection to add x to the MultiHeadAttention’s output
3. LayerNorm 1
4. A position-wise feedforward network (FFN) which consists of:
 1. A linear layer with d_{model} in_features and d_{ffn} out_features
 2. A ReLU activation
 3. A linear layer with d_{ffn} in_features and d_{model} out_features

5. A residual connection which adds the output of step 3 to the FFN's output
6. LayerNorm 2

Task: Implement the `EncoderLayer` module. 1. Implement the `__init__` method. 1. Instantiate a `MultiHeadAttention` with `d_model` and `num_heads`. 2. Initialize a 2-layer FFN with hidden size `d_ffn = 4 * d_model` and ReLU activations. 3. Instantiate 2 `LayerNorm` modules. 2. Implement the `forward` method. 1. Compute self-attention by passing `x` and `mask` (if not `None`) into the `MultiHeadAttention` block. 2. Add a residual connection from the original input `x` to the `MultiHeadAttention`'s output. 3. `LayerNorm`. 4. Pass the normalized output through the FFN. 5. Add a residual connection from the first `LayerNorm`'s output to the FFN's output. 6. `LayerNorm`. 7. Return the final normalized output.

Hints: * You can group all the layers of the FFN together using `nn.Sequential`: [documentation](#). * Don't forget to calculate what the FFN's `d_ffn` dimension should be! * When instantiating a `LayerNorm`, you need to pass in a parameter called `normalized_shape`. `normalized_shape` specifies the size of the last dimension over which `LayerNorm` will compute mean/variance and learn a scale/bias parameter. What is the shape of the input to each `LayerNorm` layer? What is the last dimension of the shape? * Make sure the matrices you are adding together with residual connections have matching dimensions!

Expected behavior: Input: `x` shape `(batch_size, seq_len, d_model)` Output: shape `(batch_size, seq_len, d_model)`

```
[78]: class EncoderLayer(nn.Module):
    def __init__(self, d_model=512, num_heads=8, verbose=False):

        # TODO: Implement the __init__ method
        super().__init__()
        self.attention = MultiHeadAttention(d_model=d_model, num_heads=num_heads)
        self.layerNorm1 = nn.LayerNorm(d_model)
        self.layerNorm2 = nn.LayerNorm(d_model)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, 4*d_model),
            nn.ReLU(),
            nn.Linear(4*d_model, d_model)
        )

    def forward(self, x, mask=None):

        # TODO: Implement the forward method
        # 1. Self-attention
        res1 = x
        self_attn = self.attention(x, mask)
        # 2. Residual connection
        out = self_attn + res1
        # 2a. Pass `x` through the decoder if decoder_only
        # if self.decoder_only:
        #     decoder_output = self.decoder(x, None)
        # 2b. Pass `x` and the encoder's output through the decoder
```

```

    # else:
    #     decoder_output = self.decoder(x, encoder_output)
    # ...
    # 3. LayerNorm
    out = self.layerNorm1(out)
    # 4. Feedforward network
    res2 = out
    ffn_out = self.ffn(out)
    # 5. Residual connection
    out = ffn_out + res2
    # 6. LayerNorm
    out = self.layerNorm2(out)
    # 7. Return the output
    return out

```

```

[79]: # Sanity check the shape of the EncoderLayer output
set_seed(SEED)
batch_size = 10
seq_len = 20
d_model = 512
num_heads = 8

x = torch.rand((batch_size, seq_len, d_model)) # (B, N, d_model)
print(f"Shape of x: {x.shape}")

encoder = EncoderLayer(d_model=d_model, num_heads=num_heads, verbose=False)

encoder_out = encoder(x)

print(f"Shape of encoder layer output: {encoder_out.shape}")

assert encoder_out.shape == torch.Size([batch_size, seq_len, d_model]), □
    ↪ "Expected output shape (B, N, d_model) {batch_size, seq_len, d_model}, but □
    ↪ got {encoder_out.shape}"

```

Shape of x: torch.Size([10, 20, 512])

Shape of encoder layer output: torch.Size([10, 20, 512])

4.8 Question 3f: Decoder Layer

The Decoder Layers will be very similar to the Encoder Layers, with just a few important changes:

1. The Decoder uses masked self-attention to prevent peeking at future tokens.
2. The Decoder also has a second MultiHeadAttention block for cross-attention that cross-attends to the encoder outputs.

4.8.1 Decoder Layer Architecture

1. MultiHeadAttention for masked self-attention with `d_model` model dimensions and `num_heads` heads. During `forward`, we use a look-ahead mask.
2. A residual connection to add `x` to the masked self-attention's output
3. LayerNorm
4. MultiHeadAttention for cross-attention with `d_model` model dimensions and `num_heads` heads. During `forward`, we also pass in the `encoder_output` to generate the key and value vectors from.
5. A residual connection to add the output of step 3 to the cross-attention's output
6. LayerNorm
7. A position-wise feedforward network (FFN) which consists of:
 1. A linear layer with `d_model` `in_features` and `d_ffn` `out_features`
 2. A ReLU activation
 3. A linear layer with `d_ffn` `in_features` and `d_model` `out_features`
8. A residual connection which adds the output of step 6 to the FFN's output
9. LayerNorm

Task: Implement the `DecoderLayer` module

1. Implement `__init__`.
 1. Initialize a `MultiHeadAttention` for masked self-attention and the first `LayerNorm` layer.
 2. Initialize a `MultiHeadAttention` for cross-attention and the second `LayerNorm` layer.
 3. Initialize a 2-layer FFN with hidden size `d_ffn = 4 * d_model` and ReLU activations.
 4. Initialize a final `LayerNorm` layer.
2. Implement `forward(x, encoder_output)`.
 1. Build a causal/look-ahead mask of shape `(seq_len, seq_len)` that masks future positions
 2. Pass `x` and the look-ahead mask to the self-attention `MultiHeadAttention` block.
 3. Add a residual connection from the original input `x` to the self-attention's output.
 4. `LayerNorm`.
 5. Compute cross-attention using the `LayerNorm`'s output and the `encoder_output`.
 6. Add a residual connection from the first `LayerNorm`'s output to the cross-attention's output.
 7. `LayerNorm`.
 8. Compute the FFN.
 9. `LayerNorm`.
 10. Return the final normalized output.

Hint: * How can we create an upper triangular matrix to signify what elements in the attention matrix we want to mask out during decoder self-attention? `torch.triu` ([documentation](#)) might be helpful.

Expected behavior: * Inputs: `x` (`B, seq_len, d_model`), `encoder_output` (`B, seq_len, d_model`) * Outputs: (`B, seq_len, d_model`)

```
[80]: class DecoderLayer(nn.Module):
    def __init__(self, d_model=512, num_heads=8, verbose=False):

        # TODO: Implement the __init__ method
        super().__init__()
        self.masked_attention = MultiHeadAttention(d_model=d_model, num_heads=num_heads)
        self.layerNorm1 = nn.LayerNorm(d_model)
        self.layerNorm2 = nn.LayerNorm(d_model)
        self.layerNorm3 = nn.LayerNorm(d_model)
```

```

        self.cross_attention = □
        ↪MultiHeadAttention(d_model=d_model,num_heads=num_heads)
        self.ffn = nn.Sequential(
            nn.Linear(d_model,4*d_model),
            nn.ReLU(),
            nn.Linear(4*d_model,d_model)
        )
    def forward(self, x, encoder_output):

        # TODO: Implement the forward method

        # Create a look-ahead mask
        # Move the mask to the same device as x
        mask = torch.triu(torch.ones(x.shape[1], x.shape[1]), diagonal=1).
        ↪bool().to(x.device)

        # 1. Masked self-attention
        res1 = x
        out_mask = self.masked_attention(x,mask=mask)
        # 2. Residual connection
        out = out_mask + res1
        # 3. LayerNorm
        norm1 = self.layerNorm1(out)
        # 4. Cross-attention
        if encoder_output is not None:
            cross_attention = self.cross_attention(x=norm1, □
        ↪encoder_output=encoder_output)
        else:
            cross_attention = self.cross_attention(x=norm1)

        # 5. Residual connection
        norm1 = norm1 + cross_attention

        # 6. LayerNorm
        norm2 = self.layerNorm2(norm1)

        # 7. Position-wise feed forward network
        res2 = norm2
        norm2 = self.ffn(norm2)

        # 8. Residual connection
        norm2 += res2
        # 9. LayerNorm
        norm2 = self.layerNorm3(norm2)
        # 10. Return the final normalized output
        return norm2

```

```
[81]: # Sanity check the shape of the DecoderLayer output
set_seed(SEED)
batch_size = 10
seq_len = 20
d_model = 512
num_heads = 8

x = torch.rand((batch_size, seq_len, d_model)) # (B, N, d_model)
print(f"Shape of x: {x.shape}")

print("=" * 50)

encoder = EncoderLayer(d_model=d_model, num_heads=num_heads, verbose=True)
encoder_output = encoder(x)

print(f"Shape of encoder layer output: {encoder_output.shape}")

print("=" * 50)

decoder = DecoderLayer(d_model=d_model, num_heads=num_heads, verbose=True)
decoder_output = decoder(x, encoder_output)

print(f"Shape of decoder layer output: {decoder_output.shape}")

assert decoder_output.shape == torch.Size([batch_size, seq_len, d_model]),  
    ↪f"Expected output shape (B, N, d_model) {batch_size, seq_len, d_model}, but  
    ↪got {decoder_output.shape}"
```

```
Shape of x: torch.Size([10, 20, 512])
=====
Shape of encoder layer output: torch.Size([10, 20, 512])
=====
Shape of decoder layer output: torch.Size([10, 20, 512])
```

4.9 Question 3g: Positional Encodings

4.9.1 What are Positional Encodings?

The last building block of a transformer we'll need is a positional encoding layer. Previous seq2seq architectures processed input tokens one-by-one. However, every word/token in the source input sequence is passed to a transformer at once. So how does the transformer know the order of the words in the sequence?

Positional encodings add token-position information into the embeddings of the tokens. This helps the model know the ordering of the tokens within the source sequence.

In the original Transformer paper, positional encodings are implemented using *sine* and *cosine* waves with different frequencies depending on the position of the token and which element within that token's embedding we are looking at.

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

In this equation, pos refers to the position of a given token in the sequence and i refers to which dimension/index of the token's embedding this value applies to.

Task: Implement the `PositionalEncoding` module. 1. Implement `__init__(d_model, max_len)`. 1. Instantiate a (max_len, d_model) tensor of dummy values 2. Fill even indices with $\sin(pos / 10000^{(2i/d_model)})$ and odd with $\cos(\dots)$. 2. Implement `forward(x)`. 1. Add the positional encodings to the the input x and return the sum.

Hint: * Make sure that the shapes of your positional encodings either match up with the shape of x or are broadcastable so that adding them is a valid operation. * Inside `forward`, make sure you move your positional encodings to the same device that x is on! You can get what device x is on by calling `x.device`.

```
[82]: class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=1024):
        """
        max_len = max number of tokens in the sequence
        """
        super().__init__()
        self.pos = torch.zeros((max_len, d_model))
        numerator = torch.arange(end=max_len, dtype=torch.float32).reshape(-1, 1)
        ↪1) # shape (seq_len, 1)
        denominator = torch.pow(10000, torch.arange(start=0, end=d_model,
        ↪step=2, dtype=torch.float32) / d_model).reshape(1, -1) # shape (1, d_model /
        ↪2)

        quotient = numerator / denominator # shape (seq_len, d_model).
        ↪quotient[i, j] = numerator[i, 0] / denominator[0, j]
        self.pos[:, 0::2] = torch.sin(quotient) # assign result of trig to all
        ↪even indices for every token positions
        self.pos[:, 1::2] = torch.cos(quotient) # assign result of trig to all
        ↪odd indices for every token position

    def forward(self, x):
        """
        Input x is of shape (B, seq_len, d_embedding)
        """
        out = x + self.pos[:x.shape[1], :].to(device=x.device)
        return out
```

```
[83]: # Sanity check the shape of the PositionalEncoding output
set_seed(SEED)
pe = PositionalEncoding(512, max_len=1024)
```

```

input = torch.rand((10, 12, 512))
print(f"Shape of input: {input.shape}")

pe_output = pe(input)

print(f"Shape of positional encoding output: {pe_output.shape}")

assert pe_output.shape == input.shape, f"Expected output shape to have the same_
↪shape as the input {input.shape}, but got {pe_output.shape}"

```

```

Shape of input: torch.Size([10, 12, 512])
Shape of positional encoding output: torch.Size([10, 12, 512])

```

4.10 Question 3h: Blocks are All You Need! - TransformerEncoder

Finally, we can piece together the building blocks of our transformer!

We'll first stack an embedding layer, the `PositionalEncoding` layer we implemented, and a sequential list of the `EncoderLayers` we implemented together into a `TransformerEncoder` module.

Task: Implement the `TransformerEncoder` module. 1. Implement `__init__`. 1. Instantiate an embedding layer using `nn.Embedding`. This embedding layer will take in a token ID representing a certain token/word/ngram from our vocabulary, and output an embedding vector of size `d_model`. 2. Instantiate a positional encoding layer using the `PositionalEncoding` class we just implemented. 3. Instantiate `num_layers` `EncoderLayers`. 2. Implement `forward(x)`. 1. Embed the input sequence of tokens. 2. Add positional encodings. 3. Sequentially pass the result through each `EncoderLayer`. 4. Return the final tensor.

Hints: * If you want to store all your `EncoderLayers` in a list, make sure to use `nn.ModuleList` instead of a plain Python list. Otherwise, PyTorch won't register the parameters of your `EncoderLayers` and properly update them during backpropagation! * `nn.Embedding` takes in 2 parameters: `num_embeddings` is the number of unique tokens in your vocabulary, and `embedding_dim` is the dimension/length that each embedding vector should have. What parameters does the `__init__` method accept that correspond to the number of unique tokens/words, and the dimension of the embeddings? Note that the `embedding_dim` should be the same dimension as the vectors we pass into the `PositionalEncoding` and `EncoderLayer` layers later.

Expected behavior: * Input: `x` shape `(batch_size, seq_len)` with integer token IDs. * Output: shape `(batch_size, seq_len, d_model)` after all encoder layers.

```

[84]: class TransformerEncoder(nn.Module):
        def __init__(
            self, vocab_size=1024, d_model=512, num_layers=6, num_heads=8,
            ↪verbose=False
        ):

            # TODO: Implement the __init__ method
            super().__init__()
            self.embedding = nn.Embedding(vocab_size, d_model)

```

```

        self.positional_encoding = PositionalEncoding(d_model)
        self.encoder_layers = nn.
↪ModuleList([EncoderLayer(d_model,num_heads,verbose) for layer in
↪range(num_layers)])
        self.verbose = verbose

    def forward(self, x):

        # 1. Embed the input sequence of tokens
        x = self.embedding(x)
        # 2. Add positional encodings
        x = self.positional_encoding(x)
        # 3. Sequentially pass the result through each EncoderLayer
        for layer in self.encoder_layers:
            x = layer(x)
        # 4. Return the final tensor
        return x

```

```

[85]: # Sanity check the shape of the TransformerEncoder output
set_seed(SEED)

transformer_encoder = TransformerEncoder(vocab_size=1024, d_model=512,
↪num_layers=6, num_heads=8, verbose=True)

# Randomly generate a batch of token IDs with shape (B, num_tokens)
input = torch.randint(low=0, high=1024, size=(10, 100)) # (B, num_tokens)

transformer_encoder_output = transformer_encoder(input)

print(f"Shape of transformer encoder output: {transformer_encoder_output.
↪shape}")

assert transformer_encoder_output.shape == torch.Size([10, 100, 512]),
↪f"Expected output shape (B, num_tokens, d_model) {(10, 100, 512)}, but got
↪{transformer_encoder_output.shape}"

```

Shape of transformer encoder output: torch.Size([10, 100, 512])

4.11 Question 3i: TransformerDecoder

Similarly, we'll stack embedding layers, `PositionalEncodings`, a sequential list of `DecoderLayers`, and lastly a linear layer to predict the next token in our vocabulary into a `TransformerDecoder` module.

The decoder turns tokens from a target sequence—or autoregressive sequence of outputs it has generated so far—into contextualized representations using masked self-attention and cross-attention to the encoder output, then projects to vocabulary logits for next-token prediction.

Task: Implement the TransformerDecoder module. 1. Implement `__init__`. 1. Instantiate an embedding layer using `nn.Embedding`. 1. This embedding layer will take in a one-hot encoded vector of size `vocab_size`, and output an embedding vector of size `d_model`. 2. Instantiate a positional encoding layer using the `PositionalEncoding` class we just implemented. 3. Instantiate `num_layers` DecoderLayers. 4. Instantiate a final linear head to produce logits. 2. Implement `forward(x)`. 1. Embed the input sequence of tokens. 2. Add positional encodings. 3. Sequentially pass the result and the `encoder_output` through each `DecoderLayer`. 4. Apply the final linear layer to get logits over the vocabulary. 5. Return the final tensor.

Hints: * If you want to store all your `DecoderLayers` in a list, make sure to use `nn.ModuleList` instead of a plain Python list. Otherwise, PyTorch won't register the parameters of your `DecoderLayers` and properly update them during backpropagation! * We want the output of the final linear layer to be a probability distribution across all the possible words in our vocabulary. You can think of this as a classification problem: the transformer decoder is trying to predict which class (i.e. which word) the next token should be, based on the original source sequence and the tokens it's seen so far. What should the number of `out_features` be for our linear layer? * Don't forget to pass the `encoder_output` to each `DecoderLayer` as well! * Your implementation of the `TransformerDecoder` should look fairly similar to your `TransformerEncoder` :)

Expected behavior: * Inputs: `x` (`batch_size`, `seq_len`) (token IDs of dtype long), `encoder_output` (`batch_size`, `src_seq_len`, `d_model`) * After embedding + positional encoding: (`batch_size`, `seq_len`, `d_model`) * After stacked `DecoderLayers`: (`batch_size`, `seq_len`, `d_model`) * Final output (logits): (`batch_size`, `seq_len`, `vocab_size`)

```
[86]: class TransformerDecoder(nn.Module):
    def __init__(self, vocab_size=1024, d_model=512, num_layers=6, num_heads=8,
        verbose=False):

        # TODO: Implement the __init__ method
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.positional_encoding = PositionalEncoding(d_model)
        self.layers = nn.ModuleList([DecoderLayer(d_model, num_heads, verbose)
        for layer in range(num_layers)])
        self.final_linear = nn.Linear(d_model, vocab_size)
        self.verbose = verbose

    def forward(self, x, encoder_output):

        # 1. Embed the input sequence of tokens
        x = self.embedding(x)
        # 2. Add positional encodings
        x = self.positional_encoding(x)
        # 3. Sequentially pass the result through each DecoderLayer
        for layer in self.layers:
            x = layer(x, encoder_output)
        # 4. Apply the linear layer to predict the next token in our vocabulary
        x = self.final_linear(x)
```

```

# 5. Return the final tensor
return x

```

```

[87]: # Sanity check the shape of the TransformerDecoder output
set_seed(SEED)
transformer_decoder = TransformerDecoder(vocab_size=1024, d_model=512,
    ↪ num_layers=6, num_heads=8, verbose=True)

# Randomly generate a batch of token IDs with shape (B, num_tokens) to use as
    ↪ the target sequence we pass into the decoder
input = torch.randint(low=0, high=1024, size=(10, 100)) # (B, num_tokens)

# Randomly generate a batch of encoder outputs with shape (B, num_tokens,
    ↪ d_model)
encoder_output = torch.rand((10, 100, 512)) # (B, num_tokens, d_model)

# Pass the target sequence and encoder outputs through the decoder
transformer_decoder_output = transformer_decoder(input, encoder_output)

print(f"Shape of transformer decoder output: {transformer_decoder_output.
    ↪ shape}")

assert transformer_decoder_output.shape == torch.Size([10, 100, 1024]),
    ↪ f"Expected output shape (B, num_tokens, vocab_size) {(10, 100, 1024)}, but
    ↪ got {transformer_decoder_output.shape}"

```

Shape of transformer decoder output: torch.Size([10, 100, 1024])

4.12 Question 3j: Transforming it all together

Putting the Encoder and Decoder together, we can create one, big, awesome Transformer!

Task: Implement the Transformer module. 1. Implement `__init__`. 1. Instantiate a `TransformerEncoder` and a `TransformerDecoder`. 2. Implement `forward`. 1. Pass `x` through the encoder. 2. Pass `x` and the encoder's output through the decoder. 3. Return the output of the decoder.

```

[88]: class Transformer(nn.Module):
    def __init__(
        self,
        vocab_size=1024,
        d_model=512,
        num_encoder_layers=6,
        num_decoder_layers=6,
        num_heads=8,
        verbose=False,
    ):

```



```

        # TODO: Implement the __init__ method
        super().__init__()
        self.encoder = _
        ↪TransformerEncoder(vocab_size,d_model,num_encoder_layers,num_heads,verbose)
        self.decoder = _
        ↪TransformerDecoder(vocab_size,d_model,num_decoder_layers,num_heads,verbose)

    def forward(self, x):

        # TODO: Implement the forward method
        # 1. Pass `x` through the encoder
        enc_out = self.encoder(x)

        # 2. Pass `x` and the encoder's output through the decoder
        dec_out = self.decoder(x,enc_out)
        # 3. Return the output of the decoder
        return dec_out

```

```

[89]: # Sanity check the shape of the Transformer output
set_seed(SEED)

transformer = Transformer(
    vocab_size=1024,
    d_model=512,
    num_encoder_layers=6,
    num_decoder_layers=6,
    num_heads=8,
    verbose=True,
)

# Randomly generate a batch of token IDs with shape (B, num_tokens)
input = torch.randint(low=0, high=1024, size=(10, 100)) # (B, num_tokens)

# Pass the input through the transformer
transformer_output = transformer(input)

print(f"Shape of transformer output: {transformer_output.shape}")

assert transformer_output.shape == torch.Size([10, 100, 1024]), f"Expected _
    ↪output shape (B, num_tokens, vocab_size) {(10, 100, 1024)}, but got _
    ↪{transformer_output.shape}"

```

Shape of transformer output: torch.Size([10, 100, 1024])

4.13 Preparing a dataset to train the transformer

We'll load in a dataset called "TinyStories" from [Hugging Face](#), which contains a list of synthetically generated short stories.

Attribution: Eldan and Li, “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?”

```
[90]: tiny_stories = load_dataset("roneneldan/TinyStories", split='train')
      tiny_stories_small = tiny_stories.shuffle(seed=SEED).select(range(1000))
      print(tiny_stories_small)
```

```
Dataset({
  features: ['text'],
  num_rows: 1000
})
```

Let's check out one of the short stories in this dataset!

```
[91]: print(tiny_stories_small[0]['text'])
```

Tim and Mia like to play in the park. They see a big club on the ground. It is brown and long and heavy.

"Look, a club!" Tim says. "I can lift it!"

He tries to lift the club, but it is too tough. He falls down and drops the club.

"Ouch!" he says. "That hurt!"

Mia laughs. She is not mean, she just thinks it is funny.

"Let me try!" she says. "I can balance it!"

She picks up the club and puts it on her head. She walks slowly and carefully. She does not fall down.

"Wow!" Tim says. "You are good at balancing!"

"Thank you!" Mia says. "It is fun!"

They take turns balancing the club on their heads, arms, and legs. They have a lot of fun with the club. They are happy and proud. They are good friends.

We define a few helper functions to pre-process the text in our dataset for transformer training.

First, we'll convert all the short stories in our text into lowercase and collect them into a list.

Then, we need to **tokenize** our text data. **Tokenization** refers to splitting data into chunks and turning the “chunks” into numbers that we can feed into our model and embed. There's many different ways to tokenize data! For instance: * Character-level tokenization: Represent each character or letter in the alphabet with its own token * Byte Pair Encoding (BPE): Merge frequent character pairs to build subword units * Word-level tokenization: Represent each word as its own token * Phrase tokenization: Represent common phrases in a sentence as a token

In this homework, we'll keep things simple and turn each word into its own token. We need to determine the set of all unique words the comprise our vocabulary.

So we'll read each story in the `tiny_stories` dataset, collect all the paragraphs together, extract full words from each paragraph, and add them to a set.

Finally, we'll create a dictionary called `word_to_id` that maps words to a number. Later on, we can use `word_to_id` to find out which token ID each word corresponds to!

Note: We will also add some special tokens to our dictionary that represent the start of a sequence, pad tokens, and the end of a sequence. In training, these are often used to help delineate when phrases start/end!

```
[92]: # Convert all the short stories to lowercase and collect them into a list
tiny_stories_text = [story['text'].lower() for story in tiny_stories_small]

# Extract all full words from text
def extract_full_words(text):
    """
    Given a string, extracts full words from the string
    Example:
        text = "The cat in the hat category"
        output = ['The', 'cat', 'in', 'the', 'hat', 'category']
    """
    pattern = r'\b\w+\b'
    return re.findall(pattern, text)

# Tokenize tiny story text - word level
paragraphs = []
for story in tiny_stories_text:
    story_paragraphs = story.split("\n\n") # split to remove new lines
    paragraphs.extend(story_paragraphs)

vocab = set()
for paragraph in paragraphs:
    full_words = extract_full_words(paragraph.lower())
    vocab.update(full_words)

word_to_id = {word: i for i, word in enumerate(vocab)}

PAD_TOKEN = '<pad>'
START_TOKEN = '<start>'
END_TOKEN = '<end>'

word_to_id[PAD_TOKEN] = len(vocab)
word_to_id[START_TOKEN] = len(vocab) + 1
word_to_id[END_TOKEN] = len(vocab) + 2

vocab.update([PAD_TOKEN, START_TOKEN, END_TOKEN])
```

```

print(vocab)
print(f"Vocab size: {len(vocab)}")

print(f"Token ID of 'park': {word_to_id['park']}")
print(f"Token ID of 'wizard': {word_to_id['wizard']}")
print(f"Token ID of 'cat': {word_to_id['cat']}")
print(f"Token ID of 'dog': {word_to_id['dog']}")

```

```

{'humble', 'kiki', 'understand', 'remote', 'cheek', 'ripped', 'they', 'clothes',
'block', 'mall', 'teams', 'advice', 'indeed', 'frog', 'fabric', 'delivering',
'pouring', 'discovery', 'recognized', 'appeared', 'solving', 'soooooo',
'concede', 'softly', 'meadow', 'manage', 'signs', 'sniffed', 'seat', 'mothers',
'ago', 'thatâ', 'wealth', 'twirling', 'led', 'crash', 'pull', 'ambur',
'unexpected', 'mesmerised', 'wash', 'plant', 'same', 'desk', 'shopkeeper',
'whirring', 'soundly', 'shoulder', 'tony', 'aeroplanes', 'letâ', 'clapping',
'friendships', 'cheered', 'ourselves', 'horse', 'imagine', 'hunt', 'scenery',
'stone', 'wheel', 'ted', 'skip', 'towels', 'apart', 'saying', 'ambulance',
'hoop', 'tower', 'dragged', 'mask', 'licks', 'surprised', 'once', 'lion',
'agree', 'kicking', 'the', 'æhey', 'scoops', 'healed', 'stomps', 'noises',
'placing', 'wild', 'chest', 'stories', 'reminding', 'glittery', 'submarine',
'rescued', 'ben', 'twirl', 'pedaling', 'whiny', 'awe', 'reunite', 'requested',
'result', 'ordinary', 'hitting', 'trusty', 'full', 'wondered', 'soft',
'turning', 'tour', 'welcoming', 'flipped', 'names', 'falls', 'then', 'hadn',
'firefighters', 't', 'dry', 'æwhat', 'trampoline', 'fort', 'peered', 'felt',
'wednesday', 'distant', 'bandage', 'prevent', 'afraid', 'money', 'mountains',
'wand', 'mule', 'searched', 'birdsong', 'hola', 'cuddle', 'ferns', 'direction',
'soup', 'alligator', 'dances', 'going', 'secure', 'drown', 'confused',
'careful', 'took', 'wants', 'practicing', 'blazing', 'roads', 'thinks',
'ceiling', 'set', 'realised', 'stay', 'scream', 'unusual', 'spike',
'excitement', 'fierce', 'herbs', 'surprise', 'bar', 'attentively', 'down',
'motorcycle', 'asking', 'discussed', 'wooden', 'drinking', 'journeyed', 'tying',
'peaches', 'lawyer', 'zebras', 'difference', 'anyways', 'paste', 'crowded',
'adventurous', 'respectful', 'online', 'dressing', 'yawn', 'cubes', 'brush',
'sew', 'jerry', 'evening', 'chamber', 'organized', 'ducky', 'bedsheets',
'eaten', 'farm', 'excitedly', 'revealing', 'bumped', 'shined', 'says', 'city',
'leak', 'scrubbed', 'rough', 'circus', 'fragile', 'crackers', 'characters',
'blur', 'balls', 'entranced', 'soaring', 'star', 'push', 'troubled',
'encourage', 'our', 'brownie', 'worm', 'spilled', 'waves', 'joins', 'lift',
'ignore', 'spun', 'frosting', 'operations', 'tied', 'wind', 'snatched',
'africa', 'stood', 'buckets', 'messy', 'yummy', 'surroundings', 'hardest',
'squeaked', 'protect', 'shares', 'attached', 'disturbed', 'organizing',
'zipping', 'toolbox', 'smarter', 'shove', 'refused', 'tomato', 'moved', 'æwe',
'tablet', 'crawled', 'surprises', 'argued', 'caterpillar', 'leaned',
'restaurant', 'goodbye', 'ran', 'door', 'll', 'cackling', 'covered',
'challenges', 'louder', 'knocking', 'home', 'ædo', 'lia', 'movies', 'walkie',
'hopped', 'beam', 'tongue', 'wrote', 'markers', 'machine', 'wept', 'damaged',

```

'scampered', 'leaning', 'poles', 'judge', 'screamed', 'nowhere', 'wealthier',
 'usually', 'until', 'gadgets', 'separated', 'desperately', 'shoots',
 'surprisingly', 'complained', 'shrunk', 'bugs', 'ostrich', 'trunk', 'stretches',
 'backward', 'park', 'royal', 'age', 'though', 'meet', 'trouble', 'bakery',
 'judges', 'predators', 'runner', 'seas', 'game', 'outrun', 'working', 'olives',
 'construct', 'cut', 'falling', 'aid', 'flexible', 'pours', 'powers',
 'fisherman', 'flower', 'm', 'about', 'sang', 'path', 'experiences', 'built',
 'showed', 'scent', 'touches', 'paintings', 'track', 'rocky', 'staying', 'didn',
 'ruined', 'near', 'playroom', 'beautiful', 'compassionate', 'whip', 'closet',
 'pepper', 'scratches', 'follows', 'hurting', 'excuse', 'view', 'thicker',
 'because', 'fishermen', 'purpose', 'stealing', 'pipe', 'panic', 'tasted',
 'measuring', 'bump', 'garden', 'camera', 'sandy', 'healthy', 'weight', 'beat',
 'cauliflower', 'shall', 'kicks', 'needed', 'dancer', 'testing', 'winning',
 'christmas', 'grater', 'pigs', 'wood', 'using', 'either', '3', 'cop', 'pushes',
 'sophie', 'joe', 'covering', 'noticed', 'chat', 'knocks', 'reverser', 'wasnâ',
 'believed', 'leaves', 'dinosaur', 'show', 'zoo', 'jacket', 'jenny', 'others',
 'waited', 'sourest', 'remains', 'church', 'scissors', 'bathroom', 'opened',
 'nest', 'skyscraper', 'meal', 'high', 'improve', 'hospital', 'volcanoes',
 'tries', 'throwing', 'selling', 'popped', 'washing', 'plain', 'bleed',
 'deciding', 'fairy', 'faded', 'laser', 'flutter', 'sooner', 'gifted',
 'compared', 'market', 'sugar', 'equally', 'nana', 'flowery', 'oink', 'kitty',
 'reminded', 'sky', 'aimed', 'casting', 'guards', 'guided', 'supply', 'spring',
 'envelope', 'actually', 'begin', 'guys', 'jasmine', 'flips', 'acting', 'notice',
 'drive', 'owned', 'story', 'seemed', 'soared', 'sleepy', 'growing', 'master',
 'wagging', 'fire', 'work', 'apologize', 'fetching', 'surrounded', 'lie',
 'wheat', 'salt', 'poppies', 'hook', 'stable', 'a', 'hung', 'fountain', 'admit',
 'lady', 'managed', 'calendar', 'puddles', 'bowl', 'flags', 'tears',
 'firefighter', 'among', 'initials', 'tentacles', 'cracked', 'upon', 'handle',
 'arm', 'flag', 'hut', 'cuddles', 'whimpered', 'veterinarian', 'sneak', 'quack',
 'some', 'unlocked', 'prize', 'envious', 'giraffe', 'sizes', 'shields', 'cawing',
 'jolly', 'relief', 'consequences', 'objects', 'faster', 'sitting', 'salad',
 'airplane', 'helper', 'rolling', 'pilots', 'bent', 'wrist', 'sue', 'sunday',
 'streamers', 'paths', 'honey', 'afterwards', 'book', 'grounds', 'angelâ',
 'lollipop', 'swished', 'stuck', 'under', 'twin', 'shelter', 'rich', 'mushy',
 'competitive', 'pear', 'cranes', 'closed', 'cosy', 'original', 'waffle', 'goat',
 'crow', 'splashed', 'fascinating', 'packed', 'expression', 'shook', 'ordered',
 'hope', 'climbed', 'corn', 'scare', 'drives', 'petals', 'delicious', 'glared',
 'salsa', 'sandwich', 'truly', 'granddaughter', 'able', 'well', 'miaâ',
 'familiar', 'appear', 'quite', 'honored', 'crashing', 'teacher', 'invited',
 'neat', 'hose', 'shining', 'ticking', 'success', 'squirrels', 'singer',
 'follow', 'teach', 'science', 'animals', 'plenty', 'beamed', 'clipboard',
 'awhile', 'spell', 'fred', 'sand', 'paws', 'invitation', 'damp', 'sea',
 'scrambled', 'pulled', 'trail', 'accidentally', 'lila', 'æif', 'can', 'most',
 'post', 'brighter', 'cared', 'tables', 'puffing', 'creativity', 'arguing',
 'rude', 'milo', 'know', 'zipper', 'timmy', 'doing', 'saving', 'warnings',
 'liquid', 'remember', 'frowning', 'points', 'dog', 'rescuing', 'tuna', 'start',
 'sore', 'rose', 'crown', 'æthatâ', 'trade', 'wondering', 'collected', 'black',
 'arrow', 'extinguished', 'parades', 'cause', 'pleas', 'boys', 'questions',

'hoping', 'sorts', 'sparkles', 'escaped', 'warmer', 'steep', 'helpfulness',
 'unpause', 'rat', 'bruises', 'towards', 'till', 'spat', 'alien', 'hilarious',
 'biting', 'prayed', 'pounced', 'grade', 'another', 'wander', 'leo', 'daisy',
 'prayers', 'crumbs', 'duck', 'avoided', 'ice', 'cheer', 'cafã', 'changing',
 'beads', 'determination', 'taxi', 'bath', 'showers', 'keys', 'part', 'sorry',
 'learn', 'girl', 'exploring', 'quiet', 'holding', 'tomorrow', 'snake', 'snow',
 'feathers', 'napkin', 'hurts', 'agreement', 'ignoring', 'me', 'shelves', 'buys',
 'earn', 'colander', 'vet', 'busy', 'reverse', 'sandbox', 'stirred', 'melted',
 'loop', 'salads', 'search', 'restless', 'crack', 'pages', 'strong', 'road',
 'james', 'mermaid', 'moves', 'pretty', 'sueã', 'sheã', 'drawings', 'held',
 'hats', 'join', 'afternoons', 'ups', 'wealthy', 'æyay', 'couch', 'melt',
 'snapped', 'egg', 'wreck', 'feather', 'bobby', 'written', 'cable', 'seems',
 'driver', 'crew', 'threatened', 'exchange', 'entrance', 'anchor', 'loading',
 'radishes', 'spoke', 'drivers', 'spout', 'awake', 'æwell', 'instead', 'having',
 'dripping', 'sprung', 'josie', 'conflict', 'teaspoon', 'rudeness', 'unlock',
 'got', 'whispered', 'æwhy', 'celebrate', 'listened', 'train', 'dive', 'pony',
 'dare', 'cuts', 'strange', 'wilting', 'soda', 'ægreat', 'pouted',
 'neighbourhood', 'joyful', 'groceries', 'hurry', 'shot', 'maps', 'balanced',
 'glass', 'squealed', 'appreciation', 'becoming', 'plates', 'law', 'still',
 'nightstand', 'shop', 'parade', 'neighbors', 'fetch', 'attract', 'wonders',
 'collapsed', 'weak', 'mommyâ', 'gym', 'taped', 'hello', 'say', 'warning',
 'along', 'draw', 'parts', 'hanger', 'motor', 'final', 'soccer', 'cars',
 'vacuum', 'urges', 'cheated', 'was', 'highest', 'setting', 'compete', 'friendâ',
 'jem', 'stated', 'greedy', 'sally', 'key', 'shape', 'johnny', 'manner',
 'forgives', 'blackboard', 'map', 'selfish', 'charlie', 'support', 'kindly',
 'rumours', 'tight', 'dig', 'sneeze', 'universe', 'name', 'melon', 'stick',
 'dust', 'crucial', 'cannon', 'toast', 'test', 'pruned', 'increasing',
 'suggestion', 'raked', 'stars', 'interesting', 'values', 'hotel', 'cook',
 'mountain', 'sees', 'late', 'paul', 'clever', 'lap', 'pair', 'brightly', 'bows',
 'here', 'mole', 'downstairs', 'talkie', 'nectar', 'throwing', 'swayed', 'skirt',
 'checkup', 'prune', 'confusion', 'entering', 'wanted', 'bully', 'discover',
 'mmm', 'friendship', 'fireman', 'cheating', 'building', 'row', 'tickle', 'hear',
 'emily', 'moving', 'makes', 'forget', 'laughs', 'cross', 'lids', 'warn',
 'matching', 'doctor', 'divers', 'board', 'clean', 'promise', 'known',
 'standing', 'owners', 'performing', 'peeped', 've', 'æthank', 'valuable', 'he',
 'owl', 'clucking', 'hunters', 'accomplishment', 'suzy', 'noise', 'frogs',
 'flock', 'improving', 'ground', 'press', 'porch', 'reliable', 'plays',
 'healing', 'spared', 'rufus', 'third', 'gray', 'elevator', 'aliens', 'gasped',
 'hurt', 'hornet', 'creatureâ', 'fulfill', 'understood', 'tummies', 'usual',
 'jonathan', 'graceful', 'were', 'fought', 'meals', 'foxes', 'onward', 'hoped',
 'night', 'tiptoed', 'helped', 'provided', 'ray', 'brats', 'destination',
 'doors', 'reflection', 'spoilt', 'have', 'if', 'smell', 'supporting', 'folding',
 'crocodile', 'fences', 'weep', 'offered', 'achieve', 'children', 'invent',
 'knocked', 'tickets', 'nod', 'structure', 'creature', 'hand', 'yay', 'magic',
 'stitch', 'resisted', 'encouraged', 'baby', 'itself', 'stroked', 'rake',
 'against', 'fighters', 'drowned', 'end', 'fortunate', 'duckling', 'princess',
 'birds', 'flying', 'useful', 'dictionary', 'complete', 'ears', 'b', 'crying',
 'situation', 'focused', 'shaking', 'seashells', 'special', 'heater', 'hate',

'clutched', 'backpacks', 'ahead', 'barked', 'answers', 'pyjamas', 'into', 'us',
'taller', 'sack', 'places', 'beginning', 'talking', 'bring', 'whisk', 'hit',
'hills', 'gathering', 'mouth', 'seconds', 'woods', 'pieces', 'flicker', 'pray',
'pedals', 'rescue', 'harder', 'laughing', 'object', 'twins', 'hockey', 'woke',
'wags', 'tube', 'independent', 'tidy', 'scarf', 'launched', 'smooth', 'seesaws',
'whisper', 'thorns', 'concert', 'toilet', 'nicest', 'wiping', 'climbing',
'sweets', 'grab', 'skipped', 'seesaw', 'œâ', 'berries', 'pleasure', 'jewels',
'contest', 'exciting', 'slept', 'studying', 'forward', 'reach', 'hits', 'bin',
'includes', 'superhero', 'brother', 'monkey', 'themselves', 'difficult',
'ouchy', 'thing', 'missed', 'vroom', 'pushed', 'lunch', 'tim', 'jogging',
'wears', 'burger', 'succeeded', 'purred', 'mixing', 'bloom', 'surrender',
'helmet', 'nearly', 'vines', 'sunrise', 'or', 'marked', 'settle', 'raise',
'puff', 'jumps', 'fair', 'print', 'live', 'eat', 'punished', 'speak', 'dialed',
'power', 'morning', 'bouncy', 'builder', 'races', 'history', 'glasses', 'gods',
'shadow', 'tank', 'bites', 'people', 'mom', 'grew', 'words', 'freedom', 'leap',
'max', 'darker', 'hairy', 'stern', 'garbage', 'sure', 'house', 'strangers',
'nibbled', 'checking', 'dozed', 'castles', 'bag', 'food', 'buzz', 'tale',
'pile', 'swung', 'wagons', 'isla', 'summer', 'kissed', 'scattered', 'licking',
'pry', 'organize', 'cried', 'freeze', 'who', 'shore', 'cousins', 'experience',
'planes', 'longer', 'forgave', 'loose', 'wallpaper', 'bumps', 'shed',
'growling', 'ducks', 'happening', 'spots', 'expert', 'æthey', 'shallow',
'respected', 'flew', 'moment', 'decorate', 'crazier', 'foods', 'thin', 'light',
'throat', 'agreed', 'rabbit', 'seen', 'bridge', 'footprints', 'chair', 'joanna',
'cheese', 'claus', 'farmer', 'higher', 'rex', 'unite', 'barely', 'bears',
'turtle', 'introduced', 'demand', 'protected', 'spin', 'permission', 'nap',
'roundabout', 'moral', 'mattress', 'bags', 'stretching', 'drawn', 'smelled',
'suppose', 'gus', 'tip', 'jug', 'solemn', 'left', 'notes', 'naughty', 'david',
'claire', 'simple', 'polite', 'snack', 'flies', 'oats', 'razor', 'surface',
'mentioned', 'feeling', 'quick', 'teeth', 'spoon', 'belts', 'chocolate',
'pretends', 'pans', 'racer', 'finding', 'couldnâ', 'pup', 'went', 'serious',
'scaring', 'truth', 'take', 'forgive', 'responsibility', 'impossible', 'turns',
'act', 'takes', 'gift', 'munched', 'pointing', 'cooks', 'relieved', 'shrug',
'trying', 'booming', 'poking', 'waving', 'spear', 'swim', 'peanut', 'visitors',
'sort', 'despite', 'snowmen', 'hairstylist', 'tub', 'wearing', 'decorations',
'basement', 'following', 'seller', 'scooted', 'birdcages', 'lined', 'disobeyed',
'pin', 'drawer', 'unrolling', 'kick', 'girls', 'sock', 'snuggled', 'hours',
'nailed', 'snooze', 'mark', 'struggle', 'urge', 'continuing', 'comets',
'sneezes', 'ignored', 'fish', 'librarian', 'lizard', 'mysterious', 'raced',
'upset', 'bonfire', 'reminder', 'turn', 'anyone', 'rocks', 'calling', 'hugging',
'devastated', 'balloons', 'ideas', 'backpack', 'learnt', 'rowing', 'value',
'pearl', 'gained', 'enjoying', 'circle', 'reward', 'nail', 'outdoors', 'frank',
'pointed', 'culprit', 'grown', 'rubber', 'collecting', 'attic', 'whenever',
'get', 'cartoons', 'phone', 'tremble', 'frown', 'tag', 'expected', 'uses',
'tried', 'fever', 'lamp', 'luxury', 'worries', 'grumpily', 'scraped', 'card',
'unpacking', 'hammer', 'sauce', 'solve', 'fat', 'traffic', 'soap', 'feed',
'torn', 'rainbow', 'singing', 'dents', 'freely', 'leader', 'yells', 'sculpture',
'favourite', 'uniforms', 'idea', 'brown', 'licked', 'store', 'won', 'weighed',
'nine', 'this', 'ticket', 'afar', 'hold', 'his', 'berry', 'threw', 'frame',

'bill', 'read', 'tear', 'wished', 'blowing', 'bathtub', 'landed', 'whistle',
'trash', 'practiced', 'unpacked', 'practically', 'leg', 'ruining', 'eight',
'playful', 'gentle', 'opera', 'respect', 'airport', 'throws', 'spare', 'toward',
'dishes', 'chewing', 'thieves', 'faint', 'now', 'empty', 'knob', 'picking',
'paying', 'leading', 'æit', 'available', 'engine', 'zoe', 'warns', 'blows',
'nations', 'wouldnâ', 'cupboard', 'great', 'thoughtful', 'missing', 'sister',
'avocados', 'enjoy', 'tug', 'cells', 'beans', 'strength', 'thanked',
'explorers', 'dolphin', 'what', 'arranged', 'cheap', 'pour', 'friend', 'handed',
'itâ', 'also', 'showing', 'call', 'puddle', 'buying', 'fallen', 'gifts',
'scary', 'thought', 'lifts', 'milk', 'wouldn', 'sadness', 'tired', 'eager',
'poured', 'homes', 'bars', 'repeated', 'blinked', 'screams', 'touching',
'pastry', 'giggled', 'hat', 'ash', 'ann', 'worse', 'æyes', 'would', 'gum',
'jack', 'shoe', 'nicer', 'sharks', 'even', 'dried', 'handsome', 'tidying',
'deliver', 'rush', 'meaner', 'chirping', 'storm', 'ink', 'cheeks', 'cat',
'smile', 'trays', 'sights', 'young', 'salty', 'mirror', 'cereal', 'meerkats',
'queen', 'fog', 'blew', 'cushion', 'admiring', 'mary', 'rode', 'them', 'veil',
'heavily', 'turned', 'attach', 'crates', 'relax', 'penny', 'hopig', 'no',
'benny', 'quickly', 'exam', 'mix', 'haircut', 'squirrel', 'lot', 'throw',
'pinched', 'gates', 'jamie', 'tommy', 'filled', 'wednesdays', 'wobbled',
'gotten', 'strongest', 'everything', 'trick', 'bottle', 'warmly', 'touch',
'brick', 'joining', 'matilda', 'settled', 'rover', 'man', 'finds', 'holds',
'control', 'billy', 'means', 'huge', 'thrilled', 'playtime', 'daring', 'asleep',
'otter', 'reaches', 'boy', 'again', 'cream', 'snaps', 'loser', 'rachel',
'apple', 'sailing', 'flopsy', 'valley', 'bookstore', 'robin', 'pit', 'stitches',
'badly', 'yesterday', 'octopuses', 'bananas', 'mushroom', 'fluffy', 'those',
'monotonous', 'hopeful', 'pirates', 'birdie', 'sidewalk', 'colorsâ', 'blast',
'almost', 'wagon', 'dragon', 'sobbed', 'acorn', 'tiny', 'removing', 'caught',
'mood', 'hole', 'rips', 'æno', 'week', 'claws', 'rejoiced', 'traveling',
'inside', 'two', 'pay', 'hated', 'wore', 'anywhere', 'tools', 'scratchy',
'raven', 'nicky', 'jeff', 'receive', 'choose', 'outfit', 'carrying', 'crab',
'watch', 'blaring', 'banana', 'anyway', 'weren', 'song', 'terrified',
'straight', 'tom', 'joined', 'response', 'strangest', 'finally', 'knock',
'wall', 'ponies', 'forever', 'scooter', 'forgetting', 'bunch', 'sweetness',
'big', 'laugh', 'sit', 'reversing', 'frying', 'wires', 'savored', 'hero',
'scared', 'kept', 'bike', 'string', 'glided', 'adult', 'froze', 'homeless',
'instructions', 'wildlife', 'fin', 'whale', 'hesitated', 'preparing', 'mumma',
'behaving', 'exercises', 'sweet', 'forks', 'purse', 'whole', 'efforts', 'beef',
'sparkling', 'freed', 'jewel', 'refuse', 'place', 'expect', 'barking', 'heavy',
'cocoa', 'dolls', 'bear', 'weeks', 'pete', 'lucyâ', 'tackle', 'beaming', 'neck',
'top', 'sammyâ', 'message', 'gasps', 'spray', 'eve', 'shy', 'round', 'humming',
'stayed', 'demanded', 'patch', 'burned', 'thoroughly', 'hi', 'couple', 'hans',
'hopping', 'æemma', 'gazed', 'learning', 'save', 'æthat', 'flea', 'rule',
'knee', 'crunchy', 'regular', 'mia', 'enjoyable', 'log', 'mooing', 'stopped',
'boats', 'shoot', 'bangs', 'bug', 'for', 'used', 'him', 'clock', 'entered',
'sent', 'players', 'sorted', 'bits', 'æask', 'blocks', 'content', 'bubbles',
'gust', 'band', 'buckled', 'goodnight', 'masseur', 'sigh', 'tastiest', 'tune',
'colleen', 'side', 'protecting', 'sinks', 'thrill', 'music', 'are', 'poison',
'ending', 'jimmy', 'vegetables', 'resists', 'pies', 'joy', 'bit', 'tide',

'paddling', 'vehicle', 'quieter', 'muscles', 'sense', 'remembering', 'chompy',
 'during', 'repeat', 'street', 'delight', 'kitchen', 'thank', 'you', 'secrets',
 'slides', 'scolded', 'crawling', 'seem', 'branches', 'evenings', 'pigeons',
 'boom', 'pond', 'floor', 'works', 'am', 'chubby', 'love', 'stomping', 'proven',
 'sydney', 'decide', 'lemonade', 'let', 'flash', 'finished', 'bow', 'tool',
 'waffles', 'broken', 'gas', 'shaping', 'videos', 'cautiously', 'next',
 'grandma', 'refreshing', 'play', 'make', 'tons', 'creating', 'count', 'twirled',
 'roll', 'cookies', 'throughout', 'when', 'anymore', 'flames', 'andy', 'grabs',
 'loosened', 'swamp', 'skipping', 'immense', 'office', 'puzzle', 'chose',
 'displayed', 'making', 'listen', 'quarrelin', 'fiddle', 'clown', 'overflow',
 'clap', 'crowd', 'coins', 'explanation', 'cries', 'comparison', 'weather',
 'powder', 'line', 'unkind', 'bend', 'stuff', 'far', 'practice', 'stamps',
 'rooms', 'warned', 'gives', 'network', 'tranquillity', 'found', 'followed',
 'telling', 'courageous', 'balancing', 'stomach', 'opening', 'thump', 'cannot',
 'curtains', 'mighty', 'fruit', 'doesn', 'capacity', 'learned', 'fearful',
 'decides', 'sara', 'lola', 'tail', '<start>', 'flour', 'bikes', 'accidents',
 'cooking', 'grinned', 'celebrating', 'misses', 'temple', 'incredibly', 'hips',
 'cartoon', 'roof', 'crept', 'ripples', 'bounces', 'offer', 'crosses', 'brings',
 'strolling', 'streaked', 'women', 'rubs', 'determined', 'air', 'dishwasher',
 'officer', 'reveal', 'hardworking', 'only', 'servant', 'baked', 'towel',
 'hello', 'dips', 'poked', 'watering', 'lips', 'dancing', 'burns', 'shuffled',
 'rip', 'calmed', 'seagulls', 'sending', 'sadly', 'stairs', 'added', 'miss',
 'cleaning', 'breadcrumbs', 'puts', 'parents', 'break', 'reddest', 'slapped',
 'anxious', 'sprinkles', 'vanished', 'swims', 'sunlight', 'backyard', 'grumpy',
 'since', 'snippets', 'prince', 'year', 'hay', 'classroom', 'moments', 'fly',
 'oven', 'tweetie', 'brave', 'tingling', 'decision', 'stretched', 'kings',
 'spaceman', 'visiting', 'quietly', 'sudden', 'frozen', 'float', 'bedroom',
 'jelly', 'driftwood', 'stops', 'embarrassed', 'fast', 'sorting', 'stare',
 'goodies', 'sausage', 'slider', 'everyone', 'emerged', 'dreams', 'defending',
 'faucet', 'green', 'helpless', 'bench', 'scoop', 'than', 'dinosaurs', 'observe',
 'excited', 'paused', 'explorer', 'find', 'pride', 'disobedient', 'jane', 'barn',
 'enjoyed', 'times', 'rushing', 'lab', 'square', 'growled', 'looking', 'castle',
 'license', 'pool', 'tissues', 'taking', 'everywhere', 'stand', 'dull', 'bed',
 'pick', 'herb', '<end>', 'wow', 'ea', 'tortoise', 'loud', 'birthday',
 'grabbing', 'helicopter', 'crossed', 'crazy', '<pad>', 'other', 'beside',
 'grin', 'bippy', 'died', 'tarp', 'being', 'ship', 'bounce', 'beloved', 'chewed',
 'shiver', 'convince', 'triangle', 'mittens', 'mystery', 'yawned', 'maria',
 'match', 'fascinated', 'ages', 'enthusiastic', 'snacks', 'explored',
 'hesitantly', 'resume', 'merry', 'worshipped', 'chasing', 'increase',
 'released', 'pretend', 'cup', 'spy', 'cost', 'removed', 'math', 'listens',
 'stranger', 'pets', 'genie', 'rot', 'stew', 'mild', 'serve', 'particular',
 'perched', 'unique', 'safer', 'knowing', 'zero', 'seal', 'jim', 'socks',
 'appreciate', 'put', 'word', 'face', 'caring', 'loneliness', 'close', 'what',
 'knife', 'march', 'be', 'ocean', 'peach', 'stands', 'sighed', 'meeting',
 'spies', 'lay', 'dark', 'calm', 'sunsets', 'oily', 'promised', 'paddles',
 'many', 'news', 'driving', 'tough', 'carrot', 'crawl', 'floated', 'harsh',
 'marriages', 'create', 'hallway', 'tap', 'costume', 'bridges', 'country',
 'waiter', 'breaks', 'juice', 'cow', 'mail', 'celebrated', 'mother', 'stumbled',

'jerk', 'pink', 'rang', 'pack', 'belongings', 'nickered', 'mission', 'club',
 'drops', 'football', 'paper', 'exploded', 'center', 'dipped', 'leapt',
 'provide', 'rebuild', 'oil', 'brothers', 'wool', 'journey', 'wave',
 'thankfully', 'balance', 'port', 'behaved', 'while', 'tapes', 'fan',
 'impressive', 'traded', 'annoying', 'tuesday', 'drink', 'æletâ', 'breathing',
 'colorful', 'wish', 'fight', 'crystal', 'kisses', 'chews', 'exact', 'guilty',
 'swallowed', 'watches', 'lean', 'route', 'squeezing', 'goner', 'lost', 'done',
 'effort', 'wrecked', 'energetic', 'spotted', 'entire', 'because', 'tunnels',
 'lies', 'toadstool', 'toby', 'strapped', 'hive', 'trust', 'unfortunately',
 'tire', 'hedge', 'brain', 'fruits', 'oat', 'scooped', 'mandy', 'kinds',
 'treasures', 'spider', 'counting', 'cats', 'replaced', 'bicycle', 'medal',
 'dots', 'stored', 'honked', 'likes', 'eagerly', 'intricate', 'daunting',
 'became', 'shared', 'playing', 'rusty', 'television', 'dragonfly', 'grass',
 'wake', 'slip', 'den', 'boots', 'picket', 'filthy', 'best', 'regretted',
 'princesses', 'twinkle', 'pastries', 'heels', 'duckie', 'jealousy', 'wings',
 'vacation', 'cherries', 'filling', 'whistling', 'coax', 'seeds', 'elevators',
 'attention', 'æfollow', 'rosemary', 'thunder', 'elephant', 'zip', 'spoken',
 'mommy', 'ear', 'painter', 'sneezed', 'vinegar', 'perhaps', 'lets', 'is', 'ha',
 'prickly', 'size', 'lunchbox', 'running', 'cutting', 'roar', 'nice', 'candles',
 'presented', 'dads', 'patterns', 'struggled', 'sheep', 'suns', 'pattern',
 'station', 'tweety', 'with', 'short', 'digging', 'budge', 'disappear', 'town',
 'piano', 'planning', 'gain', 'piece', 'hop', 'counter', 'happen', 'prepare',
 'sturdy', 'calls', 'thoughts', 'shaped', 'heading', 'starts', 'karl', 'æyouâ',
 'nicely', 'fancy', 'orange', 'colours', 'tell', 'ok', 'cloth', 'island',
 'meerkat', 'jungle', 'motorcycles', 'potion', 'brought', 'damage', 'helps',
 'savoring', 'sank', 'batch', 'plopped', 'whispers', 'its', 'stop', 'somebody',
 'bottom', 'visited', 'areas', 'puzzled', 'powerful', 'helmets', 'shone',
 'rumble', 'drawing', 'cautious', 'planted', 'paints', 'helping', 'flood',
 'letting', 'glued', 'sticks', 'catching', 'sneezing', 'dinner', 'beep', 'her',
 'workshop', 'remy', 'supplied', 'needles', 'bother', 'maggie', 'do', 'eyes',
 'honest', 'modest', 'nobody', 'incredible', 'drill', 'æambur', 'worry',
 'creaked', 'ow', 'enter', 'mattresses', 'release', 'bossy', 'dropped', 'jumpy',
 'glowed', 'stared', 'umbrella', 'earlier', 'dove', 'letters', 'win', 'around',
 'rainbows', 'flashlight', 'men', 'lives', 'tells', 'box', 'ring', 'out',
 'passengers', 'exploration', 'she', 'cuddled', 'sofa', 'coyote', 'dr', 'rub',
 'hugged', 'truck', 'believe', 'feeding', 'cool', 'knight', 'cookie', 'crane',
 'white', 'through', 'laundry', 'low', 'treats', 'puzzles', 'pillows', 'ashamed',
 'stubbornness', 'chance', 'pressed', 'middle', 'pot', 'everybody', 'healthier',
 'dryer', 'lemon', 'parted', 'uncomfortable', 'eyepiece', 'twinkling',
 'shortcuts', 'arrive', 'escort', 'opens', 'ball', 'nosy', 'bored', 'banging',
 'ate', 'meow', 'glad', 'sweetie', 'theirs', 'married', 'wakes', 'sticker',
 'grill', 'luggage', 'siren', 'help', 'cardboard', 'interested', 'copying',
 'unsure', 'yoga', 'careless', 'grip', 'model', 'carried', 'soldier', 'right',
 'trees', 'split', 'staring', 'mean', 'afloat', 'envelopes', 'library', 'both',
 'chirped', 'swap', 'loyal', 'bob', 'why', 'shapes', 'loving', 'greater',
 'wiggled', 'tomyâ', 'batter', 'cursed', 'school', 'repaired', 'creatures',
 'pleaded', 'glisten', 'copper', 'tanks', 'annabelle', 'continued', 'wishing',
 'gone', 'grapes', 'past', 'aware', 'sir', 'wise', 'widened', 'doggy', 'buckle',

'friends', 'forgets', 'several', 'police', 'zooming', 'burn', 'enemies',
 'without', 'giggles', 'knives', 'cory', 'patient', 'body', 'jump', 'case',
 'color', 'facts', 'cannonballs', 'chirp', 'ingredients', 'afternoon', 'bread',
 'quicken', 'lonely', 'lazy', 'remove', 'courage', 'weapon', 'spaghetti',
 'didnâ', 'delayed', 'threat', 'belonged', 'packets', 'leaving', 'guns',
 'project', 'itchy', 'sides', 'shoulders', 'feelings', 'bus', 'giant',
 'restored', 'woman', 'served', 'tips', 'envy', 'bitter', 'pots', 'john',
 'giving', 'growls', 'go', 'slippery', 'fill', 'tunes', 'sold', 'time', 'corner',
 'overcome', 'ellie', 'worth', 'bald', 'houses', 'listening', 'accept',
 'raincoats', 'fields', 'addition', 'bravely', 'listed', 'attendant', 'use',
 'normal', 'anna', 'imagining', 'tugging', 'relying', 'rod', 'anything', 'blow',
 'slide', 'sheet', 'sends', 'question', 'successful', 'cakes', 'occasionally',
 'wardrobe', 'chase', 'winds', 'smells', 'trusted', 'your', 'flapped', 'cast',
 'stretch', 'myself', 'jones', 'bought', 'closely', 'drop', 'tea', 'dressy',
 'purple', 'add', 'piles', 'vase', 'warm', 'pain', 'dressed', 'magnets',
 'fishing', 'isn', 'songs', 'slammed', 'tie', 'chairs', 'challenge', 'squeeze',
 'scrub', 'comes', 'extraordinary', 'towns', 'steps', 'insects', 'broccoli',
 'ventured', 'occasions', 'luck', 'off', 'slowly', 'jar', 'sale', 'brrr', 'it',
 'oh', 'emma', 'awful', 'harm', 'roars', 'wrap', 'sandcake', 'shown', 'way',
 'keen', 'need', 'kingdom', 'admired', 'tickled', 'talked', 'firm', 'fisher',
 'cans', 'roaring', 'glues', 'before', 'tweet', 'new', 'æyou', 'hides', 'cards',
 'might', 'annoyed', 'daddy', 'towers', 'neighbor', 'spreading', 'shiny',
 'suffer', 'printed', 'years', 'mouths', 'picked', 'shoes', 'nose', 'forest',
 'sunflower', 'returned', 'cute', 'formed', 'mud', 'lions', 'borrow', 'shells',
 'swept', 'artist', 'days', 'escape', 'connected', 'happened', 'minute',
 'sinking', 'sword', 'shakes', 'cape', 'half', 'æmommy', 'supplies', 'nickel',
 'lunchtime', 'items', 'dad', 'twist', 'reunited', 'costumes', 'bathrobe',
 'splashes', 'yourselves', 'remembers', 'accepts', 'hesitate', 'creative',
 'voices', 'squabbling', 'onwards', 'circles', 'beanstalk', 'punish', 'sunny',
 'dan', 'today', 'learner', 'very', 'butterflies', 'unpacker', 'knowledge',
 'karen', 'fox', 'dream', 'competition', 'cups', 'strawberry', 'branch', 'sing',
 'important', 'become', 'few', 'intrigued', 'blinking', 'games', 'æwait',
 'ælook', 'cashier', 'tighten', 'curiously', 'shrimp', 'alarm', 'kicked',
 'endless', 'spinning', 'meant', 'teaches', 'rosy', 'closer', 'course',
 'marching', 'rink', 'swings', 'understands', 'secret', 'insists', 'gold',
 'brilliant', 'mamaâ', 'exercising', 'bandages', 'cleaned', 'pure', 'raft',
 'give', 'imagination', 'these', 'shovels', 'dreamed', 'plastic', 'packing',
 'think', 'copy', 'pace', 'chips', 'peeking', 'seasoning', 'shower', 'fits',
 'service', 'candy', 'just', 'æoh', 'really', 'peace', 'bigger', 'pinchy',
 'legs', 'performance', 'circled', 'dresses', 'colors', 'mesmerized', 'susie',
 'foot', 'planned', 'move', 'annie', 'dangerous', 'wishes', 'puppy', 'brushed',
 'streets', 'suspicion', 'feel', 'breathe', 'lock', 'daisies', 'glows', 'hair',
 'wallet', 'jackets', 'splash', 'unzipped', 'eating', 'lead', 'kid', 'modern',
 'yummier', 'floaty', 'ignorant', 'family', 'nods', 'æbut', 'harmless',
 'balloon', 'parks', 'mixer', 'each', 'competitiveness', 'bunny', 'intelligent',
 'ripe', 'talent', 'awesome', 'cane', 'struggling', 'tunnel', 'gloomy',
 'causing', 'dragons', 'squares', 'stack', 'strategy', 'finger', 'open', 'tours',
 'equipment', 'granted', 'perfect', 'pliers', 'frustrated', 'whiskers',

'rumbled', 'sour', 'moonlight', 'mistakes', 'boss', 'realizing', 'five',
'state', 'newspapers', 'land', 'venture', 'chalk', 'attractive', 'fix',
'recover', 'alive', 'computer', 'grains', 'journeys', 'betsy', 'invite', 'trap',
'hall', 'shocked', 'wait', 'skills', 'sweetest', 'probably', 'ruler', 'restore',
'field', 'speaking', 'tape', 'hunger', 'plants', 'bandit', 'coming', 'problem',
'happy', 'summit', 'lovely', 'changed', 'earth', 'froggy', 'spiky',
'herbivores', 'understanding', 'tosses', 'slid', 'sailor', 'applaud', 'sleep',
'discovered', 'safe', 'coast', 'newspaper', 'solved', 'cupped', 'called',
'freezer', 'etomorrow', 'susan', 'summertime', 'honking', 'clapped', 'image',
'snoring', 'energized', 'captain', 'muffy', 'back', 'cave', 'lasted', 'plane',
'ladybug', 'paddle', 'proudly', 'congratulated', 'wolves', 'complain', 'player',
'sprinkled', 'glee', 'habit', 'disappointment', 'adults', 'colored', 'eater',
'liars', 'ready', 'carefully', 'behave', 'collect', 'pause', 'smaller',
'lollipops', 'molly', 'gradually', 'supposed', 'vine', 'pasta', 'comforted',
'silver', 'ghost', 'ollie', 'dam', 'startled', 'theyâ', 'sarah', 'smiled',
'trophy', 'kate', 'breakfast', 'bobo', 'safety', 'blocked', 'wire', 'joker',
'meanwhile', 'whined', 'smith', 'lines', 'leash', 'disturb', 'elle', 'fine',
'at', 'youth', 'poke', 'separate', 'shouldn', 'sticky', 'keeps', 'sneered',
'timer', 'seats', 'creamy', 'safari', 'fastest', 'chloe', 'beach', 'which',
'pushing', 'patted', 'minutes', 'gun', 'nuzzled', 'fresh', 'mold', 'allowed',
'stickers', 'hardly', 'broke', 'shout', 'stove', 'united', 'sink', 'failed',
'purrs', 'saved', 'that', 'closes', 'cover', 'bye', 'snap', 'happiest', 'older',
'amusement', 'file', 'whoever', 'nothing', 'flashing', 'small', 'their',
'putting', 'zooms', 'fries', 'cara', 'searching', 'mummy', 'lick', 'god',
'ache', 'letter', 'temperature', 'breaking', 'wide', 'sometimes', 'sandpit',
'messier', 'grins', 'recommended', 'sail', 'fed', 'art', 'ruin', 'praised',
'stains', 'tornado', 'eraser', 'disgusting', 'runs', 'always', 'cuddly',
'separating', 'kneel', 'bird', 'meaning', 'wag', 'paddled', 'named', 'igloo',
'someone', 'looser', 'grows', 'hungry', 'pale', 'company', 'extra', 'measure',
'achoo', 'reassuring', 'mice', 'sparkly', 'send', 'shouting', 'loudest',
'enthusiasm', 'quiz', 'oasis', 'æi', 'heavier', 'silence', 'bubble', 'stripes',
'comfort', 'on', 'guitar', 'above', 'sign', 'goes', 'musician', 'fear',
'counted', 'maya', 'i', 'slice', 'rules', 'conductor', 'try', 'said',
'explained', 'novels', 'perfectly', 'sharing', 'refrigerator', 'stung',
'coloured', 'coughed', 'homework', 'rider', 'pigeon', 'hesitating', 'fireplace',
'ours', 'giraffes', 'satisfied', 'bracelet', 'sunshine', 'mind', 'reef',
'record', 'pen', 'reliably', 'nearby', 'lying', 'perfume', 'soak', 'labour',
'everyday', 'sometime', 'one', 'fork', 'dirt', 'deaf', 'gate', 'pirate',
'planting', 'nibbling', 'ways', 'afford', 'mum', 'rest', 'jacob', 'of',
'responsible', 'breeze', 'marry', 'shine', 'overjoyed', 'diamond', 'order',
'steering', 'tracks', 'taken', 'basket', 'pie', 'tree', 'junk', 'becomes',
'touched', 'twisting', 'applauded', 'thankful', 'painted', 'hiding', 'bright',
'shyly', 'wands', 'pole', 'attacking', 'disbelief', 'mailbox', 'tease',
'cabinet', 'eggs', 'choppy', 'wizard', 'swirling', 'had', 'climb', 'voice',
'better', 'silly', 'spoiled', 'bleeding', 'trip', 'battery', 'tiger', 'grandpa',
'patiently', 'teaching', 'write', 'exit', 'farther', 'dusty', 'lake', 'hamster',
'note', 'father', 'maybe', 'smiles', 'amazing', 'dropping', 'shooting',
'village', 'lights', 'has', 'hornets', 'glistened', 'prettiest', 'watching',

'dottie', 'glimmering', 'sighs', 'owner', 'spend', 'wondrous', 'behavior',
'spoil', 'cart', 'lend', 'lightning', 'symbol', 'skill', 'deeper', 'thief',
'tightly', 'little', 'amy', 'peacefully', 'revealed', 'sweater', 'jackie',
'display', 'willing', 'ride', 'rings', 'kindest', 'click', 'yippee', 'clearing',
'crossing', 'crumb', 'kind', 'swam', 'fairies', 'fashion', 'productive', 'donâ',
'ribbon', 'experienced', 'suddenly', 'suggested', 'yuck', 'trains',
'appreciated', 'edge', 'whatever', 'stream', 'hopes', 'jig', 'ties',
'recommend', 'hid', 'boring', 'kite', 'wonder', 'desserts', 'walked', 'four',
'feet', 'bodies', 'been', 'hesitant', 'played', 'â', 'already', 'fills',
'jokes', 'lied', 'statue', 'playground', 'deep', 'as', 'stronger', 'toothbrush',
'succeed', 'scales', 'pancakes', 'betty', 'knew', 'angel', 'instantly', 'blue',
'rays', 're', 'tissue', 'terrible', 'pears', 'charming', 'locked', 'rubbed',
'own', 'refreshed', 'bowed', 'measured', 'an', 'unfulfilled', 'else',
'enormous', 'softer', 'pumped', 'rebuilt', 'dough', 'mixed', 'thinking',
'stool', 'mending', 'disagreed', 'spent', 'parrot', 'queens', 'run', 'sagging',
'meowed', 'blown', 'wagged', 'cherry', 'honesty', 'polish', 'rushed', 'bees',
'crib', 'designer', 'load', 'cheerful', 'included', 'monkeys', 'given',
'marking', 'families', 'over', 'mouse', 'amen', 'none', 'ends', 'child',
'giggle', 'baskets', 'replied', 'vowed', 'knows', 'lip', 'resist', 'good',
'bang', 'screw', 'wildly', 'pulling', 'however', 'lurking', 'plucked',
'brushes', 'horses', 'keep', 'ones', 'curled', 'dreaming', 'prepared', 'gem',
'poor', 'memories', 'monsters', 'together', 'window', 'answer', 'speck',
'messages', 'ælet', 'smelly', 'room', 'penguin', 'frightened', 'doctors',
'leave', 'hanging', 'saw', 'grace', 'jingled', 'drum', 'did', 'furry',
'answered', 'prey', 'scatter', 'toys', 'ouch', 'language', 'remain', 'missile',
'mistake', 'shrugged', 'across', 'aren', 'must', 'wink', 'every', 'coin',
'preschool', 'study', 'scurried', 'bell', 'favorite', 'untied', 'asks',
'destroyed', 'dead', 'heads', 'spot', 'reluctant', 'season', 'to', 'oak',
'numbers', 'janey', 'process', 'plans', 'pip', 'ant', 'investigate', 'lamb',
'bowls', 'buttons', 'old', 'hard', 'sandwiches', 'shines', 'ask', 'funniest',
'should', 'may', 'explains', 'tricks', 'mattered', 'below', 'clearly', 'belong',
'jogged', 'heel', 'carry', 'cones', 'menu', 'sunglasses', 'designs', 'pan',
'repairman', 'coats', 'æhi', 'hav', 'six', 'decorated', 'fridge', 'hang', 'any',
'yucky', 'stamp', 'crunching', 'potatoes', 'swooped', 'aeroplane', 'santa',
'darling', 'rivers', 'shrink', 'hedgehog', 'ugly', 'jars', 'realise', 'looks',
'robots', 'mr', 'paintbrush', 'hunter', 'wasn', 'windows', 'came', 'shock',
'wobble', 'importance', 'pinky', 'inviting', 'keychains', 'toes', 'code',
'pairs', 'sparked', 'comfortable', 'hug', 'cocoon', 'starting', 'gathered',
'floors', 'approached', 'dolphins', 'discussing', 'coloring', 'ladder', 'waste',
'faraway', 'ramp', 'achieved', 'dies', 'firmly', 'obedient', 'george',
'underneath', 'baking', 'waiting', 'accused', 'but', 'in', 'walk', 'sam',
'action', 'petted', 'attaching', 'completed', 'peeked', 'task', 'funny',
'shimmied', 'melting', 'walls', 'happens', 'shades', 'louie', 'mention', 'mess',
'nurse', 'knights', 'straightened', 'pretended', 'bury', 'design', 'closest',
'apologized', 'wonderful', 'tossed', 'reached', 'buddy', 'dived', 'cloud',
'cold', 'dish', 'encountered', 'hears', 'lulu', 'backwards', 'hearts', 'heat',
'repeating', 'œiâ', 'rune', 'how', 'looked', 'loves', 'wife', 'possible',
'discouraged', 'front', 'icy', 'treasure', 'admire', 'steady', 'disappointed',

'classmates', 'rare', 'wandered', 'bounced', 'early', 'return', 'deer', 'abby',
'crayons', 'after', 'holidays', 'cabin', 'faithful', 'nightfall', 'breath',
'rug', 'adventures', 'jumped', 'team', 'okay', 'tweeting', 'weeping', 'flap',
'snorted', 'heartbroken', 'headed', 'duster', 'heard', 'seek', 'seahorse',
'swapped', 'mustache', 'inspired', 'free', 'super', 'fun', 'fuel', 'sounding',
'travelled', 'brand', 'streetlight', 'clara', 'steam', 'rustle', 'sweetheart',
'remind', 'soil', 'funnel', 'hatched', 'welcomed', 'weird', 'leads', 'build',
'giggling', 'goodbye', 'forgotten', 'bad', 'shell', 'needle', 'come', 'x',
'napping', 'wow', 'fur', 'winner', 'obeying', 'loved', 'seed', 'jumping',
'unpack', 'does', 'buildings', 'famous', 'doll', 'problems', 'net', 'boat',
'trim', 'angry', 'behind', 'comfy', 'folded', 'reversed', 'deed', 'bringing',
'later', 'shake', 'roared', 'swinging', 'danger', 'petting', 'laps', 'rocket',
'spending', 'performed', 'materials', 'breezes', 'chess', 'automobile', 'rise',
'yell', 'bleeds', 'destruction', 'passing', 'drinks', 'whizzing', 'look',
'sound', 'colourful', 'properly', 'cake', 'worked', 'firemen', 'passed',
'placed', 'reliability', 'peel', 'outside', 'chew', 'friendly', 'more', 'glue',
'picks', 'tricky', 'dance', 'disappeared', 'confronted', 'slipped', 'books',
'my', 'reaching', 'buy', 'chores', 'least', 'danced', 'laughed', 'activity',
'vendor', 'ancient', 'everytime', 'wolf', 'real', 'recovered', 'palace',
'golden', 'less', 'please', 'sting', 'popping', 'nemo', 'step', 'lee', 'pine',
'trembling', 'pleased', 'oxygen', 'sailboat', 'river', 'grocery', 'nature',
'waved', 'clay', 'focus', 'immediately', 'life', 'disappearance', 'decreased',
'masterpiece', 'yours', 'much', 'glittering', 'tutor', 'smoke', 'prettier',
'whistled', 'grow', 'noses', 'scolds', 'space', 'avocado', 'curtain', 'regret',
'wrapped', 'meat', 'angrily', 'oyster', 'spills', 'stuffed', 'things',
'approval', 'growl', 'goo', 'business', 'laid', 'guard', 'types', 'clumsy',
'curiosity', 'enthusiastically', 'sounded', 'glow', 'allow', 'sweeping', 'jill',
'sandcastles', 'second', 'paint', 'comforting', 'figure', 'metal', 'hands',
'we', 'himself', 'coffee', 'wipe', 'hollow', 'liked', 'lucy', 'yelled', 'sun',
'cloudy', 'gesture', 'seven', 'rotting', 'attack', 'crashed', 'kindness',
'chillies', 'rhythm', 'can', 'worried', 'perform', 'sat', 'lots', 'matter',
'cube', 'misty', 'vast', 'blanket', 'saturday', 'tucked', 'papers', 'tastes',
'taste', 'd', 'sticking', 'swimming', 'muffins', 'galloping', 'planets', 'quit',
'somewhere', 'everything', 'cozy', 'witch', 'wet', 'trotted', 'spicy', 'poses',
'underwater', 'squash', 'keepers', 'steak', 'onto', 'mist', 'forehead',
'teasing', 'precious', 'dependable', 'ladle', 'mastered', 'ruby', 'gasp',
'thumb', 'mama', 'uncles', 'garage', 'theater', 'between', 'musical', 'leaf',
'beds', 'toy', 'encouragement', 'ia', 'crowns', 'aside', 'okay', 'tempting',
'receiving', 'water', 'property', 'permitted', 'perch', 'penguins', 'velvet',
'scratched', 'moms', 'told', 'mine', 'actions', 'drifted', 'imagined', 'marks',
'shark', 'repair', 'head', 'slices', 'cops', 'distance', 'rely', 'gets',
'panted', 'holes', 'nervous', 'splashing', 'fence', 'bake', 'could', 'security',
'jog', 'lowered', 'couldn', 'rubbing', 'nights', 'official', 'fixed',
'microscope', 'banged', 'holiday', 'zebra', 'picnics', 'frowned', 'caused',
'magical', 'zipped', 'ever', 'basics', 'scratch', 'reluctantly', 'argue',
'volcano', 'happiness', 'rains', 'jake', 'share', 'soar', 'fires', 'texture',
'shouted', 'stomachs', 'healer', 'reading', 'tummy', 'wherever', 'too', 'drew',
'waist', 'amazement', 'caution', 'especially', 'blankets', 'stubborn',

'dentist', 'preaching', 'introduce', 'alone', 'butter', 'pass', 'whites',
'relaxed', 'amazed', 'climbs', 'tall', 'yard', 'storekeeper', 'created',
'jackâ', 'animal', 'operation', 'living', 'world', 'bat', 'son', 'woken',
'museum', 'tractor', 'roofs', 'suffered', 'gather', 'lapped', 'insect',
'monster', 'checked', 'burning', 'echoed', 'stacked', 'needs', 'drove',
'beauty', 'movie', 'adorable', 'menus', 'æof', 'job', 'nuts', 'kiss', 'pebble',
'tested', 'from', 'nails', 'met', 'lively', 'carriage', 'delivered', 'group',
'person', 'sweeties', 'dirtier', 'rolled', 'lid', 'whirred', 'hurried', 'arms',
'plan', 'knot', 'examine', 'smartness', 'writing', 'will', 'hugs', 'sailed',
'choice', 'check', 'floating', 'peppers', 'mrs', 'alright', 'alert', 'jon',
'hairs', 'ribbons', 'prove', 'fighting', 'gazing', 'delay', 'failing', 'æsam',
'spread', 'snowy', 'stones', 'lesson', 'anytime', 'grey', 'surf', 'generous',
'boot', 'shield', 'dress', 'boxes', 'aroma', 's', 'rain', 'taught', 'yield',
'screaming', 'beneath', 'erupt', 'remembered', 'dogs', 'painting', 'trips',
'fishes', 'sisters', 'marched', 'necklaces', 'pizza', 'feels', 'style',
'spoons', 'catch', 'address', 'astonished', 'never', 'knelt', 'raining', 'up',
'gaa', 'carrots', 'paw', 'continue', 'happier', 'walking', 'and', 'joke',
'gabby', 'chicken', 'birdcage', 'flowers', 'lucky', 'nation', 'sounds', 'drank',
'dave', 'kitties', 'crime', 'winter', 'horn', 'yum', 'returning', 'deal',
'emptiness', 'broom', 'biggest', 'wisely', 'chased', 'fingers', 'creation',
'noisy', 'stage', 'yourself', 'zoomed', 'tapped', 'bubblegum', 'race', 'kayak',
'made', 'dizziness', 'yogurt', 'thanks', 'daughter', 'muddy', 'gently',
'apples', 'thoughtfully', 'event', 'lived', 'pounce', 'safely', 'label',
'knees', 'jealous', 'spill', 'nurses', 'macaroni', 'away', 'photo', 'cone',
'greet', 'babies', 'sick', 'recognize', 'hers', 'pouts', 'villagers', 'waded',
'thirsty', 'sports', 'come', 'grassy', 'brains', 'yet', 'winked', 'juicy',
'wing', 'sparkled', 'dessert', 'heal', 'easily', 'sad', 'grateful', 'dirty',
'cargo', 'elderly', 'bills', 'youâ', 'strawberries', 'fall', 'rock', 'shopping',
'oops', 'whatâ', 'robot', 'laughter', 'keyhole', 'lose', 'button', 'fault',
'luckily', 'luckiest', 'tonight', 'resting', 'impressed', 'mint', 'observed',
'humans', 'offers', 'guarding', 'giver', 'melody', 'sleeping', 'web', 'opinion',
'pillow', 'first', 'necklace', 'poppy', 'hot', 'sliding', 'herself', 'trucks',
'pretending', 'pupil', 'car', 'like', 'change', 'shade', 'wrong', 'sell',
'skipped', 'dandelion', 'aunts', 'enough', 'clouds', 'cows', 'rewarded',
'treatments', 'parked', 'table', 'grabbed', 'crayon', 'strings', 'hey',
'bracelets', 'machines', 'bush', 'picture', 'insisted', 'buzzing', 'slow',
'wear', 'photos', 'unwell', 'loudly', 'pail', 'cure', 'maze', 'tv', 'smiley',
'shouts', 'whose', 'want', 'smart', 'shadows', 'further', 'opponent', 'hidden',
'princes', 'permit', 'candle', 'signing', 'tugged', 'suitcase', 'travel',
'octopus', 'heâ', 'sunset', 'beret', 'form', 'cooked', 'guess', 'began', 'rope',
'sight', 'talk', 'wheels', 'warmth', 'buried', 'not', 'shows', 'doh',
'notebook', 'travels', 'chimney', 'marker', 'spice', 'hiss', 'clear',
'directions', 'hide', 'swaying', 'sharp', 'by', 'coldness', 'number',
'grandkids', 'aitâ', 'where', 'there', 'kids', 'tripped', 'polly', 'responded',
'flick', 'leopard', 'heart', 'van', 'true', 'arrived', 'bone', 'novel',
'realized', 'ended', 'stupid', 'husband', 'carpet', 'pocket', 'eye', 'silent',
'sausages', 'large', 'dear', 'wiped', 'mad', 'countryside', 'teddy', 'watered',
'rabbits', 'straw', 'singers', 'quarrel', 'something', 'forgot', 'flammable',

'manners', 'owning', 'different', 'welcome', 'pictures', 'foolish', 'comet',
'treat', 'yelling', 'shivering', 'medicine', 'dolly', 'forth', 'replace',
'clocks', 'receiver', 'creek', 'stepped', 'sammy', 'decided', 'thanking',
'wedding', 'raking', 'lit', 'lily', 'rested', 'ma', 'glowing', 'picnic',
'overwhelmed', 'nodded', 'bella', 'shirts', 'often', 'pinch', 'expensive',
'clowns', 'æwill', 'watched', 'ascend', 'radio', 'getting', 'confident',
'thankfulness', 'uniform', 'dreamt', 'jewellery', 'bite', 'exhausted',
'landscape', 'soaking', 'fantastic', 'day', 'easy', 'explore', 'presents',
'ten', 'satisfy', 'squeezed', 'customers', 'joyfully', 'furniture',
'quicknessâ', 'expressions', 'seeing', 'bushes', 'pulls', 'owls', 'palm',
'shelf', 'environment', 'engines', 'curl', 'racing', 'care', 'ringing', 'unzip',
'fake', 'bark', 'badge', 'barks', 'impatience', 'fold', 'delicate', 'belt',
'unknown', 'miserable', 'class', 'mane', 'swing', 'leaping', 'rainwater',
'clicks', 'future', 'long', 'floppy', 'unfair', 'remained', 'marriage',
'positive', 'wicked', 'pump', 'mill', 'worn', 'beaver', 'plate', 'adventure',
'trapped', 'unfold', 'happily', 'juggling', 'thick', 'unhappy', 'lower',
'memory', 'knots', 'joey', 'dunked', 'gorilla', 'lifted', 'walks', 'persistent',
'shovel', 'yellow', 'canâ', 'gloves', 'soul', 'exclaimed', 'henry', 'kitten',
'all', 'yes', 'curious', 'shirt', 'sandcastle', 'claps', 'underground',
'billboard', 'pout', 'butterfly', 'beak', 'perked', 'such', 'cry', 'visit',
'journal', 'upright', 'spanish', 'gross', 'impatient', 'moon', 'paid', 'ike',
'rubbish', 'red', 'don', 'present', 'snuck', 'magnet', 'layla', 'scarecrow',
'grades', 'explain', 'coughs', 'nesting', 'skin', 'frilly', 'championship',
'fit', 'swords', 'æyour', 'hill', 'dizzy', 'celery', 'peaceful', 'welcomeâ',
'massage', 'yeah', 'energy', 'mister', 'cage', 'asked', 'braver', 'papa',
'drain', 'keeping', 'apology', 'swirled', 'pig', 'crabs', 'savouring', 'gave',
'simply', 'ants', 'include', 'see', 'æwho', 'choo', 'proud', 'raindrops',
'washed', 'imagination', 'faces', 'delighted', 'vow', 'puppies', 'zoom',
'lisa', 'rockets', 'speed', 'party', 'helpful', 'stems', 'three', 'belongs',
'smashed', 'lungs', 'bee', 'finish', 'snowman', 'stir', 'pet', 'so', 'clicked',
'last', 'smiling', 'tasty', 'ænow', 'mixes', 'including', 'king', 'coat',
'backed', 'fingernails', 'olive', 'spices', 'soon', 'bucket', 'shivered',
'tripping', 'sprayed', 'fell', 'peter', 'dug', 'shut', 'rail', 'begged', 'thud',
'started', 'eventually'}

Vocab size: 4548

Token ID of 'park': 300

Token ID of 'wizard': 3162

Token ID of 'cat': 1485

Token ID of 'dog': 621

4.14 Question 3k: Creating input and target sequences for next-token prediction

We need to create targets to train our transformer and measure how well it performs at next-token prediction. You might have noticed that our dataset earlier didn't come with any labels... but that's okay because we already have everything we need! In our case, we want to train our transformer to predict the next token. So if we pass in everything up until the i -th token of our input to the transformer, we want to see how close our transformer gets to predicting that the next token should

be the $i + 1$ -th token of the story!

The key idea is that for training a transformer on next-token prediction, the inputs and the targets are the same sequence but shifted over by 1 position.

Example: * Let's say we have the sentence: "Dinosaur nuggets taste great with ketchup." * If we pass in the token "with" to the transformer, the *target* of its output is the actual next word in the sentence: "ketchup" * So each training and target pair looks like the same sequence shifted by one.

Task: Implement `create_sequences(stories, word_to_id, max_seq_length)`. 1. Split stories into paragraphs (e.g., by double newlines) and convert them into lowercase. 2. Get a list of words from each paragraph with `extract_full_words`. 3. Add a `<start>` token and `<end>` token to the beginning and end of each paragraph. 3. For each word in the paragraph (including the `<start>` and `<end>` tokens), map them to their token ID using `word_to_id` (skip words not in the vocab) 4. Skip any paragraphs with fewer words than the `max_seq_length`. 1. Note: We could have also chosen to pad shorter sequences, but for this homework we'll just skip over paragraphs that are too short 5. For all sequences of length `max_seq_length` in this paragraph, append it to the `input` array and append the sequence that comes 1 index position after to the `target` array 6. Convert all the inputs/targets to `torch.Tensors` with `dtype=torch.long`

Hints: * Ensure returned tensors are `dtype torch.long` and have shape `(num_sequences, max_seq_length)`. * You can reference some of the code we used to create our vocabulary dictionary to see how to split the stories into paragraphs and extract full words.

Expected Output: * If the first 5 tokens in the input data look like this: `[189, 42, 23, 10, 5]`, then the first 5 tokens in the corresponding target should look like this: `[42, 23, 10, 5, 2025]`

```
[93]: def create_sequences(stories, word_to_id, max_seq_length=20,
    ↪ pad_token=PAD_TOKEN, start_token=START_TOKEN, end_token=END_TOKEN):
    """
    Create input/target sequences from paragraphs

    Args:
        stories (list): List of stories to create sequences from
        word_to_id (dict): Dictionary mapping words to their token IDs
        max_seq_length (int): Maximum sequence length to create
        pad_token (str): Token for the pad token
        start_token (str): Token for the start of a sequence
        end_token (str): Token for the end of a sequence

    Returns:
        inputs (tensor): Tensor of input sequences
        targets (tensor): Tensor of target sequences
    """
    inputs = []
    targets = []

    # List of paragraphs
    paragraphs = []
```

```

    # TODO: Split the stories into paragraphs and append them to the
    ↪ `paragraphs` list
    for story in stories:
        # paragraphs.extend([paragraph.lower() for paragraph in story.
        ↪ split("\n\n")])
        paragraphs.extend(story.split("\n\n"))
        # TODO: Iterate over each paragraph, extract full words, add `` and
        ↪ `` tokens, convert to IDs, and build sequences
        for paragraph in paragraphs:
            words = extract_full_words(paragraph)
            words = [start_token] + words + [end_token]
            # Make sure to skip paragraphs that are too short to create any sequences
            if len(words) < max_seq_length:
                pass
            else:
                ids = []
                for w in words:
                    if w in word_to_id:
                        ids.append(word_to_id[w])
                for i in range(len(ids) - max_seq_length):
                    inp = ids[i : i + max_seq_length]
                    tgt = ids[i + 1 : i + 1 + max_seq_length]
                    inputs.append(inp)
                    targets.append(tgt)

        # TODO: Convert the inputs and targets to tensors of type `torch.long`
        inputs = torch.tensor(inputs, dtype=torch.long)
        targets = torch.tensor(targets, dtype=torch.long)
        return inputs, targets

# Generate sequences
max_seq_length = 20
inputs, targets = create_sequences(tiny_stories_text, word_to_id,
    ↪ max_seq_length)

print(f"Total sequences created: {len(inputs)}")
print(f"Each input sequence length: {len(inputs[0])}")
print(f"Each target sequence length: {len(targets[0])}")

```

```

Total sequences created: 88269
Each input sequence length: 20
Each target sequence length: 20

```

4.15 Question 3!: Training and validation DataLoaders

Task: Split sequences and build loaders. 1. Split the inputs and targets into train/val sets with an 80/20 split. 2. Wrap the training and validation sets as `TensorDatasets`. 3. Create training

and validation DataLoaders with `batch_size=32`.

Hints: * `TensorDataset` is a `Dataset` wrapper from PyTorch for easily creating `Datasets` from `Tensors`. It returns tuples `(x[i], y[i])`, and can be passed to a `DataLoader` just like the custom dataset we implemented earlier for our Mini ImageNet data.

Expected behavior: * About 80% of sequences go to training, 20% to validation. * A batch from the dataloaders should yield two tensors (`batch_inputs`, `batch_targets`) with shape `(32, seq_len)`. You should observe that `batch_targets` is the 1-token-left shift of `batch_inputs`.

```
[95]: from sklearn.model_selection import train_test_split
      # TODO: Split into training and validation datasets (80/20 split)
      train_inputs, val_inputs, train_targets, val_targets = \
          ↪ train_test_split(inputs, targets, test_size=0.2, random_state=SEED)

      train_inputs = train_inputs[0:2000]
      train_targets = train_targets[0:2000]
      val_inputs = val_inputs[0:200]
      val_targets = val_targets[0:200]

      print(f"Number of training sequences: {len(train_inputs)}")
      print(f"Number of validation sequences: {len(val_inputs)}")

      # TODO: Create TensorDatasets
      train_dataset = TensorDataset(train_inputs, train_targets)
      val_dataset = TensorDataset(val_inputs, val_targets)

      # TODO: Create DataLoaders
      batch_size = 32
      train_loader = DataLoader(train_dataset, batch_size=32)
      val_loader = DataLoader(val_dataset, batch_size=32)

      print(f"Training batches: {len(train_loader)}")
      print(f"Validation batches: {len(val_loader)}")

      batch = next(iter(train_loader)) # Get the first batch from the training ↪
          ↪ DataLoader

      train_input, train_target = batch # Unpack the batch into input and target ↪
          ↪ tensors
      print(f"First 10 tokens of training input in first batch: {train_input[0][:
          ↪ 10]}")
      print(f"First 10 tokens of training target in first batch: {train_target[0][:
          ↪ 10]}")
```

Number of training sequences: 2000
Number of validation sequences: 200
Training batches: 63

Validation batches: 7

First 10 tokens of training input in first batch: tensor([3372, 471, 4270, 3071, 77, 918, 3508, 875, 3407, 1979])

First 10 tokens of training target in first batch: tensor([471, 4270, 3071, 77, 918, 3508, 875, 3407, 1979, 1373])

4.16 Question 3m: Let the transforming begin!

Using the transformer training loop provided below, train your transformer.

Task: 1. Instantiate a Transformer model. 2. Use AdamW as the optimizer. 1. [PyTorch documentation for AdamW](#) 3. Use CrossEntropyLoss as our criterion. 1. [PyTorch documentation for CrossEntropyLoss](#) 4. Try training your Transformer for different numbers of epochs! Based on the training and validation loss curves we plot in the next question as well as the sample short stories generated in question 3o, pick a value for `num_epochs` that results in a model which generates the most understandable and coherent short stories. 1. We recommend trying some values between 1 - 20. 2. You can read more about how early stopping can actually result in “better” models: [Regularization by Early Stopping](#).

Hints: * Don't forget to pass the parameters of your transformer model into the optimizer. * Make sure that you turn on a GPU runtime before training!

```
[96]: # Transformer training loop
def train_transformer(
    model,
    device,
    optimizer,
    criterion,
    num_epochs,
    train_dataloader,
    val_dataloader,
    vocab,
):
    # === SETUP ===
    # Move model to device
    model.to(device)

    # Lists to store metrics across epochs
    train_losses = []
    val_losses = []

    # === EPOCH LOOP ===
    for epoch in tqdm(range(num_epochs)):
        # === TRAINING PHASE ===
        model.train() # Set model to training mode

        # Initialize metrics for this epoch
        total_train_loss = 0.0
```

```

    # === INNER LOOP (iterate over training batches) ===
    for batch_inputs, batch_targets in tqdm(train_dataloader,
↪desc="Training"):
        # Move inputs and targets to device
        batch_inputs = batch_inputs.to(device)
        batch_targets = batch_targets.to(device)

        # Reset the gradients
        optimizer.zero_grad()

        # Forward pass
        logits = model(batch_inputs) # [batch_size, seq_len, vocab_size]

        # Reshape for loss calculation
        loss = criterion(logits.reshape(-1, len(vocab)), batch_targets.
↪reshape(-1))
        # Compute loss
        loss.backward()

        # Optional: Add gradient clipping to prevent exploding gradients
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

        # Update the model parameters
        optimizer.step()

        # Accumulate the loss
        total_train_loss += loss.item()

# === END OF INNER LOOP ===

# Compute the average loss for this epoch
avg_train_loss = total_train_loss / len(train_dataloader)
train_losses.append(avg_train_loss)

# === END OF TRAINING PHASE ===

# === VALIDATION PHASE ===
model.eval() # Set model to evaluation mode
total_val_loss = 0.0

with torch.no_grad():
    # === INNER LOOP (iterate over validation batches) ===
    for batch_inputs, batch_targets in tqdm(val_dataloader,
↪desc="Validation"):
        # Move inputs and targets to device
        batch_inputs = batch_inputs.to(device)
        batch_targets = batch_targets.to(device)

```

```

        # Forward pass
        logits = model(batch_inputs)

        # Compute loss
        loss = criterion(
            logits.reshape(-1, len(vocab)), batch_targets.reshape(-1)
        )

        # Accumulate the loss
        total_val_loss += loss.item()
    # === END OF INNER LOOP ===

    # Compute the average loss for this epoch
    avg_val_loss = total_val_loss / len(val_dataloader)
    val_losses.append(avg_val_loss)

    # === END OF VALIDATION PHASE ===

    # Print epoch results
    print(f"\nEpoch {epoch + 1} Results:")
    print(f"    Training Loss: {avg_train_loss:.4f}")
    print(f"    Validation Loss: {avg_val_loss:.4f}")

    # === END OF EPOCH LOOP ===

    return train_losses, val_losses

```

```

[97]: # TODO: Instantiate a Transformer model
transformer = Transformer(vocab_size=len(vocab))

# TODO: Set the optimizer
optimizer = AdamW(transformer.parameters(), lr=0.0003)

# TODO: Set the loss criterion
criterion = nn.CrossEntropyLoss()

# TODO: Set the number of epochs
num_epochs= 10

# TODO: Train the model
transformer_train_losses, transformer_val_losses = train_transformer(
    transformer, device, optimizer, criterion, num_epochs, train_loader,
    ↪ val_loader, vocab
)

```

```

0%|          | 0/10 [00:00<?, ?it/s]

```

Training:	0%	0/63 [00:00<?, ?it/s]
Training:	2%	1/63 [00:00<00:17, 3.56it/s]
Training:	3%	2/63 [00:00<00:16, 3.77it/s]
Training:	5%	3/63 [00:00<00:15, 3.86it/s]
Training:	6%	4/63 [00:01<00:15, 3.88it/s]
Training:	8%	5/63 [00:01<00:14, 3.94it/s]
Training:	10%	6/63 [00:01<00:14, 3.94it/s]
Training:	11%	7/63 [00:01<00:14, 3.96it/s]
Training:	13%	8/63 [00:02<00:13, 3.96it/s]
Training:	14%	9/63 [00:02<00:13, 3.97it/s]
Training:	16%	10/63 [00:02<00:13, 3.96it/s]
Training:	17%	11/63 [00:02<00:13, 3.98it/s]
Training:	19%	12/63 [00:03<00:12, 3.96it/s]
Training:	21%	13/63 [00:03<00:12, 4.00it/s]
Training:	22%	14/63 [00:03<00:12, 3.99it/s]
Training:	24%	15/63 [00:03<00:11, 4.01it/s]
Training:	25%	16/63 [00:04<00:11, 3.97it/s]
Training:	27%	17/63 [00:04<00:11, 3.98it/s]
Training:	29%	18/63 [00:04<00:11, 3.96it/s]
Training:	30%	19/63 [00:04<00:11, 3.98it/s]
Training:	32%	20/63 [00:05<00:10, 3.97it/s]
Training:	33%	21/63 [00:05<00:10, 4.00it/s]
Training:	35%	22/63 [00:05<00:10, 3.99it/s]
Training:	37%	23/63 [00:05<00:10, 3.99it/s]
Training:	38%	24/63 [00:06<00:09, 3.99it/s]
Training:	40%	25/63 [00:06<00:09, 3.98it/s]
Training:	41%	26/63 [00:06<00:09, 3.96it/s]
Training:	43%	27/63 [00:06<00:09, 3.94it/s]
Training:	44%	28/63 [00:07<00:08, 3.98it/s]
Training:	46%	29/63 [00:07<00:08, 3.98it/s]
Training:	48%	30/63 [00:07<00:08, 3.97it/s]
Training:	49%	31/63 [00:07<00:08, 3.89it/s]
Training:	51%	32/63 [00:08<00:07, 3.90it/s]
Training:	52%	33/63 [00:08<00:07, 3.87it/s]
Training:	54%	34/63 [00:08<00:07, 3.86it/s]
Training:	56%	35/63 [00:08<00:07, 3.85it/s]
Training:	57%	36/63 [00:09<00:07, 3.80it/s]
Training:	59%	37/63 [00:09<00:06, 3.73it/s]
Training:	60%	38/63 [00:09<00:06, 3.76it/s]
Training:	62%	39/63 [00:09<00:06, 3.78it/s]
Training:	63%	40/63 [00:10<00:06, 3.79it/s]
Training:	65%	41/63 [00:10<00:05, 3.83it/s]
Training:	67%	42/63 [00:10<00:05, 3.87it/s]
Training:	68%	43/63 [00:10<00:05, 3.90it/s]
Training:	70%	44/63 [00:11<00:04, 3.84it/s]
Training:	71%	45/63 [00:11<00:04, 3.84it/s]
Training:	73%	46/63 [00:11<00:04, 3.87it/s]
Training:	75%	47/63 [00:12<00:04, 3.91it/s]

```

Training: 76%|      | 48/63 [00:12<00:03, 3.89it/s]
Training: 78%|      | 49/63 [00:12<00:03, 3.91it/s]
Training: 79%|      | 50/63 [00:12<00:03, 3.93it/s]
Training: 81%|      | 51/63 [00:13<00:03, 3.95it/s]
Training: 83%|      | 52/63 [00:13<00:02, 3.93it/s]
Training: 84%|      | 53/63 [00:13<00:02, 3.96it/s]
Training: 86%|      | 54/63 [00:13<00:02, 3.96it/s]
Training: 87%|      | 55/63 [00:14<00:02, 3.97it/s]
Training: 89%|      | 56/63 [00:14<00:01, 3.94it/s]
Training: 90%|      | 57/63 [00:14<00:01, 3.92it/s]
Training: 92%|      | 58/63 [00:14<00:01, 3.94it/s]
Training: 94%|      | 59/63 [00:15<00:01, 3.97it/s]
Training: 95%|      | 60/63 [00:15<00:00, 3.99it/s]
Training: 97%|      | 61/63 [00:15<00:00, 4.01it/s]
Training: 98%|      | 62/63 [00:15<00:00, 4.01it/s]
Training: 100%|     | 63/63 [00:16<00:00, 3.93it/s]

Validation: 0%|      | 0/7 [00:00<?, ?it/s]
Validation: 29%|     | 2/7 [00:00<00:00, 16.46it/s]
Validation: 57%|     | 4/7 [00:00<00:00, 16.40it/s]
Validation: 100%|    | 7/7 [00:00<00:00, 16.18it/s]
10%|      | 1/10 [00:16<02:28, 16.48s/it]

```

Epoch 1 Results:

```

Training Loss: 6.2395
Validation Loss: 6.0146

```

```

Training: 0%|      | 0/63 [00:00<?, ?it/s]
Training: 2%|      | 1/63 [00:00<00:15, 4.02it/s]
Training: 3%|      | 2/63 [00:00<00:15, 3.96it/s]
Training: 5%|      | 3/63 [00:00<00:15, 3.90it/s]
Training: 6%|      | 4/63 [00:01<00:14, 3.95it/s]
Training: 8%|      | 5/63 [00:01<00:14, 4.00it/s]
Training: 10%|     | 6/63 [00:01<00:14, 4.01it/s]
Training: 11%|     | 7/63 [00:01<00:14, 3.99it/s]
Training: 13%|     | 8/63 [00:02<00:13, 3.96it/s]
Training: 14%|     | 9/63 [00:02<00:13, 3.96it/s]
Training: 16%|     | 10/63 [00:02<00:13, 3.97it/s]
Training: 17%|     | 11/63 [00:02<00:13, 3.95it/s]
Training: 19%|     | 12/63 [00:03<00:12, 3.95it/s]
Training: 21%|     | 13/63 [00:03<00:12, 3.96it/s]
Training: 22%|     | 14/63 [00:03<00:12, 3.87it/s]
Training: 24%|     | 15/63 [00:03<00:12, 3.83it/s]
Training: 25%|     | 16/63 [00:04<00:12, 3.84it/s]
Training: 27%|     | 17/63 [00:04<00:11, 3.84it/s]
Training: 29%|     | 18/63 [00:04<00:11, 3.89it/s]
Training: 30%|     | 19/63 [00:04<00:11, 3.89it/s]

```


Training:	32%	20/63 [00:05<00:11, 3.66it/s]
Training:	33%	21/63 [00:05<00:11, 3.57it/s]
Training:	35%	22/63 [00:05<00:11, 3.60it/s]
Training:	37%	23/63 [00:05<00:10, 3.65it/s]
Training:	38%	24/63 [00:06<00:10, 3.73it/s]
Training:	40%	25/63 [00:06<00:09, 3.81it/s]
Training:	41%	26/63 [00:06<00:09, 3.85it/s]
Training:	43%	27/63 [00:06<00:09, 3.88it/s]
Training:	44%	28/63 [00:07<00:08, 3.92it/s]
Training:	46%	29/63 [00:07<00:08, 3.96it/s]
Training:	48%	30/63 [00:07<00:08, 3.98it/s]
Training:	49%	31/63 [00:07<00:08, 3.99it/s]
Training:	51%	32/63 [00:08<00:07, 4.00it/s]
Training:	52%	33/63 [00:08<00:07, 4.02it/s]
Training:	54%	34/63 [00:08<00:07, 4.02it/s]
Training:	56%	35/63 [00:08<00:07, 3.98it/s]
Training:	57%	36/63 [00:09<00:06, 3.97it/s]
Training:	59%	37/63 [00:09<00:06, 3.96it/s]
Training:	60%	38/63 [00:09<00:06, 3.96it/s]
Training:	62%	39/63 [00:10<00:06, 3.89it/s]
Training:	63%	40/63 [00:10<00:05, 3.85it/s]
Training:	65%	41/63 [00:10<00:05, 3.89it/s]
Training:	67%	42/63 [00:10<00:05, 3.91it/s]
Training:	68%	43/63 [00:11<00:05, 3.91it/s]
Training:	70%	44/63 [00:11<00:04, 3.92it/s]
Training:	71%	45/63 [00:11<00:04, 3.94it/s]
Training:	73%	46/63 [00:11<00:04, 3.96it/s]
Training:	75%	47/63 [00:12<00:04, 3.94it/s]
Training:	76%	48/63 [00:12<00:03, 3.94it/s]
Training:	78%	49/63 [00:12<00:03, 3.95it/s]
Training:	79%	50/63 [00:12<00:03, 3.90it/s]
Training:	81%	51/63 [00:13<00:03, 3.88it/s]
Training:	83%	52/63 [00:13<00:02, 3.91it/s]
Training:	84%	53/63 [00:13<00:02, 3.94it/s]
Training:	86%	54/63 [00:13<00:02, 3.96it/s]
Training:	87%	55/63 [00:14<00:02, 3.92it/s]
Training:	89%	56/63 [00:14<00:01, 3.93it/s]
Training:	90%	57/63 [00:14<00:01, 3.94it/s]
Training:	92%	58/63 [00:14<00:01, 3.93it/s]
Training:	94%	59/63 [00:15<00:01, 3.91it/s]
Training:	95%	60/63 [00:15<00:00, 3.92it/s]
Training:	97%	61/63 [00:15<00:00, 3.94it/s]
Training:	98%	62/63 [00:15<00:00, 3.82it/s]
Training:	100%	63/63 [00:16<00:00, 3.90it/s]
Validation:	0%	0/7 [00:00<?, ?it/s]
Validation:	29%	2/7 [00:00<00:00, 14.59it/s]
Validation:	57%	4/7 [00:00<00:00, 15.27it/s]

Validation: 100%| | 7/7 [00:00<00:00, 15.58it/s]
20%| | 2/10 [00:33<02:12, 16.57s/it]

Epoch 2 Results:

Training Loss: 5.2682

Validation Loss: 5.0580

Training: 0%| | 0/63 [00:00<?, ?it/s]
Training: 2%| | 1/63 [00:00<00:15, 3.97it/s]
Training: 3%| | 2/63 [00:00<00:15, 3.88it/s]
Training: 5%| | 3/63 [00:00<00:15, 3.83it/s]
Training: 6%| | 4/63 [00:01<00:15, 3.83it/s]
Training: 8%| | 5/63 [00:01<00:15, 3.79it/s]
Training: 10%| | 6/63 [00:01<00:14, 3.82it/s]
Training: 11%| | 7/63 [00:01<00:14, 3.86it/s]
Training: 13%| | 8/63 [00:02<00:14, 3.88it/s]
Training: 14%| | 9/63 [00:02<00:13, 3.91it/s]
Training: 16%| | 10/63 [00:02<00:13, 3.91it/s]
Training: 17%| | 11/63 [00:02<00:13, 3.93it/s]
Training: 19%| | 12/63 [00:03<00:12, 3.92it/s]
Training: 21%| | 13/63 [00:03<00:12, 3.94it/s]
Training: 22%| | 14/63 [00:03<00:12, 3.92it/s]
Training: 24%| | 15/63 [00:03<00:12, 3.93it/s]
Training: 25%| | 16/63 [00:04<00:11, 3.94it/s]
Training: 27%| | 17/63 [00:04<00:11, 3.95it/s]
Training: 29%| | 18/63 [00:04<00:11, 3.87it/s]
Training: 30%| | 19/63 [00:04<00:11, 3.88it/s]
Training: 32%| | 20/63 [00:05<00:11, 3.89it/s]
Training: 33%| | 21/63 [00:05<00:10, 3.89it/s]
Training: 35%| | 22/63 [00:05<00:10, 3.89it/s]
Training: 37%| | 23/63 [00:05<00:10, 3.90it/s]
Training: 38%| | 24/63 [00:06<00:09, 3.93it/s]
Training: 40%| | 25/63 [00:06<00:09, 3.92it/s]
Training: 41%| | 26/63 [00:06<00:09, 3.91it/s]
Training: 43%| | 27/63 [00:06<00:09, 3.89it/s]
Training: 44%| | 28/63 [00:07<00:09, 3.87it/s]
Training: 46%| | 29/63 [00:07<00:08, 3.86it/s]
Training: 48%| | 30/63 [00:07<00:08, 3.88it/s]
Training: 49%| | 31/63 [00:07<00:08, 3.93it/s]
Training: 51%| | 32/63 [00:08<00:07, 3.96it/s]
Training: 52%| | 33/63 [00:08<00:07, 3.98it/s]
Training: 54%| | 34/63 [00:08<00:07, 3.98it/s]
Training: 56%| | 35/63 [00:08<00:07, 3.99it/s]
Training: 57%| | 36/63 [00:09<00:06, 3.96it/s]
Training: 59%| | 37/63 [00:09<00:06, 3.95it/s]
Training: 60%| | 38/63 [00:09<00:06, 3.96it/s]
Training: 62%| | 39/63 [00:09<00:06, 3.96it/s]

```

Training: 63%|          | 40/63 [00:10<00:05, 3.94it/s]
Training: 65%|          | 41/63 [00:10<00:05, 3.96it/s]
Training: 67%|          | 42/63 [00:10<00:05, 3.97it/s]
Training: 68%|          | 43/63 [00:10<00:05, 3.94it/s]
Training: 70%|          | 44/63 [00:11<00:04, 3.91it/s]
Training: 71%|          | 45/63 [00:11<00:04, 3.77it/s]
Training: 73%|          | 46/63 [00:11<00:04, 3.75it/s]
Training: 75%|          | 47/63 [00:12<00:04, 3.74it/s]
Training: 76%|          | 48/63 [00:12<00:03, 3.76it/s]
Training: 78%|          | 49/63 [00:12<00:03, 3.78it/s]
Training: 79%|          | 50/63 [00:12<00:03, 3.80it/s]
Training: 81%|          | 51/63 [00:13<00:03, 3.71it/s]
Training: 83%|          | 52/63 [00:13<00:02, 3.73it/s]
Training: 84%|          | 53/63 [00:13<00:02, 3.74it/s]
Training: 86%|          | 54/63 [00:13<00:02, 3.82it/s]
Training: 87%|          | 55/63 [00:14<00:02, 3.86it/s]
Training: 89%|          | 56/63 [00:14<00:01, 3.90it/s]
Training: 90%|          | 57/63 [00:14<00:01, 3.94it/s]
Training: 92%|          | 58/63 [00:14<00:01, 3.96it/s]
Training: 94%|          | 59/63 [00:15<00:01, 3.96it/s]
Training: 95%|          | 60/63 [00:15<00:00, 3.96it/s]
Training: 97%|          | 61/63 [00:15<00:00, 3.98it/s]
Training: 98%|          | 62/63 [00:15<00:00, 3.98it/s]
Training: 100%|         | 63/63 [00:16<00:00, 3.90it/s]

```

```

Validation: 0%|          | 0/7 [00:00<?, ?it/s]
Validation: 29%|         | 2/7 [00:00<00:00, 16.33it/s]
Validation: 57%|         | 4/7 [00:00<00:00, 16.27it/s]
Validation: 100%|        | 7/7 [00:00<00:00, 16.25it/s]
30%|          | 3/10 [00:49<01:56, 16.58s/it]

```

Epoch 3 Results:

```

Training Loss: 4.3176
Validation Loss: 4.5856

```

```

Training: 0%|          | 0/63 [00:00<?, ?it/s]
Training: 2%|          | 1/63 [00:00<00:15, 4.07it/s]
Training: 3%|          | 2/63 [00:00<00:15, 4.02it/s]
Training: 5%|          | 3/63 [00:00<00:14, 4.04it/s]
Training: 6%|          | 4/63 [00:00<00:14, 4.01it/s]
Training: 8%|          | 5/63 [00:01<00:14, 4.01it/s]
Training: 10%|         | 6/63 [00:01<00:14, 4.00it/s]
Training: 11%|         | 7/63 [00:01<00:13, 4.01it/s]
Training: 13%|         | 8/63 [00:01<00:13, 4.00it/s]
Training: 14%|         | 9/63 [00:02<00:13, 4.00it/s]
Training: 16%|         | 10/63 [00:02<00:13, 3.94it/s]
Training: 17%|         | 11/63 [00:02<00:13, 3.95it/s]

```

Training:	19%	12/63 [00:03<00:12, 3.96it/s]
Training:	21%	13/63 [00:03<00:12, 3.99it/s]
Training:	22%	14/63 [00:03<00:12, 3.95it/s]
Training:	24%	15/63 [00:03<00:12, 3.97it/s]
Training:	25%	16/63 [00:04<00:11, 3.98it/s]
Training:	27%	17/63 [00:04<00:11, 3.98it/s]
Training:	29%	18/63 [00:04<00:11, 3.98it/s]
Training:	30%	19/63 [00:04<00:11, 3.99it/s]
Training:	32%	20/63 [00:05<00:10, 3.95it/s]
Training:	33%	21/63 [00:05<00:10, 3.96it/s]
Training:	35%	22/63 [00:05<00:10, 3.95it/s]
Training:	37%	23/63 [00:05<00:10, 3.93it/s]
Training:	38%	24/63 [00:06<00:09, 3.96it/s]
Training:	40%	25/63 [00:06<00:09, 3.97it/s]
Training:	41%	26/63 [00:06<00:09, 3.95it/s]
Training:	43%	27/63 [00:06<00:09, 3.88it/s]
Training:	44%	28/63 [00:07<00:09, 3.80it/s]
Training:	46%	29/63 [00:07<00:08, 3.81it/s]
Training:	48%	30/63 [00:07<00:08, 3.84it/s]
Training:	49%	31/63 [00:07<00:08, 3.83it/s]
Training:	51%	32/63 [00:08<00:08, 3.84it/s]
Training:	52%	33/63 [00:08<00:07, 3.87it/s]
Training:	54%	34/63 [00:08<00:07, 3.81it/s]
Training:	56%	35/63 [00:08<00:07, 3.78it/s]
Training:	57%	36/63 [00:09<00:07, 3.76it/s]
Training:	59%	37/63 [00:09<00:06, 3.84it/s]
Training:	60%	38/63 [00:09<00:06, 3.85it/s]
Training:	62%	39/63 [00:09<00:06, 3.90it/s]
Training:	63%	40/63 [00:10<00:05, 3.92it/s]
Training:	65%	41/63 [00:10<00:05, 3.96it/s]
Training:	67%	42/63 [00:10<00:05, 3.96it/s]
Training:	68%	43/63 [00:10<00:05, 3.97it/s]
Training:	70%	44/63 [00:11<00:04, 4.01it/s]
Training:	71%	45/63 [00:11<00:04, 4.01it/s]
Training:	73%	46/63 [00:11<00:04, 4.00it/s]
Training:	75%	47/63 [00:11<00:04, 4.00it/s]
Training:	76%	48/63 [00:12<00:03, 4.01it/s]
Training:	78%	49/63 [00:12<00:03, 3.97it/s]
Training:	79%	50/63 [00:12<00:03, 3.91it/s]
Training:	81%	51/63 [00:12<00:03, 3.93it/s]
Training:	83%	52/63 [00:13<00:02, 3.96it/s]
Training:	84%	53/63 [00:13<00:02, 3.98it/s]
Training:	86%	54/63 [00:13<00:02, 3.98it/s]
Training:	87%	55/63 [00:13<00:02, 3.99it/s]
Training:	89%	56/63 [00:14<00:01, 3.97it/s]
Training:	90%	57/63 [00:14<00:01, 3.97it/s]
Training:	92%	58/63 [00:14<00:01, 3.96it/s]
Training:	94%	59/63 [00:14<00:01, 3.94it/s]

```

Training: 95%|      | 60/63 [00:15<00:00, 3.96it/s]
Training: 97%|      | 61/63 [00:15<00:00, 3.95it/s]
Training: 98%|      | 62/63 [00:15<00:00, 3.96it/s]
Training: 100%|     | 63/63 [00:15<00:00, 3.94it/s]

Validation: 0%|      | 0/7 [00:00<?, ?it/s]
Validation: 29%|     | 2/7 [00:00<00:00, 16.42it/s]
Validation: 57%|     | 4/7 [00:00<00:00, 16.29it/s]
Validation: 100%|    | 7/7 [00:00<00:00, 16.41it/s]
40%|      | 4/10 [01:06<01:39, 16.52s/it]

```

Epoch 4 Results:

```

Training Loss: 3.6595
Validation Loss: 4.3528

```

```

Training: 0%|      | 0/63 [00:00<?, ?it/s]
Training: 2%|      | 1/63 [00:00<00:15, 4.01it/s]
Training: 3%|      | 2/63 [00:00<00:15, 3.96it/s]
Training: 5%|      | 3/63 [00:00<00:15, 3.95it/s]
Training: 6%|      | 4/63 [00:01<00:14, 3.96it/s]
Training: 8%|      | 5/63 [00:01<00:14, 3.94it/s]
Training: 10%|     | 6/63 [00:01<00:14, 3.95it/s]
Training: 11%|     | 7/63 [00:01<00:14, 3.94it/s]
Training: 13%|     | 8/63 [00:02<00:13, 3.94it/s]
Training: 14%|     | 9/63 [00:02<00:13, 3.91it/s]
Training: 16%|     | 10/63 [00:02<00:13, 3.90it/s]
Training: 17%|     | 11/63 [00:02<00:13, 3.82it/s]
Training: 19%|     | 12/63 [00:03<00:13, 3.81it/s]
Training: 21%|     | 13/63 [00:03<00:13, 3.79it/s]
Training: 22%|     | 14/63 [00:03<00:13, 3.73it/s]
Training: 24%|     | 15/63 [00:03<00:12, 3.76it/s]
Training: 25%|     | 16/63 [00:04<00:12, 3.78it/s]
Training: 27%|     | 17/63 [00:04<00:12, 3.75it/s]
Training: 29%|     | 18/63 [00:04<00:12, 3.74it/s]
Training: 30%|     | 19/63 [00:04<00:11, 3.72it/s]
Training: 32%|     | 20/63 [00:05<00:11, 3.74it/s]
Training: 33%|     | 21/63 [00:05<00:11, 3.76it/s]
Training: 35%|     | 22/63 [00:05<00:10, 3.79it/s]
Training: 37%|     | 23/63 [00:06<00:10, 3.82it/s]
Training: 38%|     | 24/63 [00:06<00:10, 3.85it/s]
Training: 40%|     | 25/63 [00:06<00:09, 3.87it/s]
Training: 41%|     | 26/63 [00:06<00:09, 3.91it/s]
Training: 43%|     | 27/63 [00:07<00:09, 3.93it/s]
Training: 44%|     | 28/63 [00:07<00:08, 3.93it/s]
Training: 46%|     | 29/63 [00:07<00:08, 3.94it/s]
Training: 48%|     | 30/63 [00:07<00:08, 3.93it/s]
Training: 49%|     | 31/63 [00:08<00:08, 3.96it/s]

```

Training: 51%| | 32/63 [00:08<00:07, 3.97it/s]
 Training: 52%| | 33/63 [00:08<00:07, 3.93it/s]
 Training: 54%| | 34/63 [00:08<00:07, 3.98it/s]
 Training: 56%| | 35/63 [00:09<00:06, 4.00it/s]
 Training: 57%| | 36/63 [00:09<00:06, 4.00it/s]
 Training: 59%| | 37/63 [00:09<00:06, 3.98it/s]
 Training: 60%| | 38/63 [00:09<00:06, 4.00it/s]
 Training: 62%| | 39/63 [00:10<00:05, 4.02it/s]
 Training: 63%| | 40/63 [00:10<00:05, 4.01it/s]
 Training: 65%| | 41/63 [00:10<00:05, 3.99it/s]
 Training: 67%| | 42/63 [00:10<00:05, 4.02it/s]
 Training: 68%| | 43/63 [00:11<00:04, 4.02it/s]
 Training: 70%| | 44/63 [00:11<00:04, 4.02it/s]
 Training: 71%| | 45/63 [00:11<00:04, 4.04it/s]
 Training: 73%| | 46/63 [00:11<00:04, 4.06it/s]
 Training: 75%| | 47/63 [00:12<00:03, 4.04it/s]
 Training: 76%| | 48/63 [00:12<00:03, 4.03it/s]
 Training: 78%| | 49/63 [00:12<00:03, 4.01it/s]
 Training: 79%| | 50/63 [00:12<00:03, 4.03it/s]
 Training: 81%| | 51/63 [00:13<00:02, 4.03it/s]
 Training: 83%| | 52/63 [00:13<00:02, 4.02it/s]
 Training: 84%| | 53/63 [00:13<00:02, 4.02it/s]
 Training: 86%| | 54/63 [00:13<00:02, 4.02it/s]
 Training: 87%| | 55/63 [00:14<00:01, 4.02it/s]
 Training: 89%| | 56/63 [00:14<00:01, 3.98it/s]
 Training: 90%| | 57/63 [00:14<00:01, 3.99it/s]
 Training: 92%| | 58/63 [00:14<00:01, 3.90it/s]
 Training: 94%| | 59/63 [00:15<00:01, 3.85it/s]
 Training: 95%| | 60/63 [00:15<00:00, 3.87it/s]
 Training: 97%| | 61/63 [00:15<00:00, 3.89it/s]
 Training: 98%| | 62/63 [00:15<00:00, 3.85it/s]
 Training: 100%| | 63/63 [00:16<00:00, 3.92it/s]

Validation: 0%| | 0/7 [00:00<?, ?it/s]
 Validation: 29%| | 2/7 [00:00<00:00, 16.10it/s]
 Validation: 57%| | 4/7 [00:00<00:00, 16.28it/s]
 Validation: 100%| | 7/7 [00:00<00:00, 15.93it/s]
 50%| | 5/10 [01:22<01:22, 16.52s/it]

Epoch 5 Results:

Training Loss: 2.9535
 Validation Loss: 3.6304

Training: 0%| | 0/63 [00:00<?, ?it/s]
 Training: 2%| | 1/63 [00:00<00:16, 3.81it/s]
 Training: 3%| | 2/63 [00:00<00:17, 3.58it/s]
 Training: 5%| | 3/63 [00:00<00:15, 3.77it/s]

Training:	6%	4/63 [00:01<00:15,	3.88it/s]
Training:	8%	5/63 [00:01<00:14,	3.95it/s]
Training:	10%	6/63 [00:01<00:14,	3.99it/s]
Training:	11%	7/63 [00:01<00:13,	4.02it/s]
Training:	13%	8/63 [00:02<00:13,	4.02it/s]
Training:	14%	9/63 [00:02<00:13,	4.02it/s]
Training:	16%	10/63 [00:02<00:13,	4.04it/s]
Training:	17%	11/63 [00:02<00:12,	4.06it/s]
Training:	19%	12/63 [00:03<00:12,	4.04it/s]
Training:	21%	13/63 [00:03<00:12,	4.05it/s]
Training:	22%	14/63 [00:03<00:12,	4.05it/s]
Training:	24%	15/63 [00:03<00:11,	4.05it/s]
Training:	25%	16/63 [00:04<00:11,	4.06it/s]
Training:	27%	17/63 [00:04<00:11,	4.07it/s]
Training:	29%	18/63 [00:04<00:11,	4.08it/s]
Training:	30%	19/63 [00:04<00:10,	4.09it/s]
Training:	32%	20/63 [00:04<00:10,	4.09it/s]
Training:	33%	21/63 [00:05<00:10,	4.10it/s]
Training:	35%	22/63 [00:05<00:09,	4.10it/s]
Training:	37%	23/63 [00:05<00:09,	4.11it/s]
Training:	38%	24/63 [00:05<00:09,	4.10it/s]
Training:	40%	25/63 [00:06<00:09,	4.10it/s]
Training:	41%	26/63 [00:06<00:09,	4.08it/s]
Training:	43%	27/63 [00:06<00:08,	4.09it/s]
Training:	44%	28/63 [00:06<00:08,	4.09it/s]
Training:	46%	29/63 [00:07<00:08,	4.10it/s]
Training:	48%	30/63 [00:07<00:08,	4.05it/s]
Training:	49%	31/63 [00:07<00:07,	4.05it/s]
Training:	51%	32/63 [00:07<00:07,	4.03it/s]
Training:	52%	33/63 [00:08<00:07,	4.03it/s]
Training:	54%	34/63 [00:08<00:07,	4.02it/s]
Training:	56%	35/63 [00:08<00:06,	4.02it/s]
Training:	57%	36/63 [00:08<00:06,	4.02it/s]
Training:	59%	37/63 [00:09<00:06,	4.02it/s]
Training:	60%	38/63 [00:09<00:06,	4.02it/s]
Training:	62%	39/63 [00:09<00:05,	4.03it/s]
Training:	63%	40/63 [00:09<00:05,	4.02it/s]
Training:	65%	41/63 [00:10<00:05,	4.03it/s]
Training:	67%	42/63 [00:10<00:05,	3.99it/s]
Training:	68%	43/63 [00:10<00:05,	3.91it/s]
Training:	70%	44/63 [00:10<00:04,	3.89it/s]
Training:	71%	45/63 [00:11<00:04,	3.83it/s]
Training:	73%	46/63 [00:11<00:04,	3.84it/s]
Training:	75%	47/63 [00:11<00:04,	3.84it/s]
Training:	76%	48/63 [00:11<00:03,	3.86it/s]
Training:	78%	49/63 [00:12<00:03,	3.84it/s]
Training:	79%	50/63 [00:12<00:03,	3.79it/s]
Training:	81%	51/63 [00:12<00:03,	3.80it/s]

```

Training: 83%|          | 52/63 [00:13<00:02, 3.85it/s]
Training: 84%|          | 53/63 [00:13<00:02, 3.92it/s]
Training: 86%|          | 54/63 [00:13<00:02, 3.91it/s]
Training: 87%|          | 55/63 [00:13<00:02, 3.95it/s]
Training: 89%|          | 56/63 [00:14<00:01, 3.97it/s]
Training: 90%|          | 57/63 [00:14<00:01, 4.01it/s]
Training: 92%|          | 58/63 [00:14<00:01, 4.04it/s]
Training: 94%|          | 59/63 [00:14<00:00, 4.05it/s]
Training: 95%|          | 60/63 [00:15<00:00, 3.99it/s]
Training: 97%|          | 61/63 [00:15<00:00, 3.84it/s]
Training: 98%|          | 62/63 [00:15<00:00, 3.83it/s]
Training: 100%|         | 63/63 [00:15<00:00, 3.98it/s]

Validation: 0%|          | 0/7 [00:00<?, ?it/s]
Validation: 29%|         | 2/7 [00:00<00:00, 16.27it/s]
Validation: 57%|         | 4/7 [00:00<00:00, 16.19it/s]
Validation: 100%|        | 7/7 [00:00<00:00, 16.34it/s]
60%|          | 6/10 [01:38<01:05, 16.44s/it]

```

Epoch 6 Results:

```

Training Loss: 1.8265
Validation Loss: 2.2423

```

```

Training: 0%|          | 0/63 [00:00<?, ?it/s]
Training: 2%|          | 1/63 [00:00<00:15, 4.04it/s]
Training: 3%|          | 2/63 [00:00<00:15, 4.06it/s]
Training: 5%|          | 3/63 [00:00<00:14, 4.05it/s]
Training: 6%|          | 4/63 [00:00<00:14, 4.06it/s]
Training: 8%|          | 5/63 [00:01<00:14, 4.06it/s]
Training: 10%|         | 6/63 [00:01<00:14, 3.98it/s]
Training: 11%|         | 7/63 [00:01<00:14, 3.94it/s]
Training: 13%|         | 8/63 [00:02<00:13, 3.96it/s]
Training: 14%|         | 9/63 [00:02<00:13, 3.97it/s]
Training: 16%|         | 10/63 [00:02<00:13, 3.99it/s]
Training: 17%|         | 11/63 [00:02<00:12, 4.01it/s]
Training: 19%|         | 12/63 [00:03<00:12, 3.98it/s]
Training: 21%|         | 13/63 [00:03<00:12, 3.96it/s]
Training: 22%|         | 14/63 [00:03<00:12, 3.97it/s]
Training: 24%|         | 15/63 [00:03<00:12, 3.97it/s]
Training: 25%|         | 16/63 [00:04<00:11, 3.98it/s]
Training: 27%|         | 17/63 [00:04<00:11, 3.96it/s]
Training: 29%|         | 18/63 [00:04<00:11, 3.93it/s]
Training: 30%|         | 19/63 [00:04<00:11, 3.95it/s]
Training: 32%|         | 20/63 [00:05<00:10, 3.94it/s]
Training: 33%|         | 21/63 [00:05<00:10, 3.96it/s]
Training: 35%|         | 22/63 [00:05<00:10, 3.98it/s]
Training: 37%|         | 23/63 [00:05<00:10, 3.94it/s]

```


Training:	38%	24/63 [00:06<00:09, 3.92it/s]
Training:	40%	25/63 [00:06<00:09, 3.93it/s]
Training:	41%	26/63 [00:06<00:09, 3.87it/s]
Training:	43%	27/63 [00:06<00:09, 3.82it/s]
Training:	44%	28/63 [00:07<00:09, 3.83it/s]
Training:	46%	29/63 [00:07<00:09, 3.77it/s]
Training:	48%	30/63 [00:07<00:08, 3.77it/s]
Training:	49%	31/63 [00:07<00:08, 3.80it/s]
Training:	51%	32/63 [00:08<00:08, 3.83it/s]
Training:	52%	33/63 [00:08<00:07, 3.75it/s]
Training:	54%	34/63 [00:08<00:07, 3.72it/s]
Training:	56%	35/63 [00:08<00:07, 3.72it/s]
Training:	57%	36/63 [00:09<00:07, 3.70it/s]
Training:	59%	37/63 [00:09<00:07, 3.53it/s]
Training:	60%	38/63 [00:09<00:07, 3.45it/s]
Training:	62%	39/63 [00:10<00:06, 3.50it/s]
Training:	63%	40/63 [00:10<00:06, 3.47it/s]
Training:	65%	41/63 [00:10<00:06, 3.60it/s]
Training:	67%	42/63 [00:10<00:05, 3.73it/s]
Training:	68%	43/63 [00:11<00:05, 3.81it/s]
Training:	70%	44/63 [00:11<00:04, 3.86it/s]
Training:	71%	45/63 [00:11<00:04, 3.89it/s]
Training:	73%	46/63 [00:11<00:04, 3.93it/s]
Training:	75%	47/63 [00:12<00:04, 3.96it/s]
Training:	76%	48/63 [00:12<00:03, 3.97it/s]
Training:	78%	49/63 [00:12<00:03, 3.97it/s]
Training:	79%	50/63 [00:12<00:03, 3.97it/s]
Training:	81%	51/63 [00:13<00:03, 3.98it/s]
Training:	83%	52/63 [00:13<00:02, 3.95it/s]
Training:	84%	53/63 [00:13<00:02, 3.94it/s]
Training:	86%	54/63 [00:13<00:02, 3.97it/s]
Training:	87%	55/63 [00:14<00:02, 4.00it/s]
Training:	89%	56/63 [00:14<00:01, 3.95it/s]
Training:	90%	57/63 [00:14<00:01, 3.91it/s]
Training:	92%	58/63 [00:14<00:01, 3.89it/s]
Training:	94%	59/63 [00:15<00:01, 3.88it/s]
Training:	95%	60/63 [00:15<00:00, 3.91it/s]
Training:	97%	61/63 [00:15<00:00, 3.91it/s]
Training:	98%	62/63 [00:15<00:00, 3.95it/s]
Training:	100%	63/63 [00:16<00:00, 3.88it/s]

Validation:	0%	0/7 [00:00<?, ?it/s]
Validation:	29%	2/7 [00:00<00:00, 15.49it/s]
Validation:	57%	4/7 [00:00<00:00, 16.08it/s]
Validation:	100%	7/7 [00:00<00:00, 16.01it/s]
70%	7/10	[01:55<00:49, 16.52s/it]

Epoch 7 Results:

Training Loss: 0.8554

Validation Loss: 1.6732

Training:	0%	0/63 [00:00<?, ?it/s]
Training:	2%	1/63 [00:00<00:14, 4.15it/s]
Training:	3%	2/63 [00:00<00:14, 4.13it/s]
Training:	5%	3/63 [00:00<00:14, 4.12it/s]
Training:	6%	4/63 [00:01<00:15, 3.93it/s]
Training:	8%	5/63 [00:01<00:14, 3.98it/s]
Training:	10%	6/63 [00:01<00:14, 3.90it/s]
Training:	11%	7/63 [00:01<00:14, 3.88it/s]
Training:	13%	8/63 [00:02<00:14, 3.87it/s]
Training:	14%	9/63 [00:02<00:14, 3.72it/s]
Training:	16%	10/63 [00:02<00:14, 3.78it/s]
Training:	17%	11/63 [00:02<00:13, 3.72it/s]
Training:	19%	12/63 [00:03<00:13, 3.76it/s]
Training:	21%	13/63 [00:03<00:13, 3.79it/s]
Training:	22%	14/63 [00:03<00:13, 3.77it/s]
Training:	24%	15/63 [00:03<00:12, 3.71it/s]
Training:	25%	16/63 [00:04<00:12, 3.70it/s]
Training:	27%	17/63 [00:04<00:12, 3.80it/s]
Training:	29%	18/63 [00:04<00:11, 3.87it/s]
Training:	30%	19/63 [00:04<00:11, 3.92it/s]
Training:	32%	20/63 [00:05<00:10, 3.96it/s]
Training:	33%	21/63 [00:05<00:10, 4.00it/s]
Training:	35%	22/63 [00:05<00:10, 4.04it/s]
Training:	37%	23/63 [00:05<00:09, 4.04it/s]
Training:	38%	24/63 [00:06<00:09, 4.04it/s]
Training:	40%	25/63 [00:06<00:09, 4.05it/s]
Training:	41%	26/63 [00:06<00:09, 4.03it/s]
Training:	43%	27/63 [00:06<00:08, 4.03it/s]
Training:	44%	28/63 [00:07<00:08, 4.01it/s]
Training:	46%	29/63 [00:07<00:08, 4.02it/s]
Training:	48%	30/63 [00:07<00:08, 4.03it/s]
Training:	49%	31/63 [00:07<00:07, 4.04it/s]
Training:	51%	32/63 [00:08<00:07, 4.05it/s]
Training:	52%	33/63 [00:08<00:07, 4.05it/s]
Training:	54%	34/63 [00:08<00:07, 4.08it/s]
Training:	56%	35/63 [00:08<00:06, 4.05it/s]
Training:	57%	36/63 [00:09<00:06, 4.03it/s]
Training:	59%	37/63 [00:09<00:06, 4.03it/s]
Training:	60%	38/63 [00:09<00:06, 4.02it/s]
Training:	62%	39/63 [00:09<00:06, 3.98it/s]
Training:	63%	40/63 [00:10<00:05, 3.97it/s]
Training:	65%	41/63 [00:10<00:05, 3.98it/s]
Training:	67%	42/63 [00:10<00:05, 3.99it/s]

```

Training: 68%|          | 43/63 [00:10<00:04, 4.01it/s]
Training: 70%|          | 44/63 [00:11<00:04, 4.02it/s]
Training: 71%|          | 45/63 [00:11<00:04, 4.02it/s]
Training: 73%|          | 46/63 [00:11<00:04, 4.04it/s]
Training: 75%|          | 47/63 [00:11<00:03, 4.07it/s]
Training: 76%|          | 48/63 [00:12<00:03, 4.07it/s]
Training: 78%|          | 49/63 [00:12<00:03, 4.04it/s]
Training: 79%|          | 50/63 [00:12<00:03, 4.05it/s]
Training: 81%|          | 51/63 [00:12<00:03, 4.00it/s]
Training: 83%|          | 52/63 [00:13<00:02, 4.01it/s]
Training: 84%|          | 53/63 [00:13<00:02, 4.03it/s]
Training: 86%|          | 54/63 [00:13<00:02, 4.01it/s]
Training: 87%|          | 55/63 [00:13<00:01, 4.02it/s]
Training: 89%|          | 56/63 [00:14<00:01, 4.01it/s]
Training: 90%|          | 57/63 [00:14<00:01, 3.96it/s]
Training: 92%|          | 58/63 [00:14<00:01, 3.95it/s]
Training: 94%|          | 59/63 [00:14<00:01, 3.96it/s]
Training: 95%|          | 60/63 [00:15<00:00, 3.92it/s]
Training: 97%|          | 61/63 [00:15<00:00, 3.94it/s]
Training: 98%|          | 62/63 [00:15<00:00, 3.95it/s]
Training: 100%|         | 63/63 [00:15<00:00, 3.96it/s]

Validation: 0%|          | 0/7 [00:00<?, ?it/s]
Validation: 29%|         | 2/7 [00:00<00:00, 14.42it/s]
Validation: 57%|         | 4/7 [00:00<00:00, 14.75it/s]
Validation: 100%|        | 7/7 [00:00<00:00, 14.56it/s]
80%|          | 8/10 [02:12<00:32, 16.48s/it]

```

Epoch 8 Results:

```

Training Loss: 0.3779
Validation Loss: 1.3277

```

```

Training: 0%|          | 0/63 [00:00<?, ?it/s]
Training: 2%|          | 1/63 [00:00<00:15, 4.01it/s]
Training: 3%|          | 2/63 [00:00<00:14, 4.08it/s]
Training: 5%|          | 3/63 [00:00<00:14, 4.12it/s]
Training: 6%|          | 4/63 [00:00<00:14, 4.08it/s]
Training: 8%|          | 5/63 [00:01<00:14, 4.07it/s]
Training: 10%|         | 6/63 [00:01<00:13, 4.09it/s]
Training: 11%|         | 7/63 [00:01<00:14, 3.84it/s]
Training: 13%|         | 8/63 [00:02<00:14, 3.88it/s]
Training: 14%|         | 9/63 [00:02<00:13, 3.94it/s]
Training: 16%|         | 10/63 [00:02<00:14, 3.75it/s]
Training: 17%|         | 11/63 [00:02<00:13, 3.82it/s]
Training: 19%|         | 12/63 [00:03<00:13, 3.89it/s]
Training: 21%|         | 13/63 [00:03<00:12, 3.93it/s]
Training: 22%|         | 14/63 [00:03<00:12, 3.94it/s]

```

Training:	24%	15/63 [00:03<00:12, 3.97it/s]
Training:	25%	16/63 [00:04<00:11, 3.97it/s]
Training:	27%	17/63 [00:04<00:11, 3.99it/s]
Training:	29%	18/63 [00:04<00:11, 4.00it/s]
Training:	30%	19/63 [00:04<00:10, 4.01it/s]
Training:	32%	20/63 [00:05<00:10, 4.00it/s]
Training:	33%	21/63 [00:05<00:10, 4.00it/s]
Training:	35%	22/63 [00:05<00:10, 4.00it/s]
Training:	37%	23/63 [00:05<00:09, 4.02it/s]
Training:	38%	24/63 [00:06<00:09, 4.04it/s]
Training:	40%	25/63 [00:06<00:09, 4.03it/s]
Training:	41%	26/63 [00:06<00:09, 4.02it/s]
Training:	43%	27/63 [00:06<00:08, 4.02it/s]
Training:	44%	28/63 [00:07<00:08, 4.03it/s]
Training:	46%	29/63 [00:07<00:08, 4.05it/s]
Training:	48%	30/63 [00:07<00:08, 4.05it/s]
Training:	49%	31/63 [00:07<00:07, 4.07it/s]
Training:	51%	32/63 [00:08<00:07, 4.05it/s]
Training:	52%	33/63 [00:08<00:07, 4.06it/s]
Training:	54%	34/63 [00:08<00:07, 4.00it/s]
Training:	56%	35/63 [00:08<00:06, 4.03it/s]
Training:	57%	36/63 [00:09<00:06, 4.03it/s]
Training:	59%	37/63 [00:09<00:06, 4.01it/s]
Training:	60%	38/63 [00:09<00:06, 3.99it/s]
Training:	62%	39/63 [00:09<00:05, 4.01it/s]
Training:	63%	40/63 [00:10<00:05, 4.02it/s]
Training:	65%	41/63 [00:10<00:05, 3.95it/s]
Training:	67%	42/63 [00:10<00:05, 3.91it/s]
Training:	68%	43/63 [00:10<00:05, 3.90it/s]
Training:	70%	44/63 [00:11<00:04, 3.86it/s]
Training:	71%	45/63 [00:11<00:04, 3.87it/s]
Training:	73%	46/63 [00:11<00:04, 3.89it/s]
Training:	75%	47/63 [00:11<00:04, 3.83it/s]
Training:	76%	48/63 [00:12<00:03, 3.86it/s]
Training:	78%	49/63 [00:12<00:03, 3.87it/s]
Training:	79%	50/63 [00:12<00:03, 3.93it/s]
Training:	81%	51/63 [00:12<00:03, 4.00it/s]
Training:	83%	52/63 [00:13<00:02, 4.05it/s]
Training:	84%	53/63 [00:13<00:02, 4.06it/s]
Training:	86%	54/63 [00:13<00:02, 4.04it/s]
Training:	87%	55/63 [00:13<00:01, 4.06it/s]
Training:	89%	56/63 [00:14<00:01, 4.05it/s]
Training:	90%	57/63 [00:14<00:01, 4.06it/s]
Training:	92%	58/63 [00:14<00:01, 4.03it/s]
Training:	94%	59/63 [00:14<00:00, 4.05it/s]
Training:	95%	60/63 [00:15<00:00, 4.04it/s]
Training:	97%	61/63 [00:15<00:00, 4.02it/s]
Training:	98%	62/63 [00:15<00:00, 4.02it/s]

Training: 100%| | 63/63 [00:15<00:00, 3.99it/s]
 Validation: 0%| | 0/7 [00:00<?, ?it/s]
 Validation: 29%| | 2/7 [00:00<00:00, 16.25it/s]
 Validation: 57%| | 4/7 [00:00<00:00, 16.36it/s]
 Validation: 100%| | 7/7 [00:00<00:00, 16.41it/s]
 90%| | 9/10 [02:28<00:16, 16.40s/it]

Epoch 9 Results:

Training Loss: 0.1711
 Validation Loss: 1.2446

Training: 0%| | 0/63 [00:00<?, ?it/s]
 Training: 2%| | 1/63 [00:00<00:15, 4.04it/s]
 Training: 3%| | 2/63 [00:00<00:15, 4.05it/s]
 Training: 5%| | 3/63 [00:00<00:14, 4.04it/s]
 Training: 6%| | 4/63 [00:00<00:14, 4.06it/s]
 Training: 8%| | 5/63 [00:01<00:14, 4.06it/s]
 Training: 10%| | 6/63 [00:01<00:14, 4.06it/s]
 Training: 11%| | 7/63 [00:01<00:13, 4.05it/s]
 Training: 13%| | 8/63 [00:01<00:13, 4.06it/s]
 Training: 14%| | 9/63 [00:02<00:13, 4.05it/s]
 Training: 16%| | 10/63 [00:02<00:13, 4.05it/s]
 Training: 17%| | 11/63 [00:02<00:12, 4.06it/s]
 Training: 19%| | 12/63 [00:02<00:12, 4.08it/s]
 Training: 21%| | 13/63 [00:03<00:12, 4.07it/s]
 Training: 22%| | 14/63 [00:03<00:12, 4.06it/s]
 Training: 24%| | 15/63 [00:03<00:11, 4.06it/s]
 Training: 25%| | 16/63 [00:03<00:11, 4.07it/s]
 Training: 27%| | 17/63 [00:04<00:11, 4.08it/s]
 Training: 29%| | 18/63 [00:04<00:11, 4.07it/s]
 Training: 30%| | 19/63 [00:04<00:10, 4.09it/s]
 Training: 32%| | 20/63 [00:04<00:10, 4.05it/s]
 Training: 33%| | 21/63 [00:05<00:10, 4.04it/s]
 Training: 35%| | 22/63 [00:05<00:10, 4.03it/s]
 Training: 37%| | 23/63 [00:05<00:09, 4.05it/s]
 Training: 38%| | 24/63 [00:05<00:09, 4.04it/s]
 Training: 40%| | 25/63 [00:06<00:09, 3.94it/s]
 Training: 41%| | 26/63 [00:06<00:09, 3.92it/s]
 Training: 43%| | 27/63 [00:06<00:09, 3.94it/s]
 Training: 44%| | 28/63 [00:06<00:08, 3.92it/s]
 Training: 46%| | 29/63 [00:07<00:08, 3.94it/s]
 Training: 48%| | 30/63 [00:07<00:08, 3.97it/s]
 Training: 49%| | 31/63 [00:07<00:08, 3.88it/s]
 Training: 51%| | 32/63 [00:07<00:08, 3.85it/s]
 Training: 52%| | 33/63 [00:08<00:07, 3.84it/s]
 Training: 54%| | 34/63 [00:08<00:07, 3.88it/s]

```

Training: 56%|          | 35/63 [00:08<00:07, 3.94it/s]
Training: 57%|          | 36/63 [00:08<00:06, 3.96it/s]
Training: 59%|          | 37/63 [00:09<00:06, 4.00it/s]
Training: 60%|          | 38/63 [00:09<00:06, 4.03it/s]
Training: 62%|          | 39/63 [00:09<00:05, 4.04it/s]
Training: 63%|          | 40/63 [00:09<00:05, 4.04it/s]
Training: 65%|          | 41/63 [00:10<00:05, 4.03it/s]
Training: 67%|          | 42/63 [00:10<00:05, 3.98it/s]
Training: 68%|          | 43/63 [00:10<00:05, 3.97it/s]
Training: 70%|          | 44/63 [00:10<00:04, 3.96it/s]
Training: 71%|          | 45/63 [00:11<00:04, 3.98it/s]
Training: 73%|          | 46/63 [00:11<00:04, 3.99it/s]
Training: 75%|          | 47/63 [00:11<00:03, 4.02it/s]
Training: 76%|          | 48/63 [00:11<00:03, 4.02it/s]
Training: 78%|          | 49/63 [00:12<00:03, 4.04it/s]
Training: 79%|          | 50/63 [00:12<00:03, 4.06it/s]
Training: 81%|          | 51/63 [00:12<00:02, 4.05it/s]
Training: 83%|          | 52/63 [00:12<00:02, 4.05it/s]
Training: 84%|          | 53/63 [00:13<00:02, 4.08it/s]
Training: 86%|          | 54/63 [00:13<00:02, 4.06it/s]
Training: 87%|          | 55/63 [00:13<00:01, 4.03it/s]
Training: 89%|          | 56/63 [00:13<00:01, 4.03it/s]
Training: 90%|          | 57/63 [00:14<00:01, 4.04it/s]
Training: 92%|          | 58/63 [00:14<00:01, 4.05it/s]
Training: 94%|          | 59/63 [00:14<00:00, 4.05it/s]
Training: 95%|          | 60/63 [00:14<00:00, 4.07it/s]
Training: 97%|          | 61/63 [00:15<00:00, 4.07it/s]
Training: 98%|          | 62/63 [00:15<00:00, 4.08it/s]
Training: 100%|         | 63/63 [00:15<00:00, 4.02it/s]

Validation: 0%|          | 0/7 [00:00<?, ?it/s]
Validation: 29%|         | 2/7 [00:00<00:00, 16.25it/s]
Validation: 57%|         | 4/7 [00:00<00:00, 16.19it/s]
Validation: 100%|        | 7/7 [00:00<00:00, 16.36it/s]
100%|          | 10/10 [02:44<00:00, 16.44s/it]

```

Epoch 10 Results:

```

    Training Loss: 0.0863
    Validation Loss: 1.1888

```

4.17 Question 3n: Visualize the Transformer's Training Performance

Task: Plot the Transformer's training loss and validation loss. You can use the `plot_metrics` function we defined earlier :)

```
[101]: # TODO: Plot the training and validation loss across the epochs
plot_metrics(transformer_train_losses, transformer_val_losses,
             num_epochs=num_epochs)

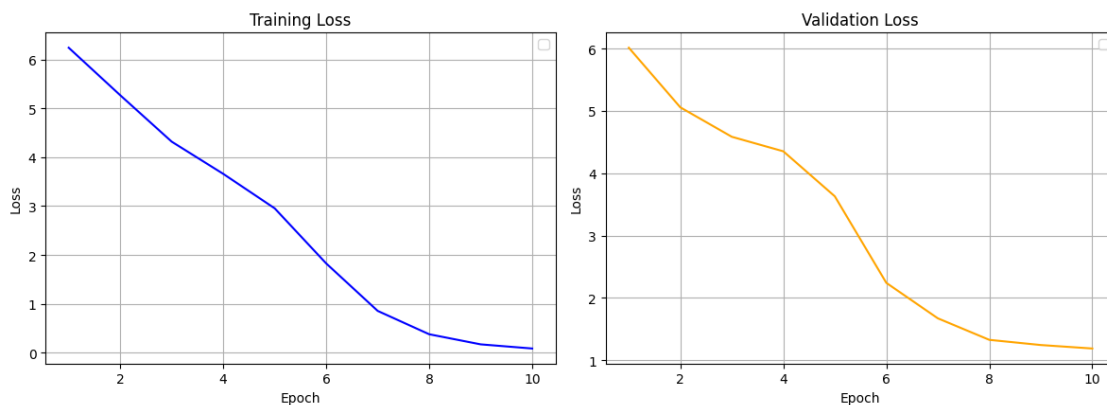
print(f"Final Training Loss: {transformer_train_losses[-1]:.4f}")
print(f"Final Validation Loss: {transformer_val_losses[-1]:.4f}")
```

/tmp/ipython-input-384635747.py:62: UserWarning:

No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

/tmp/ipython-input-384635747.py:70: UserWarning:

No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.



Final Training Loss: 0.0863
Final Validation Loss: 1.1888

4.18 Question 3o: Transformer Inference

Our tiny transformer should now be ready to produce some of its own tiny stories! We’ve defined a function below that can be used to autoregressively generate new tokens from the transformer given a starting prompt. Feel free to play around with the `max_seq_len`, `max_length`, and `prompt` to see what stories you can generate with your transformer! The transformer may not be able to produce fully fluent sentences yet, but hopefully you see some coherence in the new stories it outputs.

Note: Because the transformer’s final output is a vector of class probabilities, we can get the ID of the next token it predicts by applying an `argmax` over the final output vector. To map this token ID back into a word, we can “flip” our dictionary which mapped words to ids into one that maps ids to words. Then, we can add this newly generated word to our tiny story so far, and keep sampling the next token from the transformer, ad infinitum!

```
[102]: def generate_sample(model, word_to_id, id_to_word, max_seq_len=20, prompt="once upon a time", max_length=50):
    """
    Autoregressively generate a sample from the transformer

    Args:
        model (Transformer): The transformer model
        word_to_id (dict): Dictionary mapping words to their token IDs
        id_to_word (dict): Dictionary mapping token IDs to words
        max_seq_len (int): Maximum sequence length used when training the transformer
        prompt (str): The starting prompt for the generated story
        max_length (int): Maximum length of the generated story

    Returns:
        str: The generated story
    """
    model.eval()
    with torch.no_grad():
        tokens = [word_to_id.get(word, 0) for word in prompt.split()]

        for _ in range(max_length):
            if len(tokens) >= max_seq_len: # max sequence length used when training the transformer
                input_seq = torch.tensor([tokens[-1 * max_seq_len:]], dtype=torch.long, device=device)
            else:
                input_seq = torch.tensor([tokens], dtype=torch.long, device=device)

            logits = transformer(input_seq)
            next_token = torch.argmax(logits[0, -1]).item()
            tokens.append(next_token)

            if next_token == word_to_id.get(END_TOKEN, -1):
                break

        return ' '.join([id_to_word.get(token, '<unk>') for token in tokens])

id_to_word = {v: k for k, v in word_to_id.items()}
```

```
[104]: # TODO: Generate your own tiny story!
sample_text = generate_sample(transformer, word_to_id, id_to_word, max_seq_len=20)
print(f"Sample generation: {sample_text}")
```

Sample generation: once upon a time a time time time time time time time

time time time time time time time time time time time time time time time time time
time time time time time time time time time time time time time time time time time
time time time time time time time time time time time

5 Submission Checklist

Please make sure you have completed the following steps before submitting your assignment.

5.1 Have you...

- Saved your notebook as a PDF, ensuring that all the code cells, written questions (if applicable), and graphs are visible?
- Run the export cell in the notebook to generate a zip file containing all the code from this notebook?
- Included an acknowledgement of any GenAI assistance you used while completing this assignment, if applicable? (See the [CS 189/289 syllabus](#) for full details of the course's GenAI policy and for an example acknowledgement format you can use).

5.2 Submissions

- Submitted a PDF version of your notebook to the HW 4.1 Coding [PDF] assignment on Gradescope?
- Submitted a zip file of your code (generated by the export cell) to the HW 4.1 Coding [Code] assignment on Gradescope?

Congratulations! You have completed Homework 4 Part 1!

5.3 Submission

Make sure you have run all cells in your notebook in order before running the cell below, so that all images/graphs appear in the output. The cell below will generate a zip file for you to submit. **Please save before exporting!**

```
[ ]: ## Use this cell if you are running the notebook in Google Colab to install the  
↪necessary dependencies, this may take a few minutes  
if IS_COLAB:  
    !apt-get install -y texlive texlive-xetex pandoc
```

```
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
pandoc is already the newest version (2.9.2.1-3ubuntu2).  
pandoc set to manually installed.  
The following additional packages will be installed:  
  dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono  
  fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java  
  libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12  
  libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0  
  libsynchronet2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzzip-0-13
```

lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration tlutils
teckit tex-common tex-gyre texlive-base texlive-binaries
texlive-fonts-recommended texlive-latex-base texlive-latex-extra
texlive-latex-recommended texlive-pictures texlive-plain-generic tipa
xfonts-encodings xfonts-utils

Suggested packages:

fonts-noto fonts-freefont-otf | fonts-freefont-ttf libavalon-framework-java
libcommons-logging-java-doc libexcalibur-logkit-java liblog4j1.2-java
poppler-utils ghostscript fonts-japanese-mincho | fonts-ipafont-mincho
fonts-japanese-gothic | fonts-ipafont-gothic fonts-arphic-ukai
fonts-arphic-uming fonts-nanum ri ruby-dev bundler debhelper gv
| postscript-viewer perl-tk xpdf | pdf-viewer xzdec
texlive-fonts-recommended-doc texlive-latex-base-doc python3-pygments
icc-profiles libfile-which-perl libspreadsheet-parseexcel-perl
texlive-latex-extra-doc texlive-latex-recommended-doc texlive-luatex
texlive-pstricks dot2tex prerex texlive-pictures-doc vprerex
default-jre-headless tipa-doc

The following NEW packages will be installed:

dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
libsyntax2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13
lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration tlutils
teckit tex-common tex-gyre texlive texlive-base texlive-binaries
texlive-fonts-recommended texlive-latex-base texlive-latex-extra
texlive-latex-recommended texlive-pictures texlive-plain-generic
texlive-xetex tipa xfonts-encodings xfonts-utils

0 upgraded, 54 newly installed, 0 to remove and 41 not upgraded.

Need to get 182 MB of archives.

After this operation, 571 MB of additional disk space will be used.

Get:1 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-droid-fallback all 1:6.0.1r16-1.1build1 [1,805 kB]

Get:2 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-lato all 2.0-2.1 [2,696 kB]

Get:3 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 poppler-data all 0.4.11-1 [2,171 kB]

Get:4 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 tex-common all 6.17 [33.7 kB]

Get:5 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-urw-base35 all 20200910-1 [6,367 kB]

Get:6 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libgs9-common all 9.55.0-0ubuntu5.13 [753 kB]

Get:7 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libidn12 amd64 1.38-4ubuntu1 [60.0 kB]

Get:8 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libijs-0.35 amd64

0.35-15build2 [16.5 kB]
 Get:9 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libjbig2dec0 amd64
 0.19-3build2 [64.7 kB]
 Get:10 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libgs9 amd64
 9.55.0~dfsg1-0ubuntu5.13 [5,032 kB]
 Get:11 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libkpathsea6
 amd64 2021.20210626.59705-1ubuntu0.2 [60.4 kB]
 Get:12 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libwoff1 amd64
 1.0.2-1build4 [45.2 kB]
 Get:13 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 dvisvgm amd64
 2.13.1-1 [1,221 kB]
 Get:14 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 fonts-lmodern all
 2.004.5-6.1 [4,532 kB]
 Get:15 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-noto-mono all
 20201225-1build1 [397 kB]
 Get:16 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 fonts-texgyre all
 20180621-3.1 [10.2 MB]
 Get:17 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libapache-pom-java
 all 18-1 [4,720 B]
 Get:18 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libcommons-parent-
 java all 43-1 [10.8 kB]
 Get:19 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libcommons-logging-
 java all 1.2-2 [60.3 kB]
 Get:20 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libptexenc1
 amd64 2021.20210626.59705-1ubuntu0.2 [39.1 kB]
 Get:21 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rubygems-integration
 all 1.18 [5,336 B]
 Get:22 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby3.0 amd64
 3.0.2-7ubuntu2.11 [50.1 kB]
 Get:23 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-rubygems
 all 3.3.5-2ubuntu1.2 [228 kB]
 Get:24 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby amd64 1:3.0~exp1
 [5,100 B]
 Get:25 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rake all 13.0.6-2 [61.7
 kB]
 Get:26 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby-net-telnet all
 0.1.1-2 [12.6 kB]
 Get:27 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-webrick
 all 1.7.0-3ubuntu0.2 [52.5 kB]
 Get:28 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-xlrrpc all
 0.3.2-1ubuntu0.1 [24.9 kB]
 Get:29 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libruby3.0
 amd64 3.0.2-7ubuntu2.11 [5,114 kB]
 Get:30 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libsynchronet2
 amd64 2021.20210626.59705-1ubuntu0.2 [55.6 kB]
 Get:31 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libteckit0 amd64
 2.5.11+ds1-1 [421 kB]
 Get:32 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexlua53

```

amd64 2021.20210626.59705-1ubuntu0.2 [120 kB]
Get:33 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libtexluaajit2
amd64 2021.20210626.59705-1ubuntu0.2 [267 kB]
Get:34 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libzzip-0-13 amd64
0.13.72+dfsg.1-1.1 [27.0 kB]
Get:35 http://archive.ubuntu.com/ubuntu jammy/main amd64 xfonts-encodings all
1:1.0.5-0ubuntu2 [578 kB]
Get:36 http://archive.ubuntu.com/ubuntu jammy/main amd64 xfonts-utils amd64
1:7.7+6build2 [94.6 kB]
Get:37 http://archive.ubuntu.com/ubuntu jammy/universe amd64 lmodern all
2.004.5-6.1 [9,471 kB]
Get:38 http://archive.ubuntu.com/ubuntu jammy/universe amd64 preview-latex-style
all 12.2-1ubuntu1 [185 kB]
Get:39 http://archive.ubuntu.com/ubuntu jammy/main amd64 t1utils amd64
1.41-4build2 [61.3 kB]
Get:40 http://archive.ubuntu.com/ubuntu jammy/universe amd64 teckit amd64
2.5.11+ds1-1 [699 kB]
Get:41 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tex-gyre all
20180621-3.1 [6,209 kB]
Get:42 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 texlive-
binaries amd64 2021.20210626.59705-1ubuntu0.2 [9,860 kB]
Get:43 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-base all
2021.20220204-1 [21.0 MB]
Get:44 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-fonts-
recommended all 2021.20220204-1 [4,972 kB]
Get:45 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-base
all 2021.20220204-1 [1,128 kB]
Get:46 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-
recommended all 2021.20220204-1 [14.4 MB]
Get:47 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive all
2021.20220204-1 [14.3 kB]
Get:48 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libfontbox-java all
1:1.8.16-2 [207 kB]
Get:49 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libpdfbox-java all
1:1.8.16-2 [5,199 kB]
Get:50 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-pictures
all 2021.20220204-1 [8,720 kB]
Get:51 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-extra
all 2021.20220204-1 [13.9 MB]
Get:52 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-plain-
generic all 2021.20220204-1 [27.5 MB]
Get:53 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tipa all 2:1.3-21
[2,967 kB]
Get:54 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-xetex all
2021.20220204-1 [12.4 MB]
Fetched 182 MB in 8s (24.0 MB/s)
Extracting templates from packages: 100%
Preconfiguring packages ...

```

```

Selecting previously unselected package fonts-droid-fallback.
(Reading database ... 121713 files and directories currently installed.)
Preparing to unpack .../00-fonts-droid-fallback_1%3a6.0.1r16-1.1build1_all.deb
...
Unpacking fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Selecting previously unselected package fonts-lato.
Preparing to unpack .../01-fonts-lato_2.0-2.1_all.deb ...
Unpacking fonts-lato (2.0-2.1) ...
Selecting previously unselected package poppler-data.
Preparing to unpack .../02-poppler-data_0.4.11-1_all.deb ...
Unpacking poppler-data (0.4.11-1) ...
Selecting previously unselected package tex-common.
Preparing to unpack .../03-tex-common_6.17_all.deb ...
Unpacking tex-common (6.17) ...
Selecting previously unselected package fonts-urw-base35.
Preparing to unpack .../04-fonts-urw-base35_20200910-1_all.deb ...
Unpacking fonts-urw-base35 (20200910-1) ...
Selecting previously unselected package libgs9-common.
Preparing to unpack .../05-libgs9-common_9.55.0~dfsg1-0ubuntu5.13_all.deb ...
Unpacking libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libidn12:amd64.
Preparing to unpack .../06-libidn12_1.38-4ubuntu1_amd64.deb ...
Unpacking libidn12:amd64 (1.38-4ubuntu1) ...
Selecting previously unselected package libijs-0.35:amd64.
Preparing to unpack .../07-libijs-0.35_0.35-15build2_amd64.deb ...
Unpacking libijs-0.35:amd64 (0.35-15build2) ...
Selecting previously unselected package libjbig2dec0:amd64.
Preparing to unpack .../08-libjbig2dec0_0.19-3build2_amd64.deb ...
Unpacking libjbig2dec0:amd64 (0.19-3build2) ...
Selecting previously unselected package libgs9:amd64.
Preparing to unpack .../09-libgs9_9.55.0~dfsg1-0ubuntu5.13_amd64.deb ...
Unpacking libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libkpathsea6:amd64.
Preparing to unpack .../10-libkpathsea6_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libwoff1:amd64.
Preparing to unpack .../11-libwoff1_1.0.2-1build4_amd64.deb ...
Unpacking libwoff1:amd64 (1.0.2-1build4) ...
Selecting previously unselected package dvisvgm.
Preparing to unpack .../12-dvisvgm_2.13.1-1_amd64.deb ...
Unpacking dvisvgm (2.13.1-1) ...
Selecting previously unselected package fonts-lmodern.
Preparing to unpack .../13-fonts-lmodern_2.004.5-6.1_all.deb ...
Unpacking fonts-lmodern (2.004.5-6.1) ...
Selecting previously unselected package fonts-noto-mono.
Preparing to unpack .../14-fonts-noto-mono_20201225-1build1_all.deb ...
Unpacking fonts-noto-mono (20201225-1build1) ...

```

```

Selecting previously unselected package fonts-texgyre.
Preparing to unpack .../15-fonts-texgyre_20180621-3.1_all.deb ...
Unpacking fonts-texgyre (20180621-3.1) ...
Selecting previously unselected package libapache-pom-java.
Preparing to unpack .../16-libapache-pom-java_18-1_all.deb ...
Unpacking libapache-pom-java (18-1) ...
Selecting previously unselected package libcommons-parent-java.
Preparing to unpack .../17-libcommons-parent-java_43-1_all.deb ...
Unpacking libcommons-parent-java (43-1) ...
Selecting previously unselected package libcommons-logging-java.
Preparing to unpack .../18-libcommons-logging-java_1.2-2_all.deb ...
Unpacking libcommons-logging-java (1.2-2) ...
Selecting previously unselected package libptexenc1:amd64.
Preparing to unpack .../19-libptexenc1_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package rubygems-integration.
Preparing to unpack .../20-rubygems-integration_1.18_all.deb ...
Unpacking rubygems-integration (1.18) ...
Selecting previously unselected package ruby3.0.
Preparing to unpack .../21-ruby3.0_3.0.2-7ubuntu2.11_amd64.deb ...
Unpacking ruby3.0 (3.0.2-7ubuntu2.11) ...
Selecting previously unselected package ruby-rubygems.
Preparing to unpack .../22-ruby-rubygems_3.3.5-2ubuntu1.2_all.deb ...
Unpacking ruby-rubygems (3.3.5-2ubuntu1.2) ...
Selecting previously unselected package ruby.
Preparing to unpack .../23-ruby_1%3a3.0~exp1_amd64.deb ...
Unpacking ruby (1:3.0~exp1) ...
Selecting previously unselected package rake.
Preparing to unpack .../24-rake_13.0.6-2_all.deb ...
Unpacking rake (13.0.6-2) ...
Selecting previously unselected package ruby-net-telnet.
Preparing to unpack .../25-ruby-net-telnet_0.1.1-2_all.deb ...
Unpacking ruby-net-telnet (0.1.1-2) ...
Selecting previously unselected package ruby-webrick.
Preparing to unpack .../26-ruby-webrick_1.7.0-3ubuntu0.2_all.deb ...
Unpacking ruby-webrick (1.7.0-3ubuntu0.2) ...
Selecting previously unselected package ruby-xmlrpc.
Preparing to unpack .../27-ruby-xmlrpc_0.3.2-1ubuntu0.1_all.deb ...
Unpacking ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Selecting previously unselected package libruby3.0:amd64.
Preparing to unpack .../28-libruby3.0_3.0.2-7ubuntu2.11_amd64.deb ...
Unpacking libruby3.0:amd64 (3.0.2-7ubuntu2.11) ...
Selecting previously unselected package libsyntax2:amd64.
Preparing to unpack .../29-libsyntax2_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libsyntax2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libteckit0:amd64.

```

```

Preparing to unpack .../30-libteckit0_2.5.11+ds1-1_amd64.deb ...
Unpacking libteckit0:amd64 (2.5.11+ds1-1) ...
Selecting previously unselected package libtexlua53:amd64.
Preparing to unpack .../31-libtexlua53_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libtexluajit2:amd64.
Preparing to unpack
.../32-libtexluajit2_2021.20210626.59705-1ubuntu0.2_amd64.deb ...
Unpacking libtexluajit2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libzip-0-13:amd64.
Preparing to unpack .../33-libzip-0-13_0.13.72+dfsg.1-1.1_amd64.deb ...
Unpacking libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Selecting previously unselected package xfonts-encodings.
Preparing to unpack .../34-xfonts-encodings_1%3a1.0.5-0ubuntu2_all.deb ...
Unpacking xfonts-encodings (1:1.0.5-0ubuntu2) ...
Selecting previously unselected package xfonts-utils.
Preparing to unpack .../35-xfonts-utils_1%3a7.7+6build2_amd64.deb ...
Unpacking xfonts-utils (1:7.7+6build2) ...
Selecting previously unselected package lmodern.
Preparing to unpack .../36-lmodern_2.004.5-6.1_all.deb ...
Unpacking lmodern (2.004.5-6.1) ...
Selecting previously unselected package preview-latex-style.
Preparing to unpack .../37-preview-latex-style_12.2-1ubuntu1_all.deb ...
Unpacking preview-latex-style (12.2-1ubuntu1) ...
Selecting previously unselected package t1utils.
Preparing to unpack .../38-t1utils_1.41-4build2_amd64.deb ...
Unpacking t1utils (1.41-4build2) ...
Selecting previously unselected package teckit.
Preparing to unpack .../39-teckit_2.5.11+ds1-1_amd64.deb ...
Unpacking teckit (2.5.11+ds1-1) ...
Selecting previously unselected package tex-gyre.
Preparing to unpack .../40-tex-gyre_20180621-3.1_all.deb ...
Unpacking tex-gyre (20180621-3.1) ...
Selecting previously unselected package texlive-binaries.
Preparing to unpack .../41-texlive-
binaries_2021.20210626.59705-1ubuntu0.2_amd64.deb ...
Unpacking texlive-binaries (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package texlive-base.
Preparing to unpack .../42-texlive-base_2021.20220204-1_all.deb ...
Unpacking texlive-base (2021.20220204-1) ...
Selecting previously unselected package texlive-fonts-recommended.
Preparing to unpack .../43-texlive-fonts-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-fonts-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-base.
Preparing to unpack .../44-texlive-latex-base_2021.20220204-1_all.deb ...
Unpacking texlive-latex-base (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-recommended.

```

```
Preparing to unpack .../45-texlive-latex-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-latex-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive.
Preparing to unpack .../46-texlive_2021.20220204-1_all.deb ...
Unpacking texlive (2021.20220204-1) ...
Selecting previously unselected package libfontbox-java.
Preparing to unpack .../47-libfontbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libfontbox-java (1:1.8.16-2) ...
Selecting previously unselected package libpdfbox-java.
Preparing to unpack .../48-libpdfbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libpdfbox-java (1:1.8.16-2) ...
Selecting previously unselected package texlive-pictures.
Preparing to unpack .../49-texlive-pictures_2021.20220204-1_all.deb ...
Unpacking texlive-pictures (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-extra.
Preparing to unpack .../50-texlive-latex-extra_2021.20220204-1_all.deb ...
Unpacking texlive-latex-extra (2021.20220204-1) ...
```

```
[ ]: # Save your notebook first, then run this cell to export your submission.
      grader.export(pdf=False)
```